

# Continuous Mathematical Methods, Emphasis on Machine Learning

Ron Fedkiw<sup>\*1</sup>, Yilin Zhu<sup>\*23</sup>, Winnie Lin<sup>\*3</sup>, Jane Wu<sup>\*3</sup>, Kevin Li<sup>\*3</sup>

<sup>\*</sup> Stanford University, <sup>1</sup> Slide Design and Content (and Instructor), <sup>2</sup> Slide Illustrator, <sup>3</sup> Teaching Assistant



# Table of Contents

- Unit 1: Introduction
- Unit 2: Linear Systems
- Unit 3: Understanding Matrices
- Unit 4: Special Matrices
- Unit 5: Iterative Solvers
- Unit 6: Local Approximations
- Unit 7: Curse of Dimensionality
- Unit 8: Least Squares
- Unit 9: Basic Optimization
- Unit 10: Solving Least Squares
- Unit 11: Zero Singular Values
- Unit 12: Regularization
- Unit 13: Optimization
- Unit 14: Nonlinear Systems
- Unit 15: 1D Root Finding
- Unit 16: 1D Optimization
- Unit 17: Computing Derivatives
- Unit 18: Avoiding Derivatives
- Unit 19: Descent Methods
- Unit 20: Momentum Methods
- Appendix: Notation

# Unit 1



What is Learning?

# What is Learning?

- There are lots of answers to this question, and explanations often become philosophical
- A more practical question might be:

What can we teach/train a person, animal, or machine to do?

# Example: Addition “+”

- How is addition taught in schools?
  - Memorize rules for pairs of numbers from the set  $\{0,1,2,3,4,5,6,7,8,9\}$
  - Memorize redundant rules collectively, for efficiency, e.g.  $0+x=x$
  - Learn to treat powers of 10 implicitly, e.g.  $12+34=46$  since  $1+3=4$  and  $2+4=6$
  - Learn to carry, when the sum of two numbers is larger than 9
  - Learn to add larger sets of numbers, by considering them one pair at a time
  - Learn how to treat negative numbers
  - Learn how to treat decimals and fractions
  - Learn how to treat irrational numbers

# Knowledge Based Systems (KBS)

Contains two parts:

## 1) Knowledge Base

- Explicit knowledge or facts
- Often populated by an expert (expert systems)

## 2) Inference Engine

- Way of reasoning about the facts in order to generate new facts
- Typically follows the rules of Mathematical Logic



# KBS Approach to Addition

- Rule:  $x$  and  $y$  commute
- Start with  $x$  and  $y$  as single digits, and record all  $x + y$  outcomes as facts
- Add rules to deal with multi-digit numbers by pulling out powers of 10
- Add rules for negative numbers, decimals, fractions, irrationals, etc.
- Mimics human learning (or at least human **teaching**)
- This is a discrete approach, and it has no inherent error



# Machine Learning (ML)

Contains two parts:

## 1) Training Data

- Data Points - typically as domain/range pairs
- Hand labeled by a user, measured from the environment, generated procedurally, etc.

## 2) Model

- Derived from the Training Data in order to estimate new data points with (hopefully) minimal error
- Uses Algorithms, Statistical Reasoning, Rules, Networks, Etc.

# KBS vs. ML

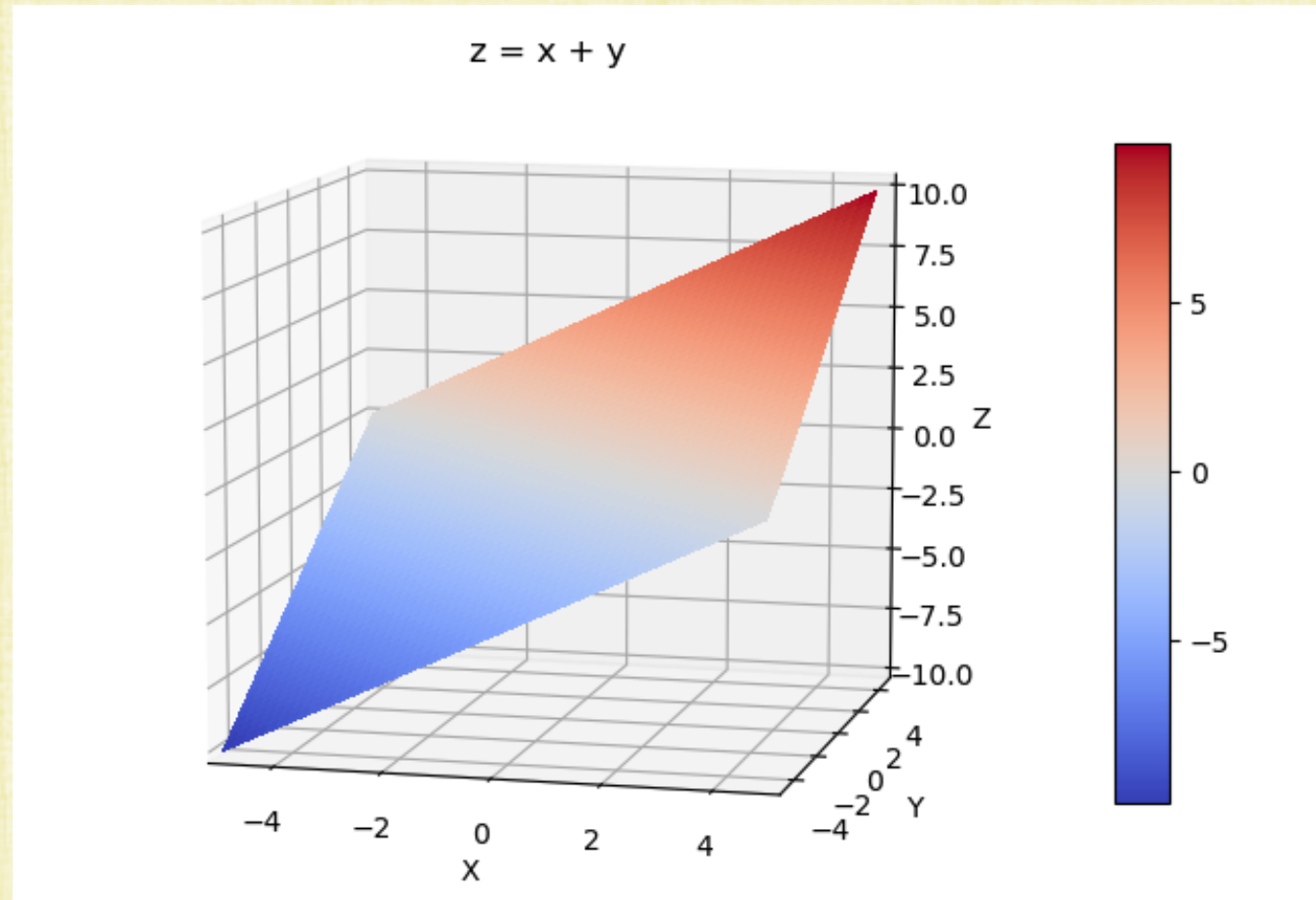
- KBS and ML can be seen as the discrete math and continuous math approaches (respectively) to the same problem
- KBS's Knowledge Base serves the same role as ML's Training Data
- Logic is the algorithm used to discover new discrete facts for KBS, whereas many numerical algorithms/methods are used to approximate continuous facts/data for ML
  - Logic (in particular) happens to be especially useful for discrete facts
  - Numerical algorithms are especially usefully for continuous approximations
- Since ML is derived from continuous math, it tends to have inherent approximation errors

# ML Approach to Addition

- Make a  $2D$  domain in  $R^2$  and a  $1D$  range in  $R^1$ , for the addition function
- As training data, choose a number of input points  $(x_i, y_i)$  with output  $x_i + y_i$
- Plot the 3D points  $(x_i, y_i, x_i + y_i)$  and determine a model function  $z = f(x, y)$  that best approximates the training data
- The plane  $z = x + y$  exactly fits the training data
  - Only need 3 training points to determine a plane
- Don't need special rules for negative numbers, decimals, fractions, irrationals, etc.
- However, small errors in the training data lead to a slightly incorrect plane, which has quite large errors far away from the training data
- This can be alleviated to some degree by adding training data where one wants smaller errors (and computing the best fitting plane to all the training data)



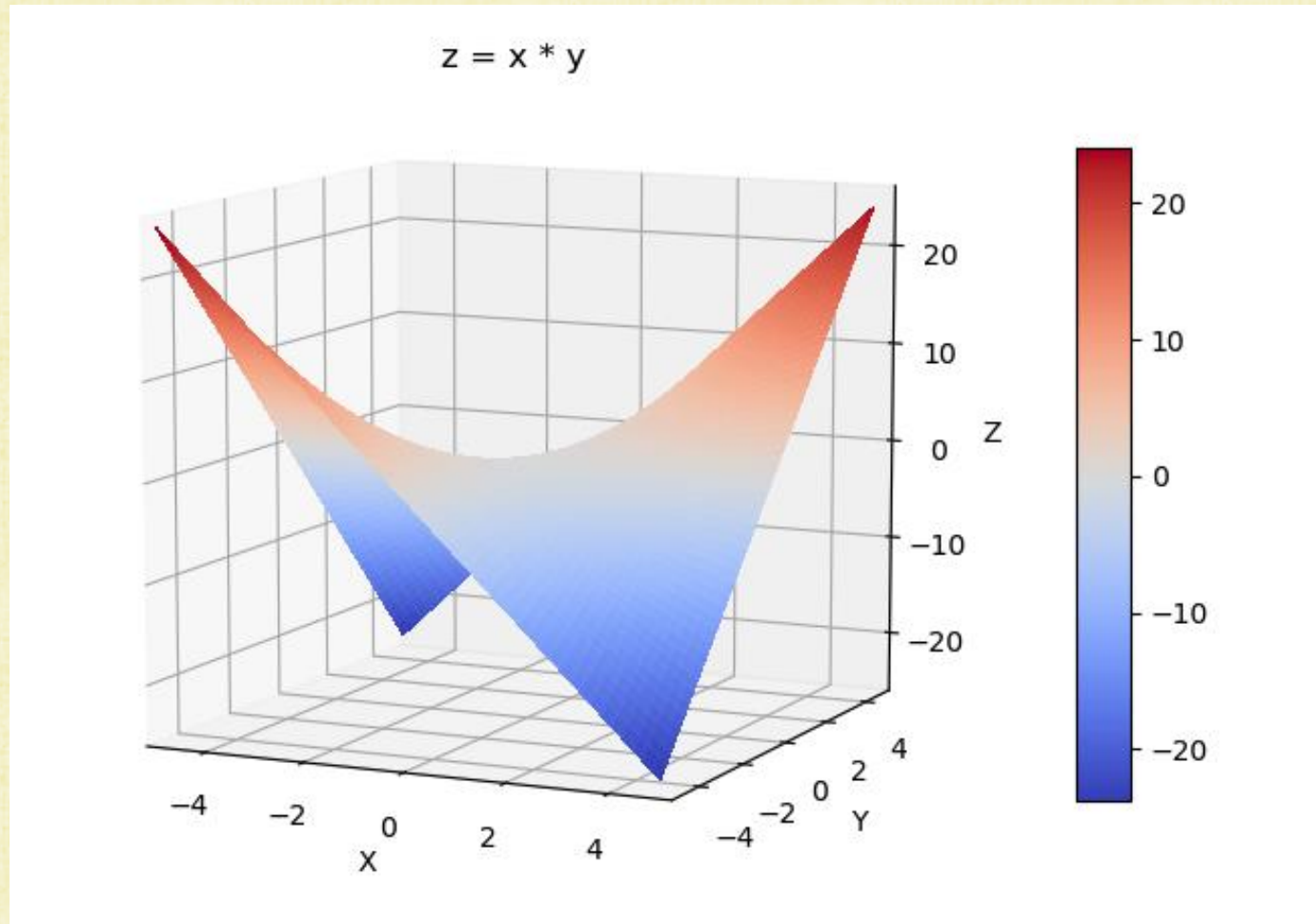
# ML Approach to Addition



# Example: Multiplication “\*”

- KBS creates new rules for  $x * y$ , utilizing the rules from addition too
- ML utilizes a set of 3D points  $(x_i, y_i, x_i * y_i)$  as training data, and the nonlinear model function  $z = x * y$  can be found to fit the training data
  - Note: one may claim that it is “cheating” to use an inherently represented floating point operation (i.e., multiplication) as the model

# ML Approach to Multiplication





# Example: Unknown Operation “#”

- KBS fails!
- How can KBS create rules for  $x\#y$  when we don't even know what # means?
- This is the case for many real-world phenomena that are not fully understood
- However, sometimes it is possible to get some examples of  $x\#y$ 
  - That is, through experimentation or expert knowledge, one might be able to discover  $z_i = x_i\#y_i$  for some  $(x_i, y_i)$  pairs
- Subsequently, these known (or estimated) 3D points  $(x_i, y_i, z_i)$  can be used as training data to determine a model function  $z = f(x, y)$  that approximately fits the data

# Determining the Model Function

- How does one determine  $z = f(x, y)$  near the training data, so that it robustly predicts/infers  $\hat{z}$  for new inputs  $(\hat{x}, \hat{y})$  not contained in the training data?
- How does one minimize the effect of inaccuracies or noise in the training data?
- Caution: away from the training data, the model function  $f$  is likely to be highly inaccurate (**extrapolation is ill-posed**)

# Nearest Neighbor

- If asked to multiply 51.023 times 298.5, one might quickly estimate that 50 times 300 is 15,000
- This is a nearest neighbor algorithm, relying on nearby data where the answer is known, better known, or easier to determine
- Given  $(\hat{x}, \hat{y})$ , find the closest (Euclidean distance) training data  $(x_i, y_i)$  and return its associated  $z_i$  (with error  $\|z_i - \hat{z}\|$ )
- This represents  $z = f(x, y)$  as a piecewise constant function with discontinuities on the boundaries of Voronoi regions around the training data
- This is the simplest possible Machine Learning algorithm (a piecewise constant function), and it works in an arbitrary number of dimensions

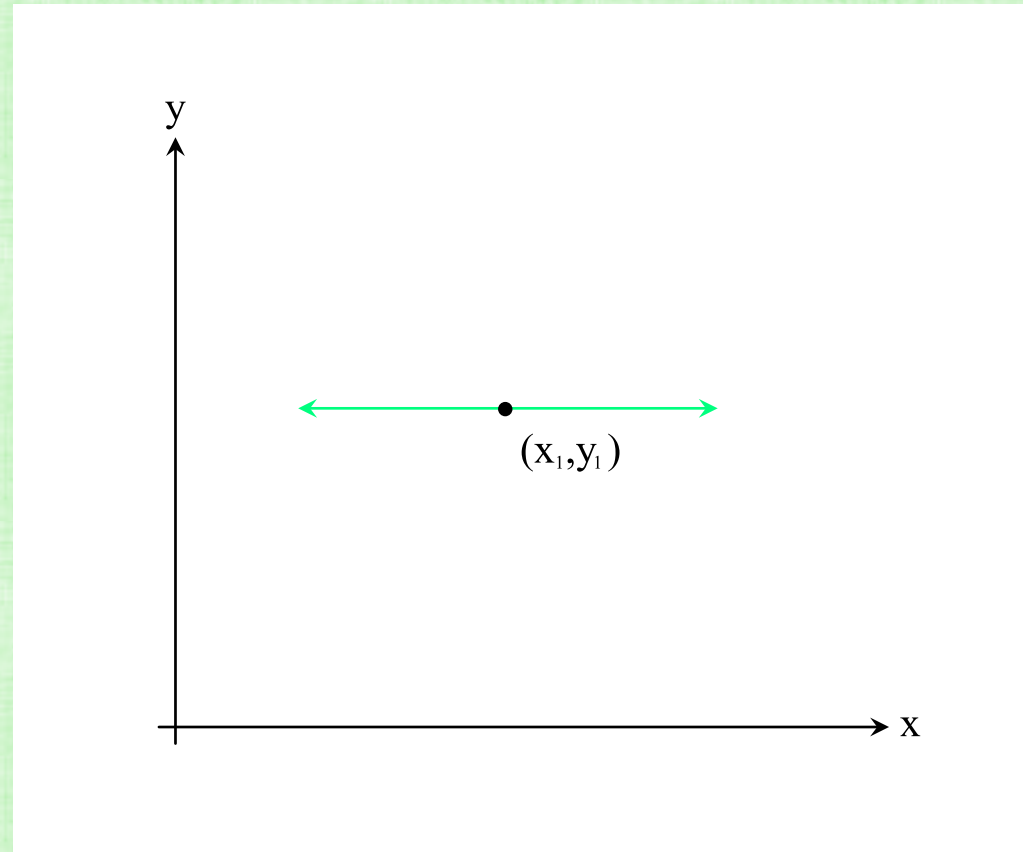


# Data Interpolation

- In order to elucidate various concepts, let's consider data interpolation in more detail
- Let's begin with a very simple case with  $1D$  inputs and  $1D$  outputs, i.e.  $y = f(x)$

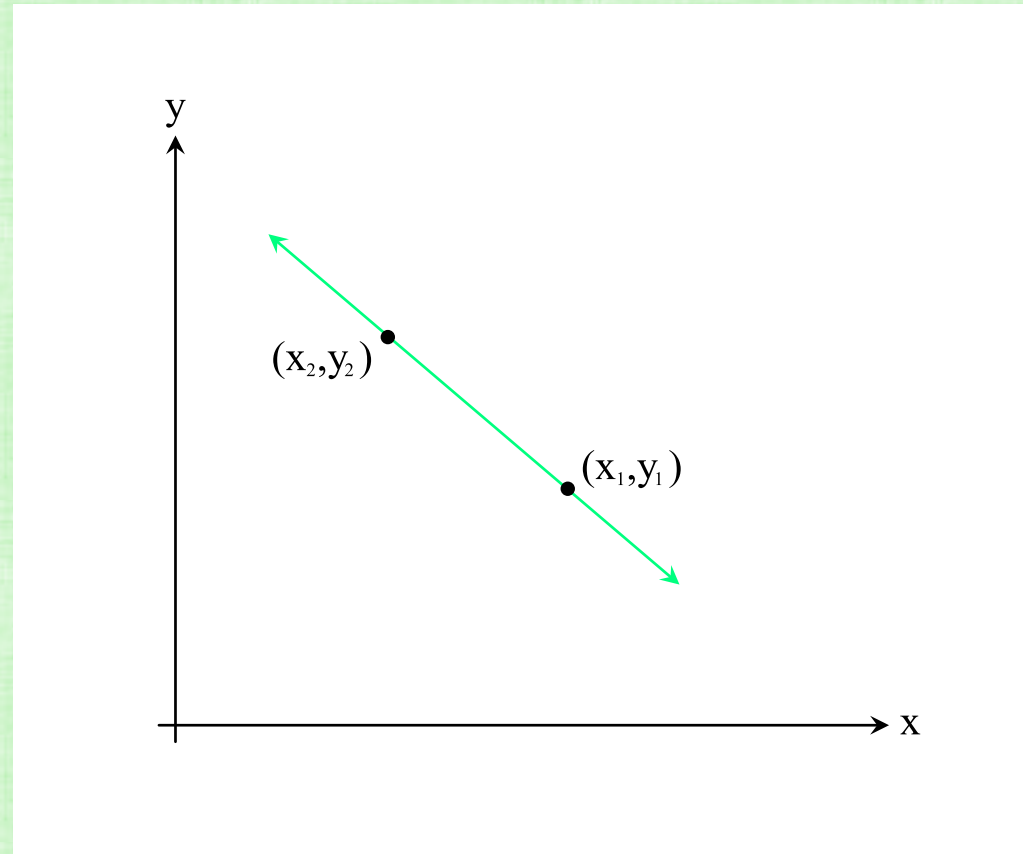
# Polynomial Interpolation

- Given 1 data point, one can (at best) draw a constant function



# Polynomial Interpolation

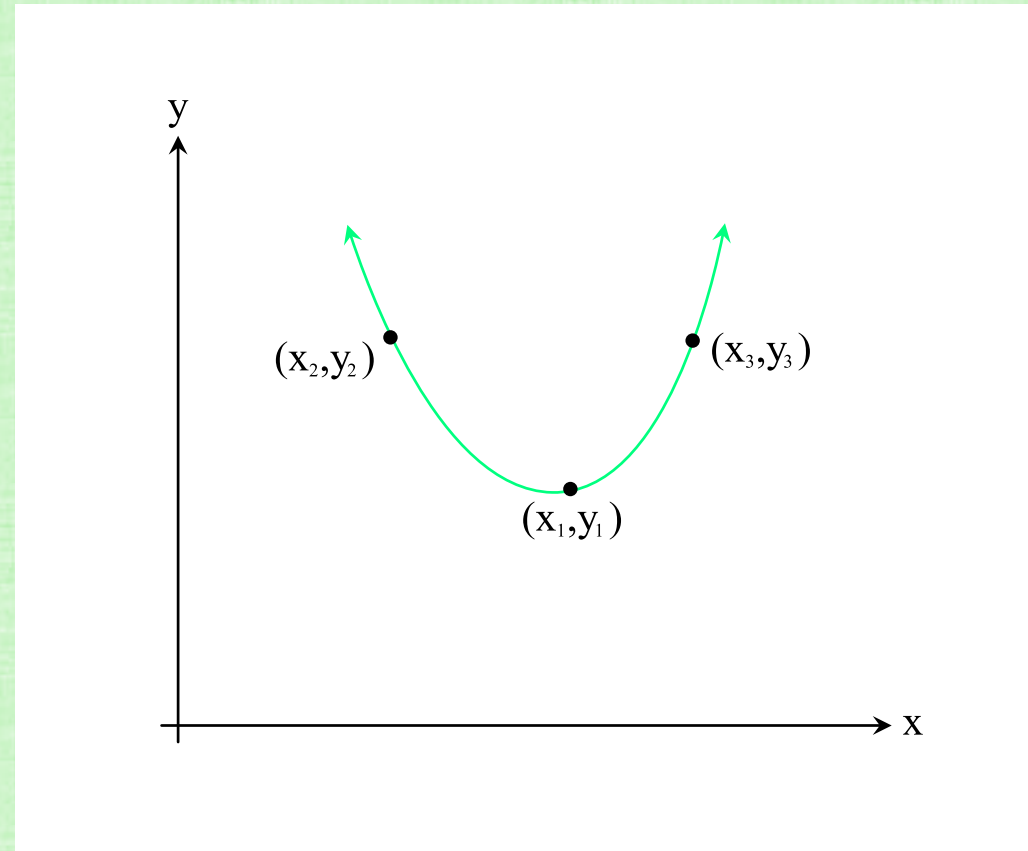
- Given 2 data points, one can (at best) draw a linear function





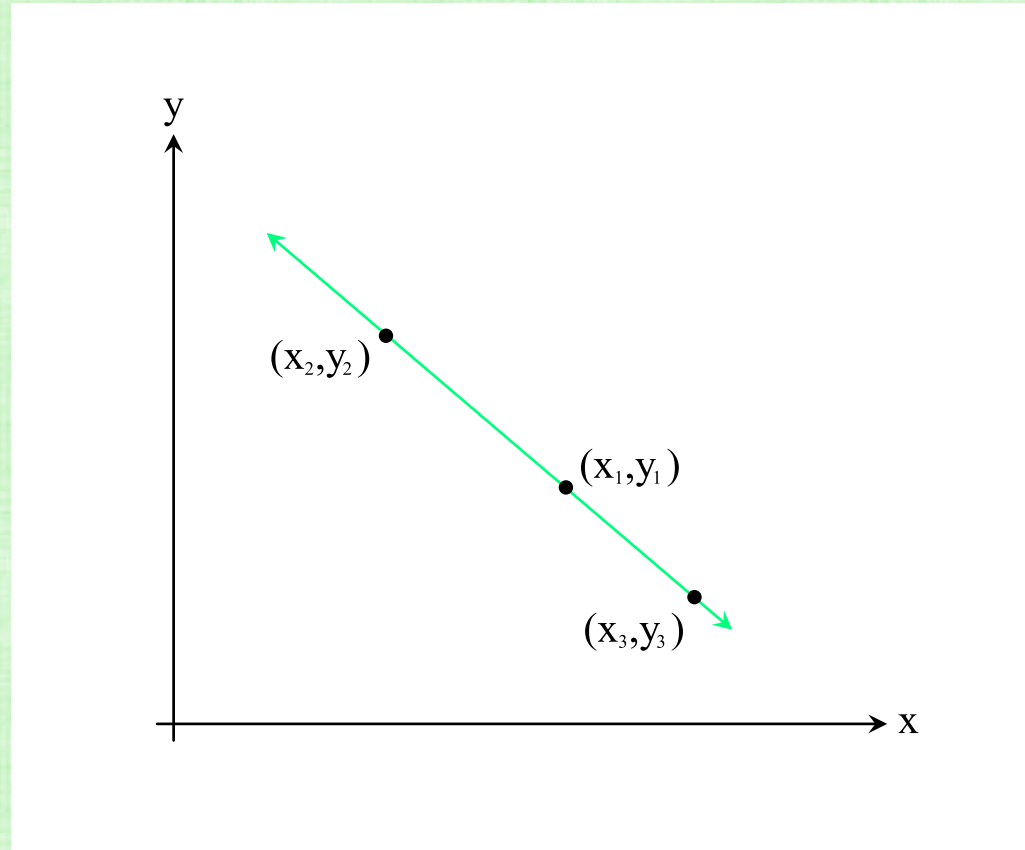
# Polynomial Interpolation

- Given 3 data points, one can (at best) draw a quadratic function



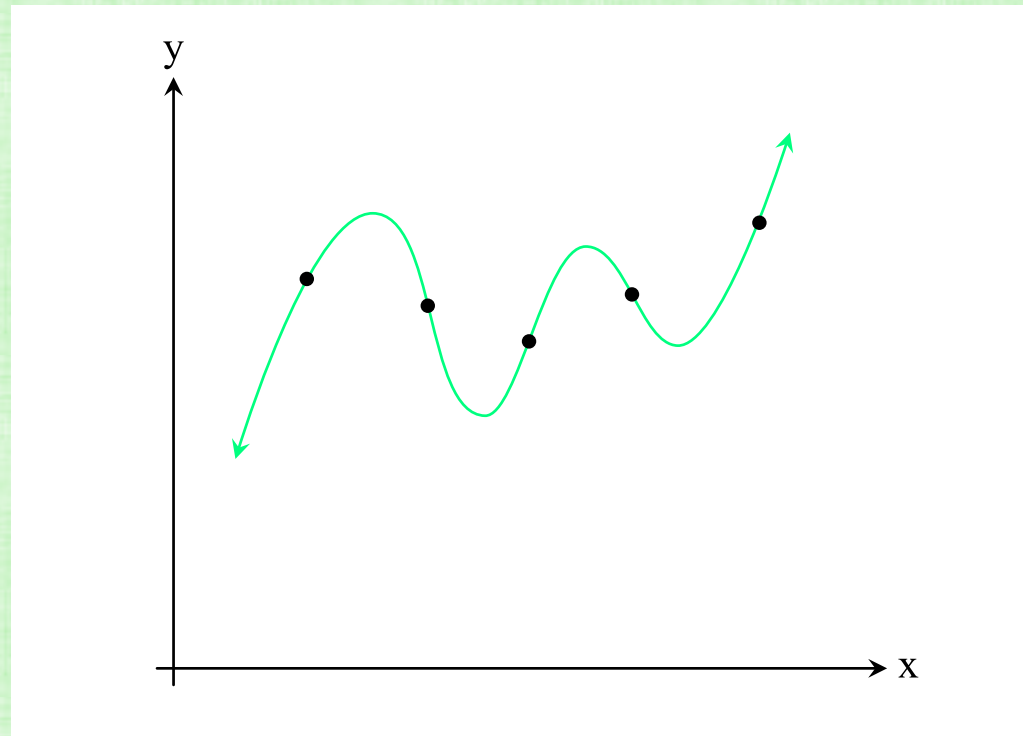
# Polynomial Interpolation

- Unless all 3 points are on the same line, in which case one can only draw a linear function



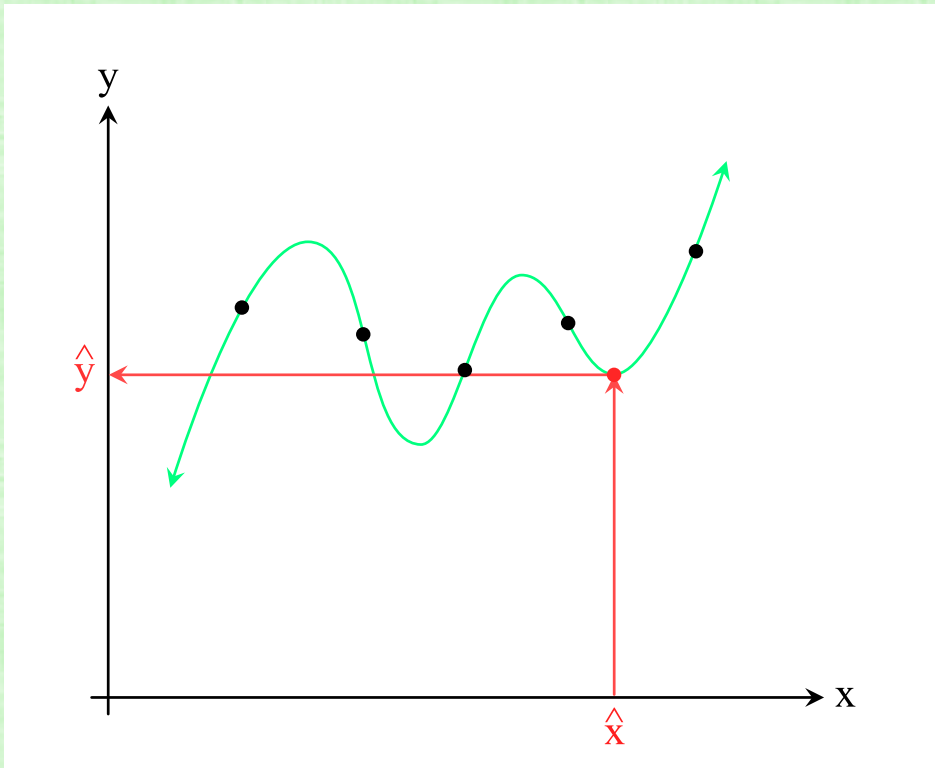
# Polynomial Interpolation

- Given  $m$  data points, one can (at best) draw a unique  $m - 1$  degree polynomial that goes through all of them
  - As long as they're not degenerate (e.g. 3 points on a line)



# Overfitting

- Given a new input  $\hat{x}$ , the interpolating polynomial infers/predicts an output  $\hat{y}$  that may be far from what one may expect

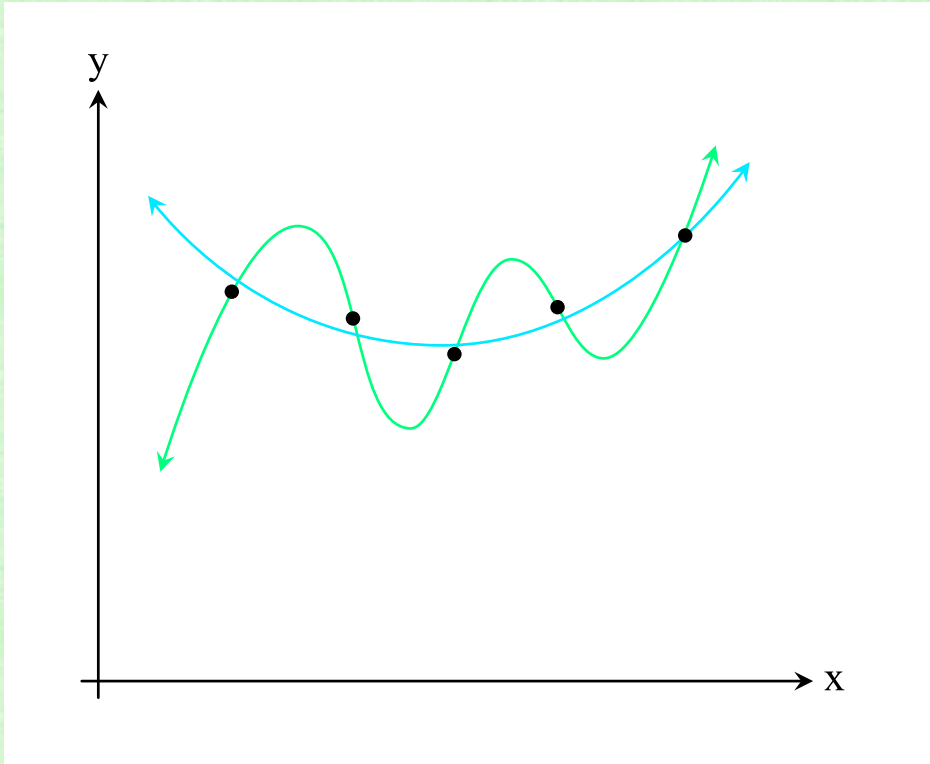


- Interpolating polynomials are smooth (continuous function and derivatives)
- Thus, they wiggle/overshoot in between data points (so that they can smoothly turn back and hit the next point)
- Overly forcing polynomials to exactly hit every data point is called overfitting (overly fitting to the data)
- Overfitting leads to inference/predictions that can vary wildly from the training data



# Regularization

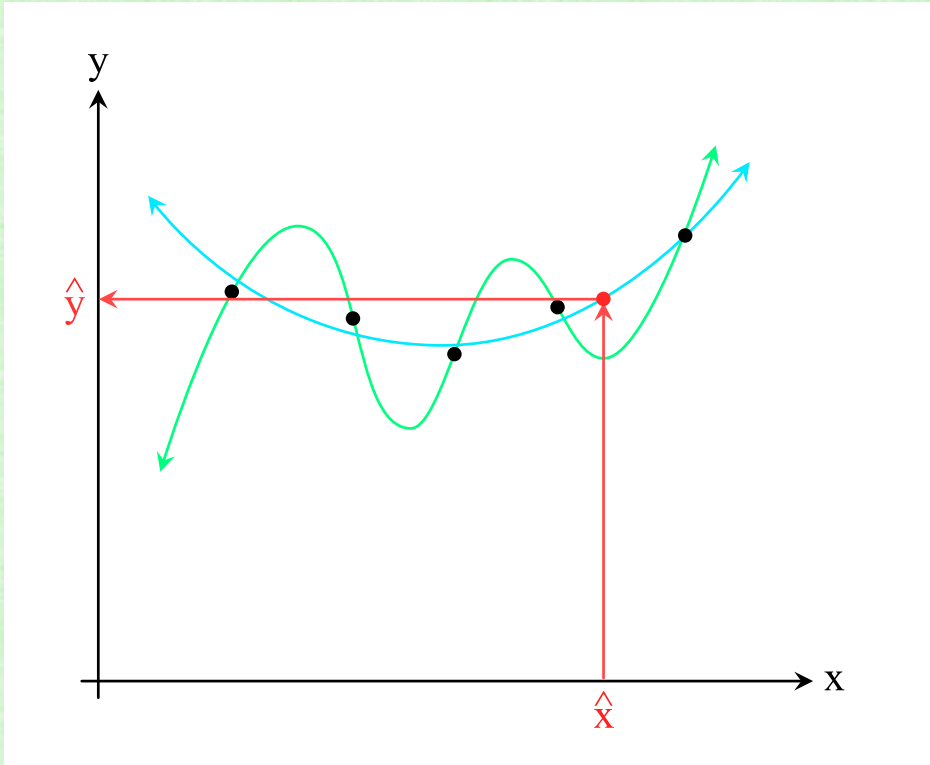
- Using a lower order polynomial that doesn't (can't) exactly fit the data points provides regularization



- A regularized interpolant contains intentional error, missing some/all of the data points
- However, this hopefully makes the function more predictable/smooth in between the data points
- The data points themselves may contain noise/error, so it is not clear whether they should be interpolated exactly anyways

# Regularization

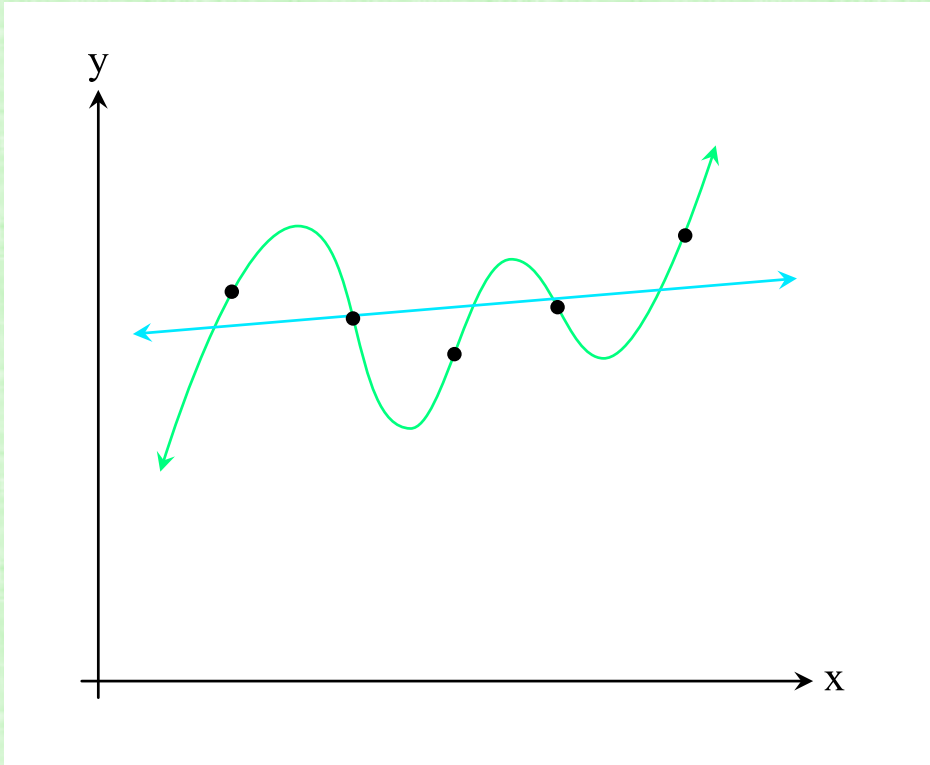
- Given  $\hat{x}$ , the regularized interpolant infers/predicts a more reasonable  $\hat{y}$



- There is a trade-off between sacrificing accuracy on fitting the original input data, and obtaining better accuracy on inference/prediction for new inputs

# Underfitting

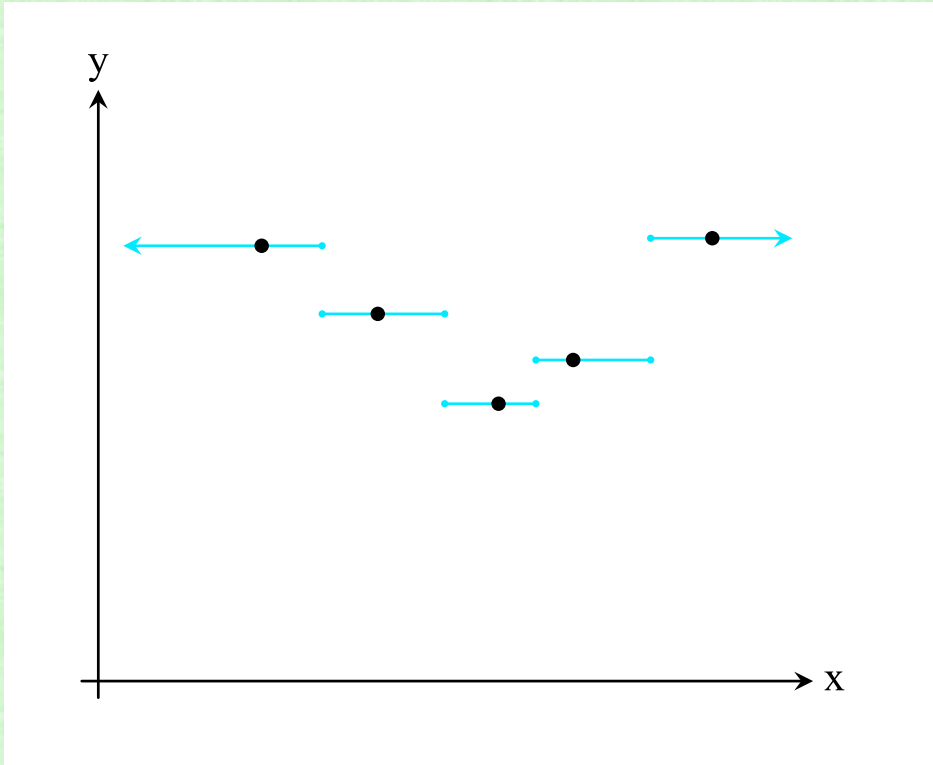
- Using too low of an order polynomial causes it to miss the data by too much



- A linear function doesn't capture the essence of this data as well as a quadratic function does
- Choosing too simple of a model function or regularizing too much prevents one from properly representing the data

# Nearest Neighbor

- Piecewise-constant interpolation (equivalent to nearest neighbor)

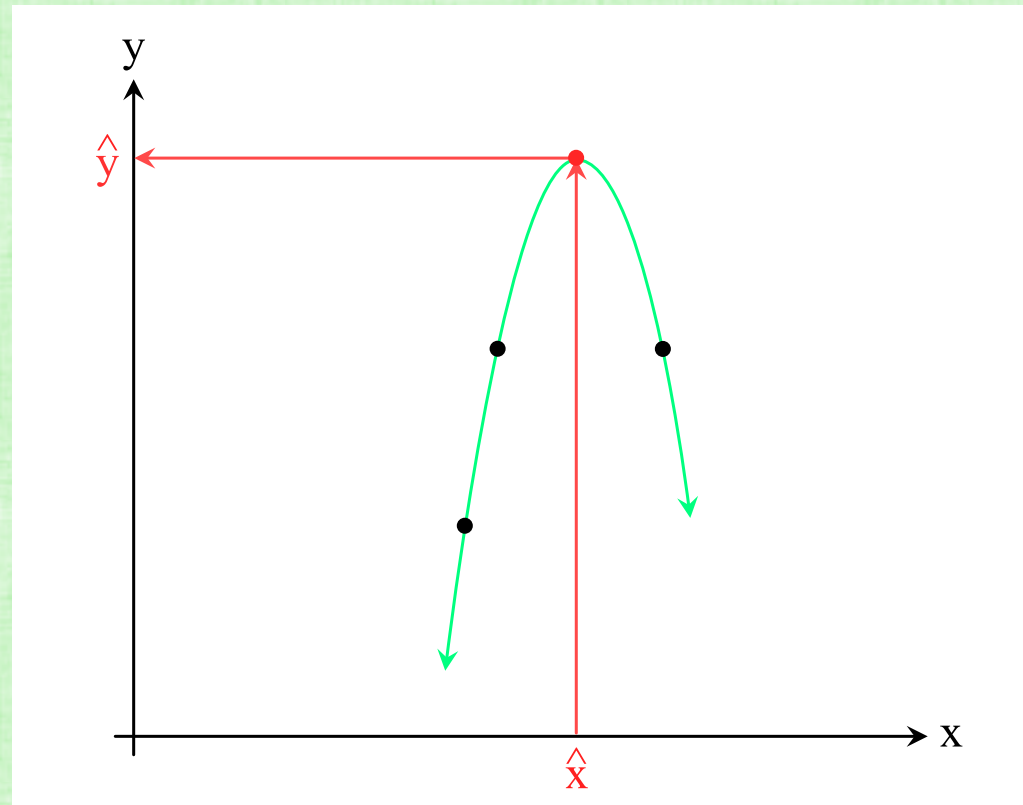


- The reasonable behavior of the piecewise constant (nearest neighbor) function stresses the importance of approximating data locally
- We address Local Approximations in Unit 6



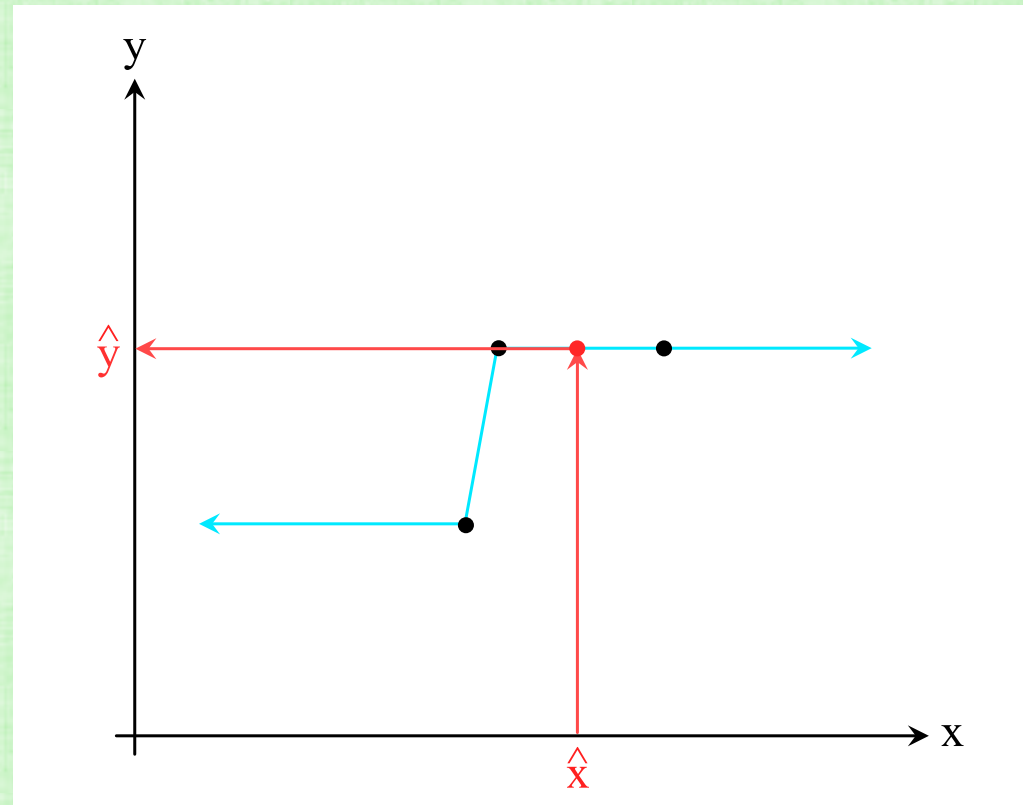
# Caution: Overfitting

- Higher order polynomials tend to oscillate wildly, but even a simple quadratic polynomial can overfit by quite a bit



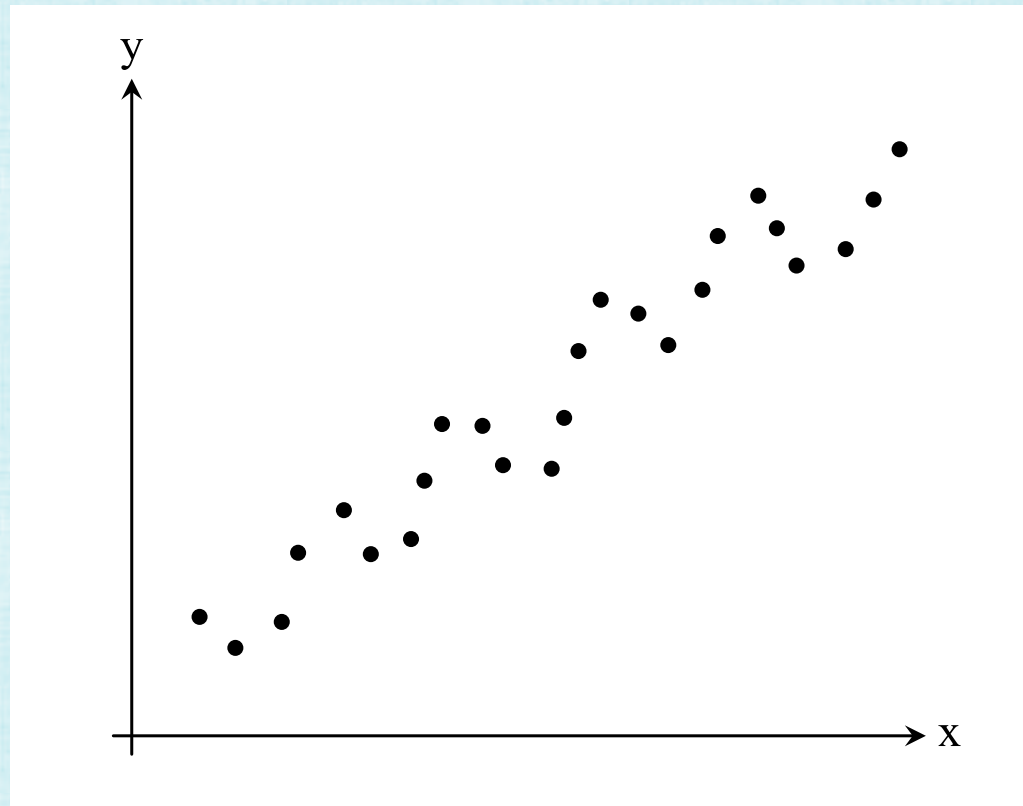
# Caution: Overfitting

- A piecewise linear approach works much better on this data



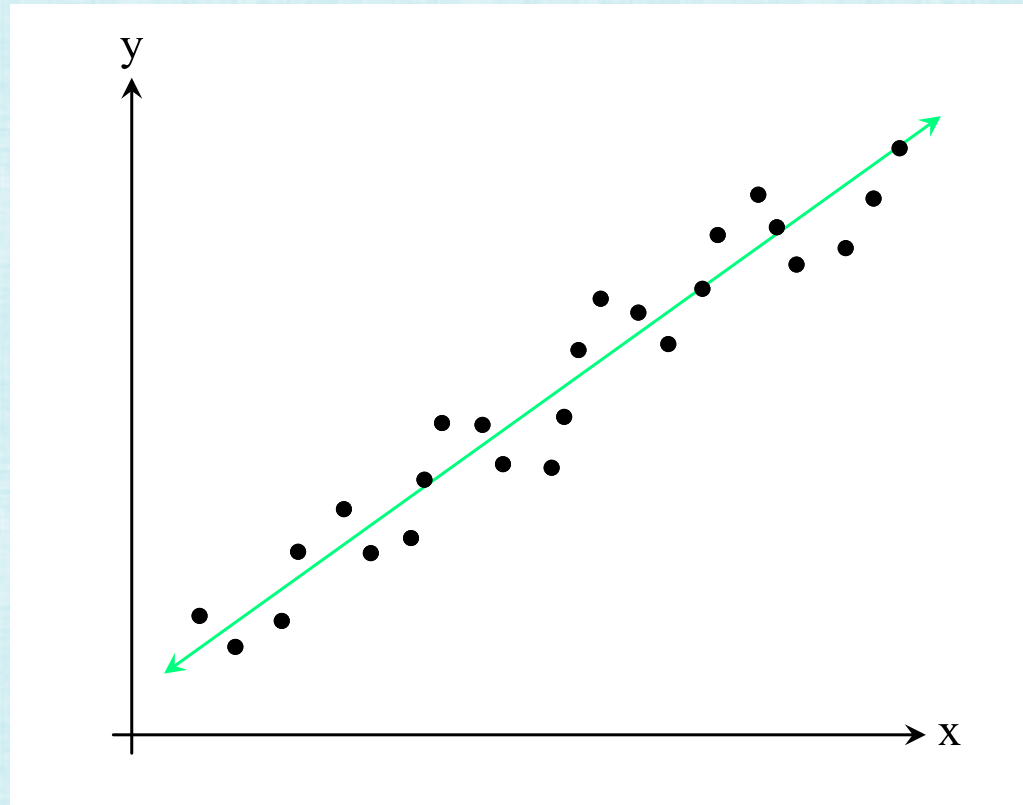
# Noisy Data

- There may be many sources of error in data, so it can be unwise to attempt to fit data too closely



# Linear Regression

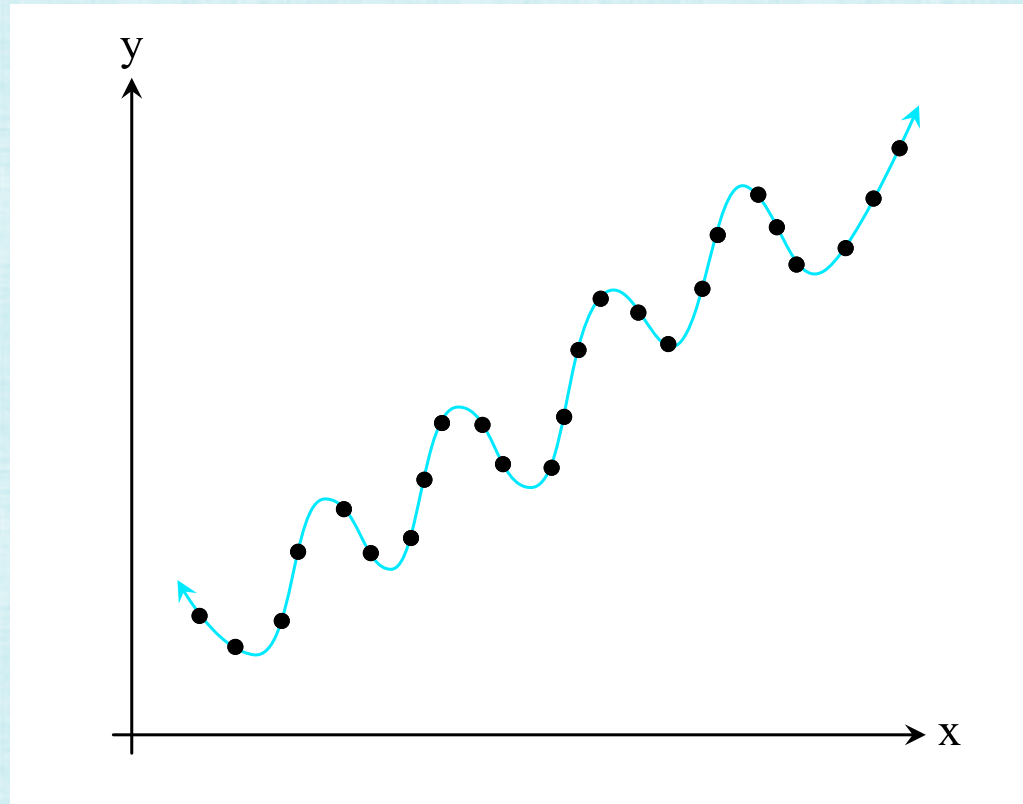
- One commonly fits a low order model to such data, while minimizing some metric of mis-interpolating or mis-representing the data





# Noise vs. Features

- But how does one differentiate between noise and features?



# Noise vs. Features

- When training a neural network, split the available data into 3 sets
  - E.g., 80% training data, 10% model validation data, and 10% test data
- Training data is used to train the neural network
  - An interpolating function is fit to the training data (potentially overfitting it)
- When considering features vs. noise, overfitting, etc., model validation data is used to select the best model function or the best fitting strategy
  - Compare inference/prediction on the model validation data to the known answers
- When disseminating results advocating the “best” validated model, inferencing on the test data suggests how well the validated model might generalize to unseen data
  - Competitions on unseen data have become a good way to stop “cheating” on test data

Break until Thursday

# Errors in Equations

- **Modeling errors** – Parts of a problem under consideration might be ignored. E.g., when simulating solids/fluids, sometimes frictional/viscous effects are not included or only approximated.
- **Empirical constants** – Some numbers are unknown and measured with limited precision. Others may be known more accurately, but limited precision hinders the ability to express them. E.g. Avogadro's number, the speed of light in a vacuum, the charge on an electron, Planck's constant, Boltzmann's constant,  $\pi$ , etc. (The speed of light is 299792458 m/s exactly, so ok for double precision but not for single precision.)



# Errors in Numerical Methods

- **Rounding errors**: Even integer calculations lead to floating point numbers, e.g.  $5/2=2.5$ , and floating-point calculations frequently admit rounding errors, e.g.  $1./3.=.3333333...$  cannot be expressed on the computer. Machine precision is  $10^{-7}$  for single precision and  $10^{-16}$  for double precision.
- **Truncation errors** – Also called discretization errors. These occur in the mathematical approximation of an equation as opposed to an approximation of the physics (modeling errors). E.g. one (often) cannot take a derivative/integral exactly on a computer and instead approximates them (recall Simpson's rule from Calculus).

# Errors in Inputs

- **Inaccurate inputs** – Often, one is only concerned with part of a calculation, where a given set of inputs is used to produce outputs. Those inputs may have previously been subjected to any of the errors listed above and thus may already have limited accuracy. This has implications for various algorithms. E.g., if inputs are only accurate to 4 decimal places, it probably doesn't make sense to carry out an algorithm to an accuracy of 8 decimal places.
- **Inaccurate Measurements** – It can be difficult to accurately measure real-world phenomena, generating another source of inaccurate inputs.

# A Robust Computational Approach

- **Well-Posedness**: A **problem** is ill-posed if small changes in the inputs lead to large changes in the outputs, implying that any source of error could dominate the results.
- **Condition Number**: An **algorithm** is ill-conditioned if small changes in the inputs lead to large changes in the outputs. Large condition numbers are bad (sensitive), and small condition numbers are good (insensitive). If the relative changes in the inputs and outputs are identical, the condition number is 1.
- **Stability**: An algorithm is stable if it can complete itself in any meaningful way. Unstable algorithms give wildly explosive results, usually leading to NaN's.
- **Accuracy**: Accuracy refers to the size of the error, or how close the answer is to the correct solution.

# A Robust Computational Approach

- Only well-posed problems can be tackled computationally
- Computational Approach:
  - 1) Conditioning - formulate a well-conditioned approach, or as well-conditioned as is possible
  - 2) Stability - devise a stable algorithm; otherwise, the result could be NaNs
  - 3) Accuracy – even a well-conditioned and stable approach can result in large errors; so, make the algorithm as accurate as is warranted or practical



## Being Careful: Vector Norms

- Consider the norm of a vector:  $\|x\|_2 = \sqrt{x_1^2 + \dots + x_m^2}$
- Straightforward algorithm: for (i=1,m) sum+=x(i)\*x(i); return sqrt(sum);
- This can overflow MAX\_FLOAT/MAX\_DOUBLE for large  $m$

- Safer algorithm:

$$\|x\|_2 = z \sqrt{\left(\frac{x_1}{z}\right)^2 + \dots + \left(\frac{x_m}{z}\right)^2}$$

find z=max(abs(x(i)))

for (i=1,m) sum+=sqr(x(i)/z); return z\*sqrt(sum);

# Being Careful: Quadratic Formula

- Consider  $.0501x^2 - 98.78x + 5.015 = 0$ 
  - To 10 digits of accuracy, the solutions are:  $x \approx 1971.605916$  and  $x \approx .05077069387$

- Using 4 digits of accuracy in the quadratic formula gives:

$$\frac{98.78+98.77}{.1002} = 1972 \quad \text{and} \quad \frac{98.78-98.77}{.1002} = .0998$$

- The **second root is** completely **wrong** in the leading significant digit!
- De-rationalize:  $\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$  to  $\frac{2c}{-b \mp \sqrt{b^2 - 4ac}}$
- Using 4 digits of accuracy in the de-rationalized quadratic formula gives:

$$\frac{10.03}{98.78-98.77} = 1003 \quad \text{and} \quad \frac{10.03}{98.78+98.77} = .05077$$

- The second root is fine, but the **first root is wrong** by a factor of two!
- Conclusion: use one formula for each root

# Being Careful: L'Hopitals Rule

- Consider  $\frac{x^2-4}{x-2}$  near  $x = 2$  where it becomes  $\frac{0}{0}$
- Adding a small number to the denominator  $\frac{x^2-4}{x-2+\epsilon}$  incorrectly gives 0 near  $x = 2$
- Using  $\frac{x^2-4}{x-2} = x + 2$  leads to correct values near 4 when  $x$  is near 2
- Similar issues occur for  $\frac{\sin x}{x}$  near  $x = 0$
- Similar issues occur for  $0$  times  $\infty$  and other cases where L'Hopitals rule is needed to address removable singularities

*Did you know about these issues?*

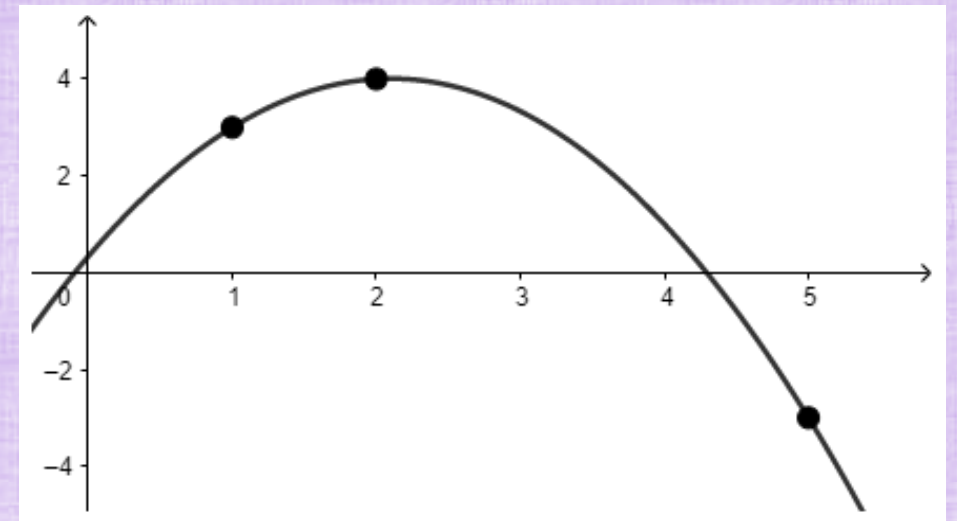
- Imagine debugging code with the correct quadratic formula implementation and getting zero digits of accuracy on a test case!



# Polynomial Interpolation

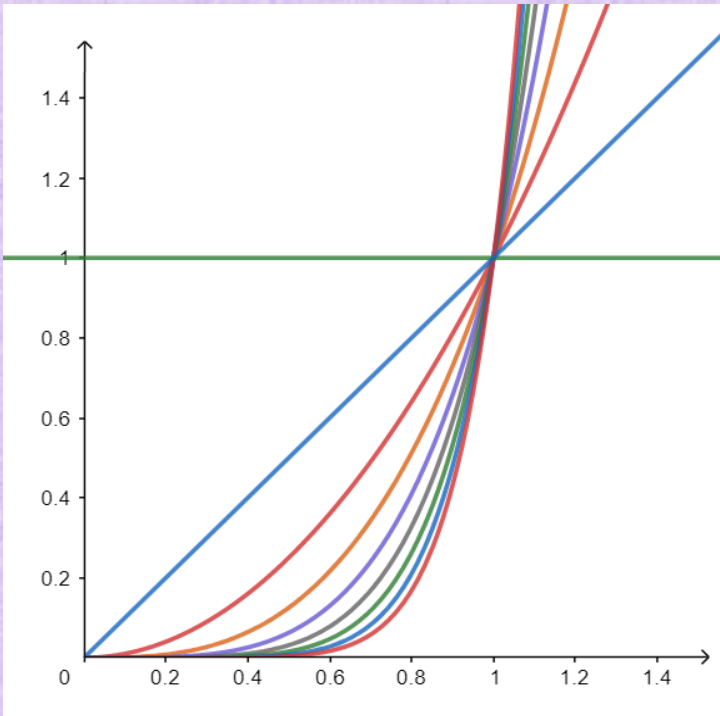
- Given  $m$  data points  $(x_i, y_i)$ , find the unique polynomial that passes through them:  $y = c_1 + c_2x + c_3x^2 + \cdots + c_mx^{m-1}$
- Write an equation for each data point, note that the equations are linear, and put into matrix form
- For example, consider  $(1,3)$ ,  $(2,4)$ ,  $(5,-3)$  and a quadratic polynomial

- Then, 
$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 5 & 25 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \\ -3 \end{pmatrix}$$
 gives  
$$\begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 1/3 \\ 7/2 \\ -5/6 \end{pmatrix} \text{ and } f(x) = \frac{1}{3} + \frac{7}{2}x - \frac{5}{6}x^2$$



# Polynomial Interpolation

- Solve  $Ac = y$  where the Vandermonde matrix  $A$  has a row for each data point. Each row looks like  $(1 \quad x_i \quad x_i^2 \quad \cdots \quad x_i^{m-1})$

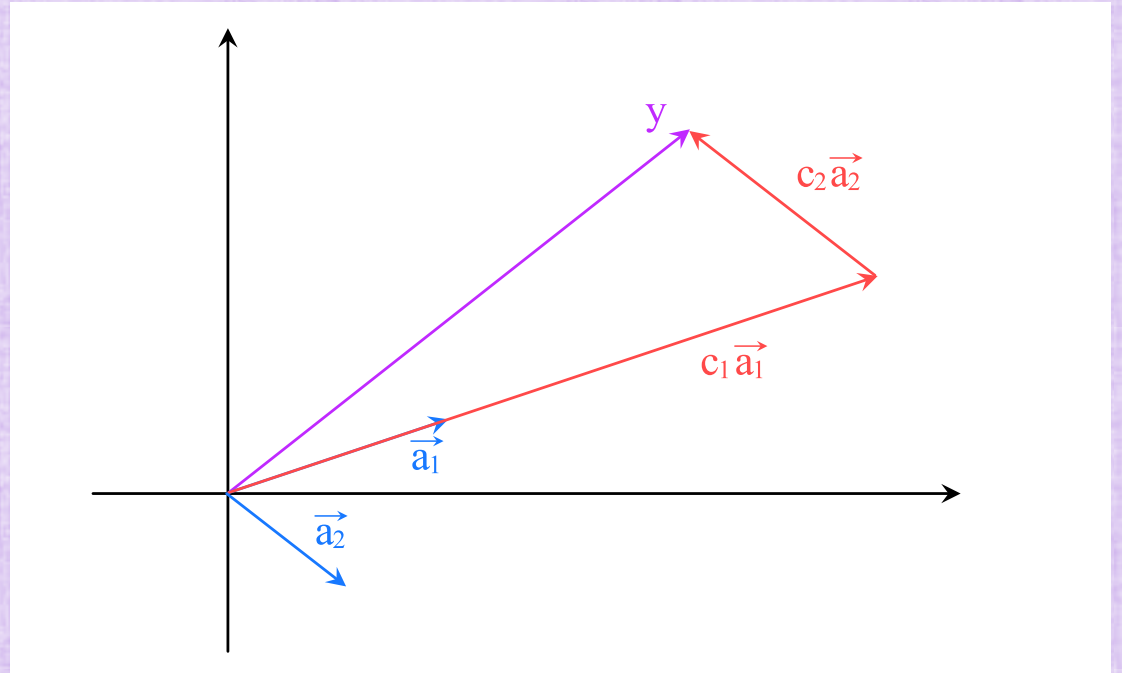
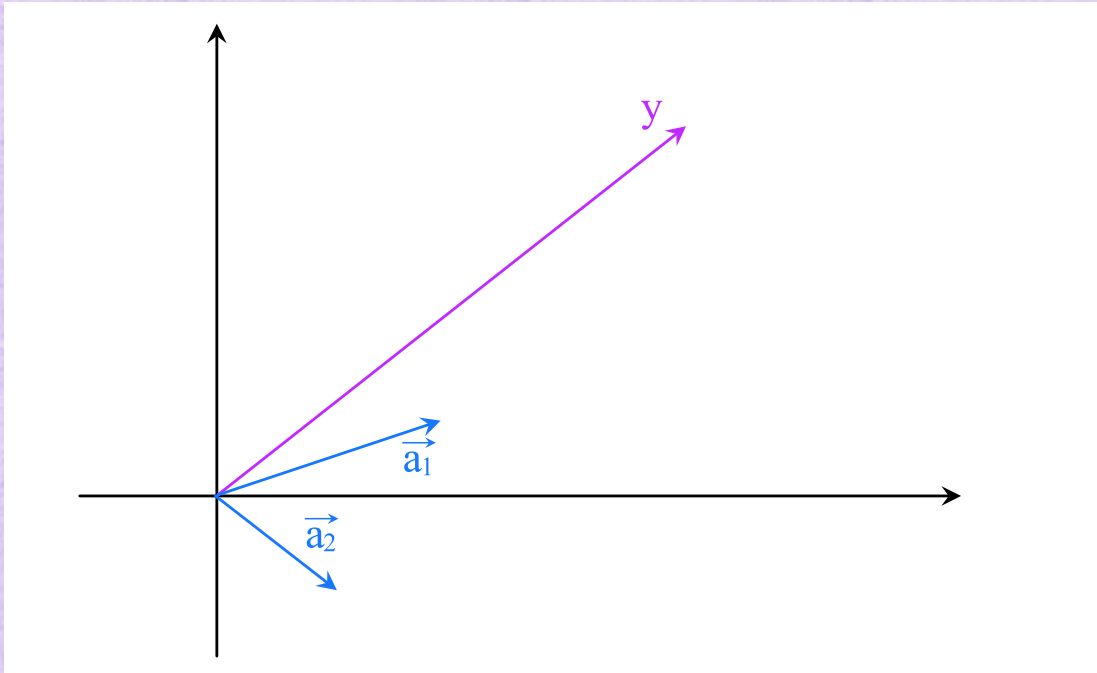


- Monomials look more similar to each other at higher powers
- This makes the rightmost columns of a Vandermonde matrix tend to become more parallel
- More parallel columns make the matrix less invertible, making it more difficult to solve for the  $c_k$  parameters
- More parallel columns make the matrix more **ill-conditioned** to invert, and it becomes difficult or impossible to invert it with a computer

$$f(x) = 1, x, x^2, x^3, x^4, x^5, x^6, x^7, x^8$$

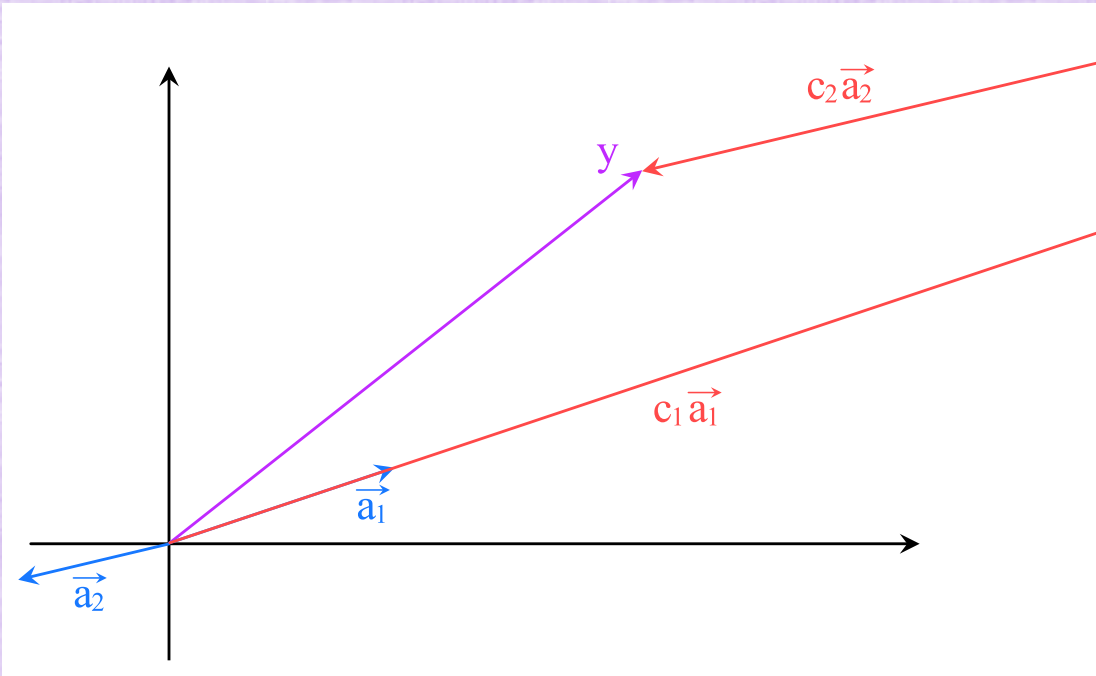
# Matrix Columns as Vectors

- Define the  $k$ -th column of  $A$  as vector  $a_k$ , so  $Ac = y$  is equivalent to  $\sum_k c_k a_k = y$
- That is, find a linear combination of the columns of  $A$  that gives the right-hand side vector  $y$



# Matrix Columns as Vectors

- As columns become more parallel, the values of  $c$  become arbitrarily large, ill-conditioned, and prone to error



- The **red** vectors go too far to the right to fully illustrate



# Singular Matrices

- If two columns of a matrix are parallel, they may be combined in an infinite number of ways while still obtaining the same result
  - Thus, the problem does not have a unique solution
- In addition, the  $n$  columns of  $A$  span at most an  $n - 1$  dimensional subspace
  - Thus, the range of  $A$  is at most  $n - 1$  dimensional
- If the right hand side vector is not contained in this  $n - 1$  dimensional subspace, the problem has no solution
  - Otherwise, there are infinite solutions

# Singular Matrices

- If any column of a matrix is a linear combination of other columns, they may be combined in an infinite number of ways while still obtaining the same result
  - Thus, the problem does not have a unique solution
- In addition, the  $n$  columns of  $A$  span at most an  $n - 1$  dimensional subspace
  - Thus, the range of  $A$  is at most  $n - 1$  dimensional
- If the right hand side vector is not contained in this  $n - 1$  dimensional subspace, the problem has no solution
  - Otherwise, there are infinite solutions

# Near Singular Matrices

- With limited numerical precision, one struggles when columns (or linear combinations of columns) are too close to being parallel to each other
- Analytically invertible matrices may not be computationally invertible!
- A condition number can be used to describe how close a matrix is to being non-invertible on a computer
- The condition number is  $\infty$  for a singular matrix and 1 for the identity matrix

# Being Careful: Polynomial Interpolation

- Given basis functions  $\phi$  and unknown parameters  $c$ :

$$y = c_1\phi_1 + c_2\phi_2 + \cdots + c_n\phi_n$$

- Monomial basis:  $\phi_k(x) = x^{k-1}$
- As we have seen, the Vandermonde matrix may become near-singular and thus be computationally difficult to invert

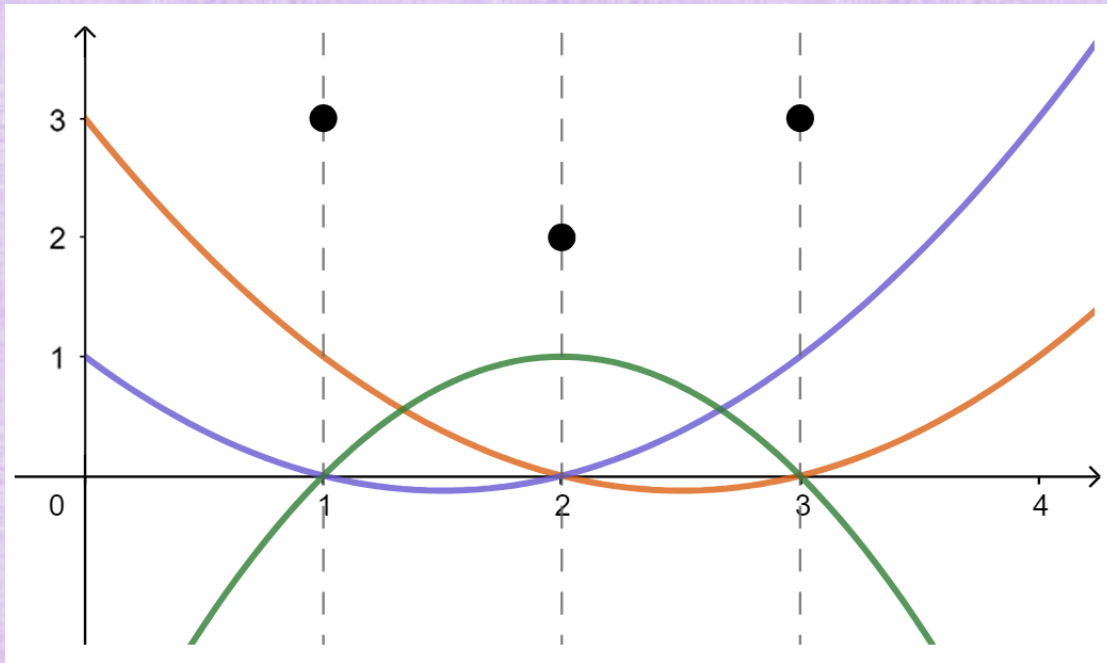


# Lagrange Basis

- Basis functions:  $\phi_k(x) = \frac{\prod_{i \neq k} x - x_i}{\prod_{i \neq k} x_k - x_i}$
- Thus,  $\phi_k(x_k) = 1$
- Thus,  $\phi_k(x_i) = 0$  for  $i \neq k$
- As usual: write an equation for each point, note that the equations are linear, and put into matrix form
- Obtain  $Ac = y$  where  $A$  is now the identity matrix (i.e.  $Ic = y$ ); so,  $c = y$  trivially
- Easy to solve for  $c$ , but evaluation of the polynomial (with lots of terms) is expensive
  - i.e. inference is expensive

# Lagrange Basis

- Consider  $(1,3)$ ,  $(2,2)$ ,  $(3,3)$  with quadratic basis functions that are 1 at their corresponding data point and 0 at the other data points



- $\phi_1(x) = \frac{(x-2)(x-3)}{(1-2)(1-3)} = \frac{1}{2}(x-2)(x-3)$
- $\phi_1(1) = 1, \phi_1(2) = 0, \phi_1(3) = 0$
- $\phi_2(x) = \frac{(x-1)(x-3)}{(2-1)(2-3)} = -(x-1)(x-3)$
- $\phi_2(1) = 0, \phi_2(2) = 1, \phi_2(3) = 0$
- $\phi_3(x) = \frac{(x-1)(x-2)}{(3-1)(3-2)} = \frac{1}{2}(x-1)(x-2)$
- $\phi_3(1) = 0, \phi_3(2) = 0, \phi_3(3) = 1$

# Newton Basis

- Basis functions:  $\phi_k(x) = \prod_{i=1}^{k-1} x - x_i$
- $Ac = y$  now has a lower triangular  $A$  (as opposed to being dense or diagonal)
- The columns don't overlap, and it's not too expensive to evaluate/inference
- Can solve via a divided difference table:
  - Initially:  $f[x_i] = y_i$
  - Then, at each level, recursively:  $f[x_1, x_2, \dots, x_k] = \frac{f[x_2, x_3, \dots, x_k] - f[x_1, x_2, \dots, x_{k-1}]}{x_k - x_1}$
  - Finally:  $c_k = f[x_1, x_2, \dots, x_k]$
- As usual, high order polynomials still tend to be oscillatory
  - Using unequally spaced data points can help, e.g. Chebyshev points

# Summary

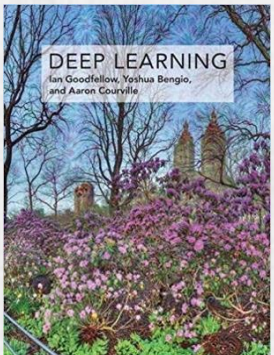
- The Monomial/Lagrange/Newton basis all give the same exact unique polynomial
  - as one can see by multiplying out and collecting like terms
- But the representation used makes it easier/harder to find the polynomial as well as more/less computationally expensive to subsequently evaluate the polynomial
- The representation (or the basis in this case) matters!



# Representation Matters

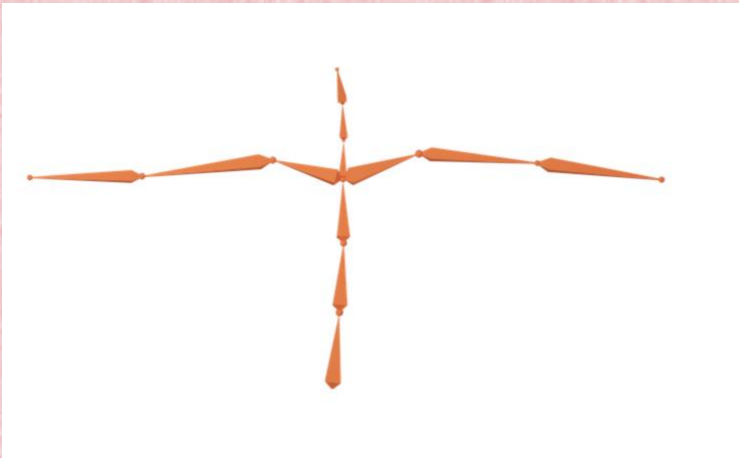
- Consider: Divide CCX by VI
- As compared to: Divide 210 by 6

- See Chapter 15 on Representation Learning in the Deep Learning book



# Predict 3D Cloth Shape from Body Pose

- Input: pose parameters  $\theta$  are joint rotation matrices
  - 10 upper body joints with a  $3 \times 3$  rotation matrix for each gives a  $90D$  pose vector ( $30D$  when using quaternions)
  - Ignore global translation/rotation of the root frame
- Output:  $3D$  cloth shape  $\varphi$ 
  - 3,000 vertices in a cloth triangle mesh gives a  $9,000D$  shape vector
- Function  $f: \mathbf{R}^{90} \rightarrow \mathbf{R}^{9000}$

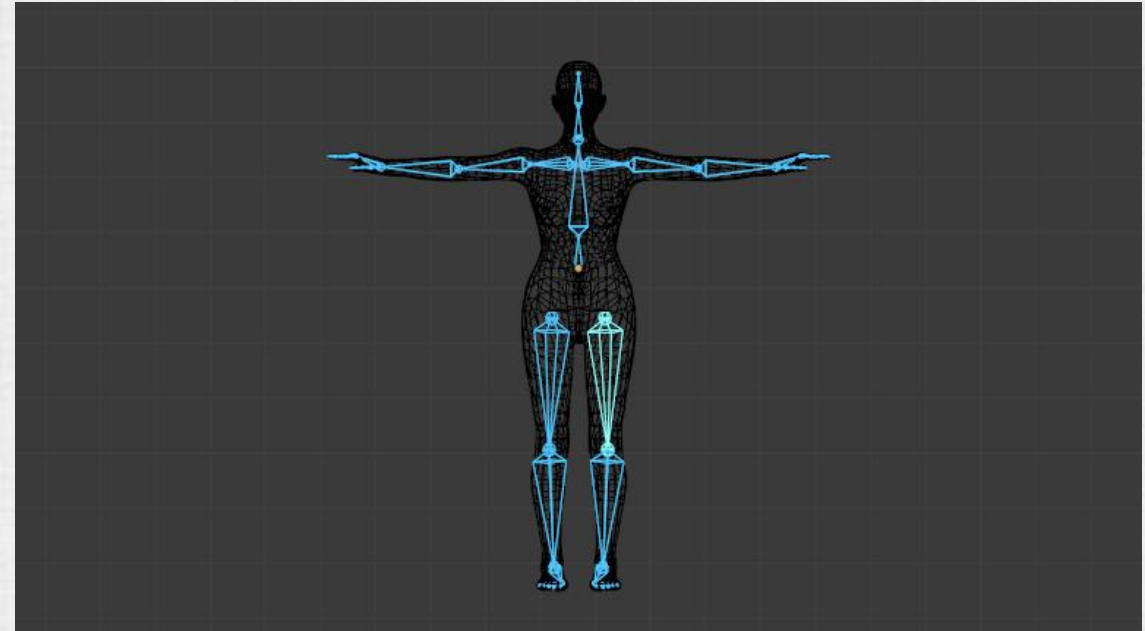


# Approach

- Given:  $m$  training data points  $(\theta_i, \varphi_i)$  generated from the true/approximated function  $\varphi_i = f(\theta_i)$ 
  - E.g. using physical simulation or computer vision techniques
- Goal: learn an  $\hat{f}$  that approximates  $f$ 
  - i.e.  $\hat{f}(\theta) = \hat{\varphi} \approx \varphi = f(\theta)$
- Issue: As joints rotate (rotation is highly nonlinear), cloth vertices move in complex nonlinear ways that are difficult to capture with a neural network
  - i.e. it is difficult to ascertain a suitable  $\hat{f}$  using straightforward techniques
- How should the nonlinear rotations be handled?

# Skinning

- Deforms a body surface mesh to match a skeletal pose
  - well studied and widely used in graphics
- Each vertex of the body surface mesh is associated with one or more nearby bones
- A weight (for each bone/vertex pair) dictates how much a change in a bone's position/orientation impacts a vertex's position
- As the pose changes, bone changes dictate new positions for body surface mesh vertices



Credit: Blender website

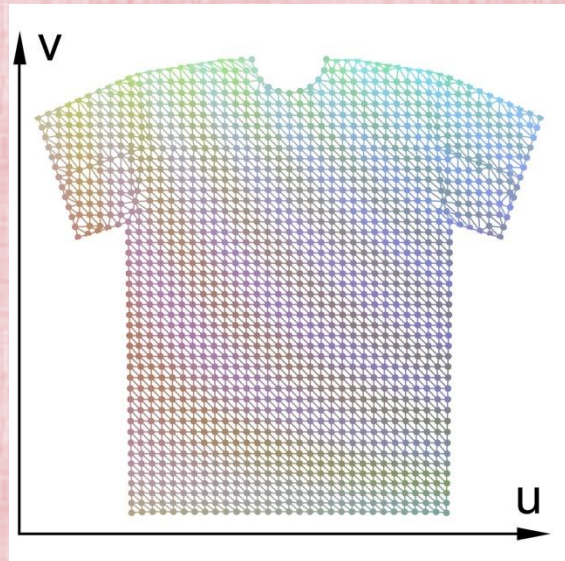


## Being Careful: Leveraging Skinning

- Leverage the plethora of prior work on procedural skinning to estimate the body surface mesh  $S$  based on pose parameters  $\theta$
- Then, represent the cloth mesh as offsets  $D(\theta)$  from the skinned mesh  $S(\theta)$
- Overall,  $\varphi = f(\theta) = S(\theta) + D(\theta)$ , where only  $D(\theta)$  needs to be learned
- The skinning **prior**  $S(\theta)$  captures much of the nonlinearities, so that the remaining  $D(\theta)$  is a smoother function and thus easier to approximate/learn

# Shrink Wrap the Cloth Mesh

- Shrink-wrap the cloth vertices to the body triangle mesh
- Barycentrically embed the cloth vertices to follow body mesh triangles
- As the body deforms, cloth vertices move with their parent triangles
- This is  $S(\theta)$



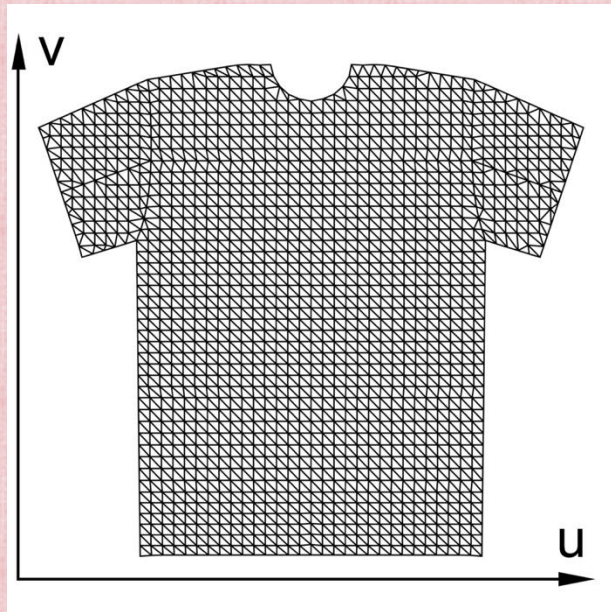
pattern space



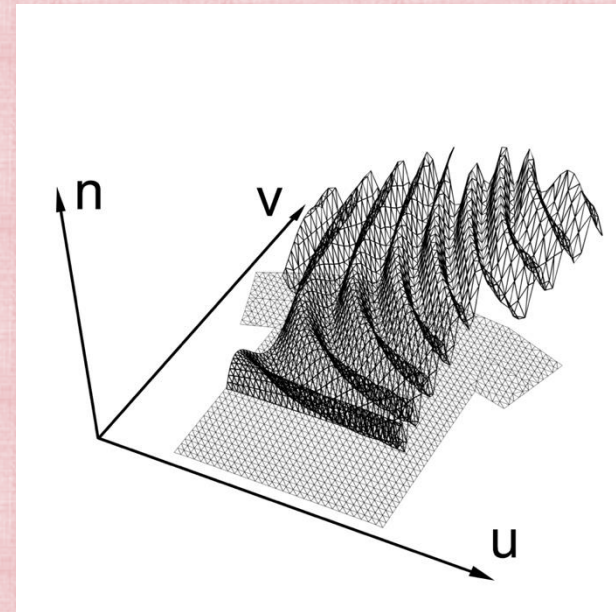
world space

# Displacement Map

- Store per-vertex  $(u, v, n)$  offsets ( $n$  is the normal) in pattern (i.e. texture) space
- This is  $D(\theta)$



pattern space



$(u, v, n)$  offsets  
in pattern space



# Skinning + Displacement Map

$S(\theta)$



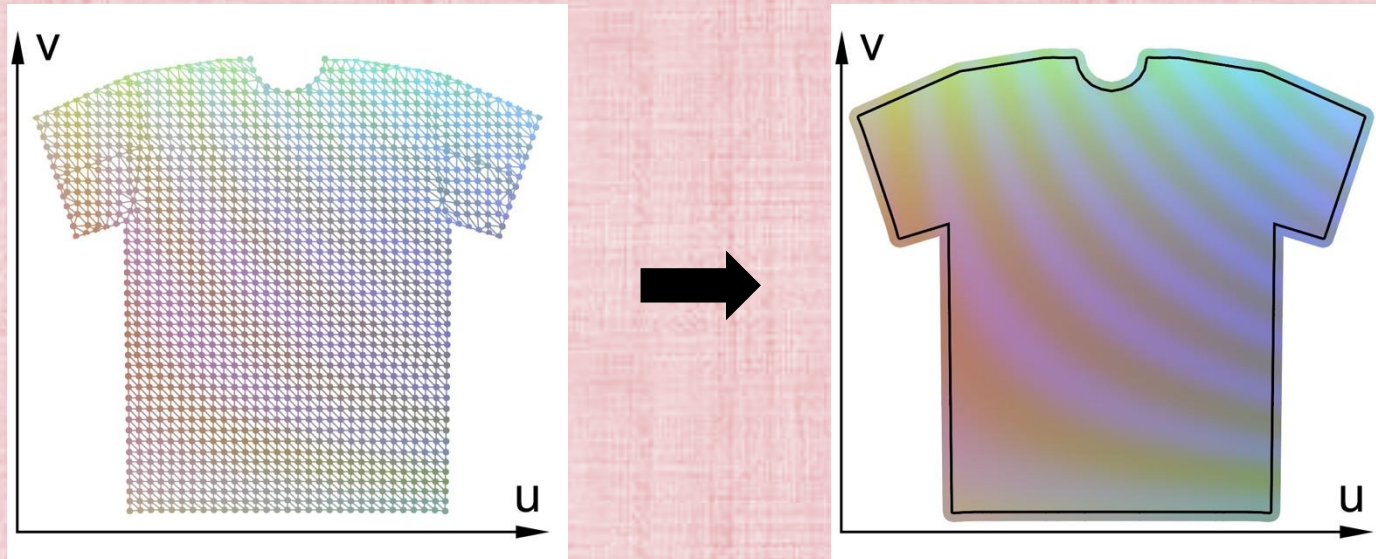
$S(\theta) + D(\theta)$





# Image Based Cloth

- Convert the  $(u, v, n)$  offsets to RGB-triple color values
- Then, rasterize the triangle vertex colors to 2D image pixels (in the pattern space)
- The network learns to output a 2D RGB image (instead of 3D displacements)
- Images are more continuous than the cloth vertices, which have an irregular mesh/graph topology
- Learn to predict images as a function of pose  $\theta$ , using Convolutional Neural Networks (CNNs) instead of Graph Neural Networks (GNNs)



# Training Data

- For each pose in the training data, calculate per-vertex offsets and rasterize them into an image in pattern space
- Training data are  $(\theta, I(\theta))$  pairs, where  $I$  is the image



# Inference

- Learn to predict an image from pose parameters, i.e. learn  $\hat{I}(\theta) \approx I(\theta)$
- Given an inferred image  $\hat{I}(\theta)$ , interpolate RGB colors to cloth vertices and convert colors to offsets; then, the offsets are added to the skinned vertex positions:  $\hat{\phi}(\theta) = S(\theta) + \underbrace{\psi(\hat{I}(\theta))}_{D(\theta)}$



# Unit 2



# Linear Systems

# Motivation

- “Matrices are bad, vector spaces are good”
  - Don’t think of matrices as a collection of numbers
  - Instead, think of the columns as vectors in a high dimensional space
- We don’t have great intuition going from  $R^1$  to  $R^2$  to  $R^3$  to  $R^n$  (for large  $n$ )
- Thinking about vectors in high dimensional spaces is a good way of gaining intuition about what’s going on
- Linear algebra contains a lot of machinery for dealing with, discussing, and gaining intuition about vectors in high dimensional spaces
- We will cover linear algebra from the viewpoint of understanding higher dimensional spaces

# System of Linear Equations

- System of equations:  $3c_1 + 2c_2 = 6$  and  $-4c_1 + c_2 = 7$
- Matrix form:  $\begin{pmatrix} 3 & 2 \\ -4 & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 6 \\ 7 \end{pmatrix}$  or  $Ac = b$
- Given  $A$  and  $b$ , determine  $c$
- Theoretically, there is a unique solution, no solution, or infinite solutions
- Ideally, software would determine whether there was a unique solution, no solution, or infinite solutions; in the last case, it would list a parameterized family of solutions. Unfortunately, this is quite difficult to accomplish.
- Note: in this class,  $x$  is used for **data**, and  $c$  is used for **unknowns** (such as for the unknown parameters of a neural network)

# “Zero”

- On the computer, defining “zero” is not straightforward
- When dealing with large numbers (e.g. Avogadro’s number:  $6.022e23$ ) zero can be quite large
  - E.g.  $6.022e23 - 1e7 = 6.022e23$  in double precision, making  $1e7$  behave like “zero”
- When dealing with small numbers (e.g.  $1e - 23$ ), “zero” is much smaller
  - In this case, on the order of  $1e - 39$  in double precision
- Mixing big and small numbers often wreaks havoc on algorithms
- So, we typically non-dimensionalize and normalize to make equations  $O(1)$  as opposed to  $O(\text{“big”})$  or  $O(\text{“small”})$



# Row/Column Scaling

- Consider: 
$$\begin{pmatrix} 3e6 & 2e10 \\ 1e-4 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 5e10 \\ 6 \end{pmatrix}$$

- Row Scaling - divide first row by  $1e10$  to obtain:

$$\begin{pmatrix} 3e-4 & 2 \\ 1e-4 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 6 \end{pmatrix}$$

- Column Scaling - define a new variable  $c_3 = (1e-4)c_1$  to obtain:

$$\begin{pmatrix} 3 & 2 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} c_3 \\ c_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 6 \end{pmatrix}$$

- The final matrix is much easier to treat with finite precision arithmetic
- Solve for  $c_3$  and  $c_2$ ; then,  $c_1 = (1e4)c_3$

# Some Definitions...

- Elements of a matrix are often referred to by their row and column
- For example,  $a_{ik}$  is the element of matrix  $A$  in row  $i$  and column  $k$
- Transpose swaps the row and column of every entry
- $A^T$  moves element  $a_{ik}$  to row  $k$  column  $i$  (and vice versa)
- Non-square matrices change size:  $\begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$

Symmetric Matrices have  $A^T = A$  meaning that  $a_{ik} = a_{ki}$  for all  $i$  and  $k$

# Square Matrices

- A size  $m \times n$  matrix has  $m$  rows and  $n$  columns
- For now, let's just consider square  $n \times n$  matrices
- We will consider non-square (rectangular) matrices with  $m \neq n$  a bit later

# Solvability

- A square matrix  $A$  is called singular when it is not invertible (does not have an inverse)
- Nonsingular square matrices are invertible:  $AA^{-1} = A^{-1}A = I$ 
  - So,  $Ac = b$  can be solved for  $c$  via  $c = A^{-1}b$
  - *Note: we typically do not compute the inverse, but instead have a solution algorithm that exploits its existence*
- Rank - maximum number of linearly independent columns
- $A$  is singular if and only if its rank  $< n$  (the # of columns), i.e. the columns are rank-deficient
  - A singular matrix has either no solution or infinite solutions (can't have a unique solution, see Unit 1)
- Various equivalent ways of determining whether  $A$  is singular:
  - At least one column is linearly dependent on others (not full rank)
  - It has a nonempty null space, i.e.  $\exists c \neq 0$  with  $Ac = 0$  (columns are linearly dependent)
  - The determinant is zero:  $\det A = 0$  (zero  $n$ -dimensional “volume” in the geometry spanned by the vectors)
  - Etc.



# Matrices as Vectors (an example)

- Recall  $Ac = \sum_k c_k a_k$  where the  $a_k$  are the columns of  $A$
- Consider  $Ac = 0$ , i.e.  $\sum_k c_k a_k = 0$
- If one column is a linear combination of others, then the linear combination weights can be used to obtain  $Ac = 0$  with  $c$  nonzero
  - This nonzero  $c$  is in the null space of  $A$ , and so  $A$  is singular
- Alternatively: If the only solution to  $Ac = 0$  is a  $c$  identically 0, then no column is linearly dependent on the others
  - Thus,  $A$  is nonsingular

# Diagonal Matrices

- All off-diagonal entries are 0
- Equations are decoupled, and easy to solve
- E.g.  $\begin{pmatrix} 5 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 10 \\ -1 \end{pmatrix}$  has  $5c_1 = 10$  and  $2c_2 = -1$ ; so,  $c_1 = 2$  and  $c_2 = -.5$
- A zero on the diagonal indicates a singular system
  - Either no solution (e.g.  $0c_1 = 10$ ) or infinite solutions (e.g.  $0c_1 = 0$ )
- The determinant of a diagonal matrix is obtained by multiplying all the diagonal elements together
  - Thus, a 0 on the diagonal implies a zero determinant and a singular matrix

# Upper Triangular Matrices

- All entries below the diagonal are 0
- Nonsingular when the diagonal elements are all nonzero
  - Determinant is obtained by multiplying all the diagonal elements together
- Solve via back substitution

- E.g. consider 
$$\begin{pmatrix} 2 & 3 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 5 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 10 \\ 10 \end{pmatrix}$$

- Start at the bottom:  $5c_3 = 10$ ; so,  $c_3 = 2$
- Move up one row:  $c_2 - c_3 = 10$ ; so,  $c_2 - 2 = 10$  and  $c_2 = 12$
- Move up one row:  $2c_1 + 3c_2 + c_3 = 0$ ; so,  $2c_1 + 36 + 2 = 0$  and  $c_1 = -19$

# Lower Triangular Matrices

- All entries above the diagonal are 0
- Nonsingular when the diagonal elements are all nonzero
  - Determinant is obtained by multiplying all the diagonal elements together
- Solve via forward substitution

- E.g. consider 
$$\begin{pmatrix} 5 & 0 & 0 \\ -1 & 1 & 0 \\ 1 & 3 & 2 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 10 \\ 10 \\ 0 \end{pmatrix}$$

- Start at the top:  $5c_1 = 10$ ; so,  $c_1 = 2$
- Move down one row:  $-c_1 + c_2 = 10$ ; so,  $-2 + c_2 = 10$  and  $c_2 = 12$
- Move down one row:  $c_1 + 3c_2 + 2c_3 = 0$ ; so,  $2 + 36 + 2c_3 = 0$  and  $c_3 = -19$



# LU Factorization

- In order to solve  $Ac = b$  more generally, factor  $A$  into a lower triangular and an upper triangular matrix:  $A = LU$ 
  - The LU factorization only needs to be computed once; then, it can be used on many right-hand side vectors (on many  $b$  vectors)

Process for solving  $LUc = b$ :

- Define an auxiliary variable  $\hat{c} = Uc$
- First, solve  $L\hat{c} = b$  for  $\hat{c}$  via forward substitution
- Second, solve  $Uc = \hat{c}$  for  $c$  via back substitution

# Elimination Matrix

- Given the  $k$ -th column vector  $\begin{pmatrix} a_{1k} \\ \vdots \\ a_{ik} \\ a_{i+1,k} \\ \vdots \\ a_{nk} \end{pmatrix}$ , first define  $m_{ik} = \frac{1}{a_{ik}} \begin{pmatrix} 0 \\ \vdots \\ 0 \\ a_{i+1,k} \\ \vdots \\ a_{nk} \end{pmatrix}$
- Then, the size  $n \times n$  elimination matrix  $M_{ik} = I_{n \times n} - m_{ik} \hat{e}_i^T$  subtracts multiples of row  $i$  from rows  $> i$  in order to create zeroes in column  $k$

- Standard basis vector  $\hat{e}_i = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$  has a 1 in the  $i$ -th row

# Elimination Matrix

- Let  $a_k = \begin{pmatrix} 2 \\ 4 \\ -8 \end{pmatrix}$

- $M_{1k} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{1}{2} \begin{pmatrix} 0 \\ 4 \\ -8 \end{pmatrix} (1 \ 0 \ 0) = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 4 & 0 & 1 \end{pmatrix}$  and  $M_{1k}a_k = \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix}$

- $M_{2k} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} - \frac{1}{4} \begin{pmatrix} 0 \\ 0 \\ -8 \end{pmatrix} (0 \ 1 \ 0) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \end{pmatrix}$  and  $M_{2k}a_k = \begin{pmatrix} 2 \\ 4 \\ 0 \end{pmatrix}$

# Elimination Matrix Inverse

- Inverse of an elimination matrix is  $L_{ik} = M_{ik}^{-1} = I_{n \times n} + m_{ik} \hat{e}_i^T$
- $L_{ik}$  is a size  $n \times n$  elimination matrix that **adds** multiples of row  $i$  to rows  $> i$  in order to reverse the effect of  $M_{ik}$

- $L_{1k} = M_{1k}^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -4 & 0 & 1 \end{pmatrix}$

- $L_{2k} = M_{2k}^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix}$



# Combining Elimination Matrices

- $M_{i_1 k_1} M_{i_2 k_2} = I - m_{i_1 k_1} \hat{e}_{i_1}^T - m_{i_2 k_2} \hat{e}_{i_2}^T$  when  $i_1 < i_2$  (but not when  $i_1 > i_2$ )

$$M_{1k} M_{2k} = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 4 & 2 & 1 \end{pmatrix}, \text{ but } M_{2k} M_{1k} = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 2 & 1 \end{pmatrix}$$

- $L_{i_1 k_1} L_{i_2 k_2} = I + m_{i_1 k_1} \hat{e}_{i_1}^T + m_{i_2 k_2} \hat{e}_{i_2}^T$  when  $i_1 < i_2$  (but not when  $i_1 > i_2$ )

$$L_{1k} L_{2k} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -4 & -2 & 1 \end{pmatrix}, \text{ but } L_{2k} L_{1k} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -8 & -2 & 1 \end{pmatrix}$$

# Gaussian Elimination

- Consider  $\begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 8 \\ 10 \end{pmatrix}$
- $M_{11}A = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix} = \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 1 & 5 \end{pmatrix}$  and  $M_{11}b = \begin{pmatrix} 2 \\ 4 \\ 12 \end{pmatrix}$
- $M_{22}M_{11}A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 1 & 5 \end{pmatrix} = \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 4 \end{pmatrix}$  and  $M_{22}M_{11}b = \begin{pmatrix} 2 \\ 4 \\ 8 \end{pmatrix}$
- Then, solve the upper triangular  $\begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 4 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 8 \end{pmatrix}$  via back substitution

# LU Factorization

- Gaussian Elimination gives an upper triangular  $U = M_{n-1,n-1} \cdots M_{22}M_{11}A$
- Using inverses,  $A = L_{11}L_{22} \cdots L_{n-1,n-1}M_{n-1,n-1} \cdots M_{22}M_{11}A = L_{11}L_{22} \cdots L_{n-1,n-1}U$
- Since  $L_{i_1i_1}L_{i_2i_2} = I + m_{i_1i_1}\hat{e}_{i_1}^T + m_{i_2i_2}\hat{e}_{i_2}^T$  when  $i_1 < i_2$ ,  $L = L_{11}L_{22} \cdots L_{n-1,n-1}$  is lower triangular and  $A = LU$

- Here  $L = L_{11}L_{22} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 1 & 1 \end{pmatrix}$

$$A = \begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 4 \end{pmatrix} = LU$$

# Pivoting

- $A = \begin{pmatrix} 0 & 4 \\ 4 & 9 \end{pmatrix}$  requires division by **zero** when creating  $M_{11}$
- (Partial) Pivoting - swap rows to use the largest (magnitude) element in the column under consideration
  - Don't forget to swap the right-hand side  $b$  too
- Full Pivoting swap rows and columns to use the **largest possible element**
  - Don't forget to change the order of the unknowns  $c$
- When considering column  $k$ , can only swap with rows/columns  $\geq k$



# Permutation Matrix

- Constructed by switching the 2 rows of  $I$  that one wants swapped
  - E.g.  $P_{13} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$  swaps the first and third rows of  $A$  via  $P_{13}A$
- Permutation matrices are their own inverses (swapping again restores the rows)
- Switching rows  $i_1$  and  $i_2$  moves a 1 from  $a_{i_1 i_1}$  to  $a_{i_2 i_1}$  as well as from  $a_{i_2 i_2}$  to  $a_{i_1 i_2}$ , preserving symmetry (i.e.  $P_{i_1 i_2}^T = P_{i_1 i_2}$ )
- To swap the first and third unknowns:  $Ac = AP_{13}P_{13}c = (AP_{13})(P_{13}c)$  where  $P_{13}c$  row swaps the unknowns and  $AP_{13}$  swaps the columns
  - To understand  $AP_{13}$ , consider  $(AP_{13})^{TT} = (P_{13}A^T)^T$  where  $P_{13}A^T$  swaps the rows of  $A^T$

# LU Factorization with Pivoting

- Let  $P_{r_i}$  be the permutation matrix that switches row  $i$  with a row  $> i$  when desired
- Let  $P_{c_k}$  be the permutation matrix that switches column  $k$  with a col  $> k$  when desired
- Then, full pivoting can be written as:

$$(M_{n-1,n-1}P_{r_{n-1}} \cdots M_{22}P_{r_2}M_{11}P_{r_1}AP_{c_1}P_{c_2} \cdots P_{c_{n-1}})(P_{c_{n-1}} \cdots P_{c_2}P_{c_1}c)$$

- Once known,  $P_r = P_{r_{n-1}} \cdots P_{r_2}P_{r_1}$  and  $P_c = P_{c_{n-1}} \cdots P_{c_2}P_{c_1}$  can be used to do all the permutations ahead of time (the resulting matrix doesn't require pivoting)
- Thus,  $Ac = b$  becomes  $(P_rAP_c^T)(P_cc) = P_rb$  or  $A_Pc_P = b_P$ ; then,  $A_P = L_PU_P$  can be computed without pivoting
- Subsequently, given any right-hand side  $b$ , solve  $L_PU_Pc_P = P_rb$  to find  $c_P$  using forward/back substitution (as usual); then,  $c = P_c^T c_P$

# Permuting before Elimination

- Assume  $i > j$ ,

$$\begin{aligned}P_{r_i} M_{jj} P_{r_i} &= I_{n \times n} - P_{r_i} m_{jj} \hat{e}_j^T P_{r_i} = I_{n \times n} - \hat{m}_{jj} \hat{e}_j^T = \hat{M}_{jj} \\P_{r_i} M_{jj} &= P_{r_i} M_{jj} P_{r_i} P_{r_i} = \hat{M}_{jj} P_{r_i}\end{aligned}$$

- Thus, for some suitable definition of the hat notation (there are multiple permutation operators to consider for most  $M_{jj}$ ):

$$M_{n-1,n-1} P_{r_{n-1}} \cdots M_{22} P_{r_2} M_{11} P_{r_1} A = M_{n-1,n-1} \cdots \hat{M}_{22} \hat{M}_{11} P_r A$$

- Thus, you can permute first and do elimination afterwards

# Sparsity

- Most large matrices (of interest) operate on variables that only interact with a sparse set of other variables
- This makes the matrix sparse (as opposed to dense), with most entries identically 0
- Unfortunately, the inverse of a sparse matrix can contain an unwieldy amount of non-zero entries
- E.g. the 3D Poisson equation on a relatively small  $100^3$  Cartesian grid has a single unknown for each of the  $10^6$  grid points
- For each unknown, the discretized Poisson equation depends on the unknown itself and its 6 immediate Cartesian grid neighbors
- Thus, the size  $10^6 \times 10^6$  matrix only has  $7 \times 10^6$  nonzero entries
- But, the inverse could have as many as  $10^{12}$  nonzero entries!



# Computing the Inverse

- When  $A$  is relatively small (and dense), computing  $A^{-1}$  is fine
- Since  $AA^{-1} = I$ , the solution  $c_k$  to  $Ac_k = \hat{e}_k$  is the  $k$ -th column of  $A^{-1}$

Method:

- First, compute  $A_P = L_P U_P$  as usual
- Then, solve  $Ac_k = \hat{e}_k$  once for each column (i.e.,  $n$  times)

# Unit 3

# Understanding Matrices

# Eigensystems

- Eigenvectors - special directions  $v_k$  in which a matrix only applies scaling
- Eigenvalues - the amount  $\lambda_k$  of that scaling
- Right Eigenvectors (or just eigenvectors) satisfy  $Av_k = \lambda_k v_k$ 
  - **Eigenvectors represent directions**, so  $A(\alpha v_k) = \lambda_k(\alpha v_k)$  is also true for all  $\alpha$
- Left Eigenvectors satisfy  $u_k^T A = \lambda_k u_k^T$  (or  $A^T u_k = \lambda_k u_k$ )
- Diagonal matrices have eigenvalues on the diagonal, and eigenvectors  $\hat{e}_k$

$$\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 3 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- Upper/lower triangular matrices also have eigenvalues on the diagonal

$$\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = 3 \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$



# Complex Numbers

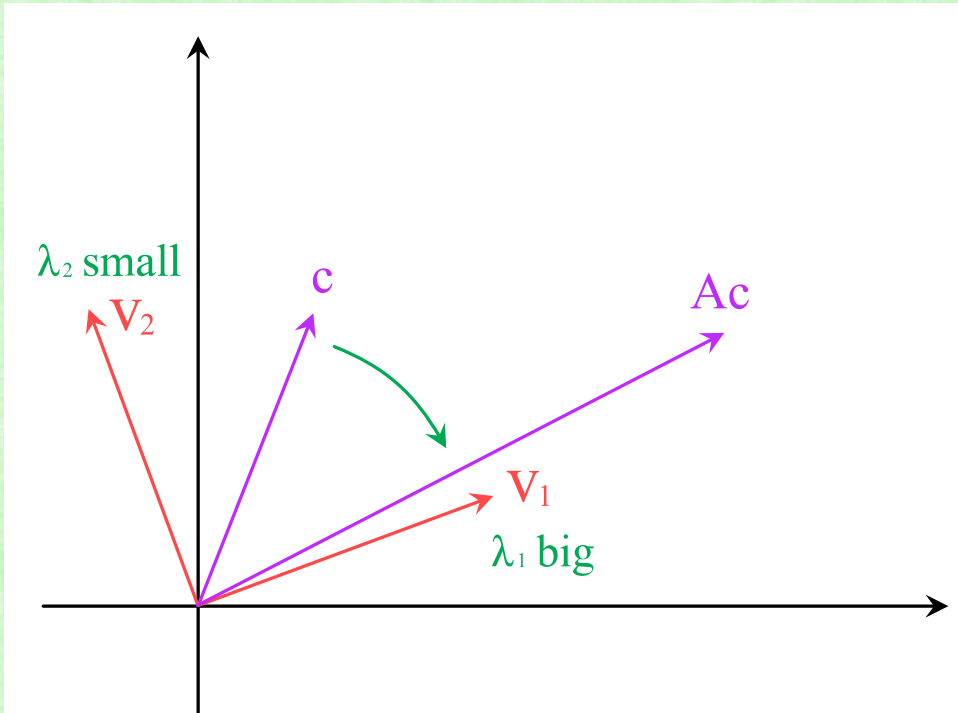
- Complex numbers may appear in both eigenvalues and eigenvectors

$$\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ i \end{pmatrix} = i \begin{pmatrix} 1 \\ i \end{pmatrix}$$

- Recall: complex conjugate:  $(a + bi)^* = a - bi$
- Hermitian Matrix:  $A^{*T} = A$  (often,  $A^{*T}$  is written as  $A^H$ )
  - $Av = \lambda v$  implies  $(Av)^{*T} = (\lambda v)^{*T}$  or  $v^{*T} A = \lambda^* v^{*T}$
  - Using this,  $Av = \lambda v$  implies  $v^{*T} A v = v^{*T} \lambda v$  or  $\lambda^* v^{*T} v = \lambda v^{*T} v$  or  $\lambda^* = \lambda$
  - Thus, Hermitian matrices have  $\lambda \in \mathbb{R}$  (no complex eigenvalues)
- Symmetric real-valued matrices have real-valued eigenvalues/eigenvectors
  - However, complex eigenvectors work too, e.g.  $A(\alpha v_k) = \lambda_k(\alpha v_k)$  with  $\alpha$  complex

# Vector Deformation

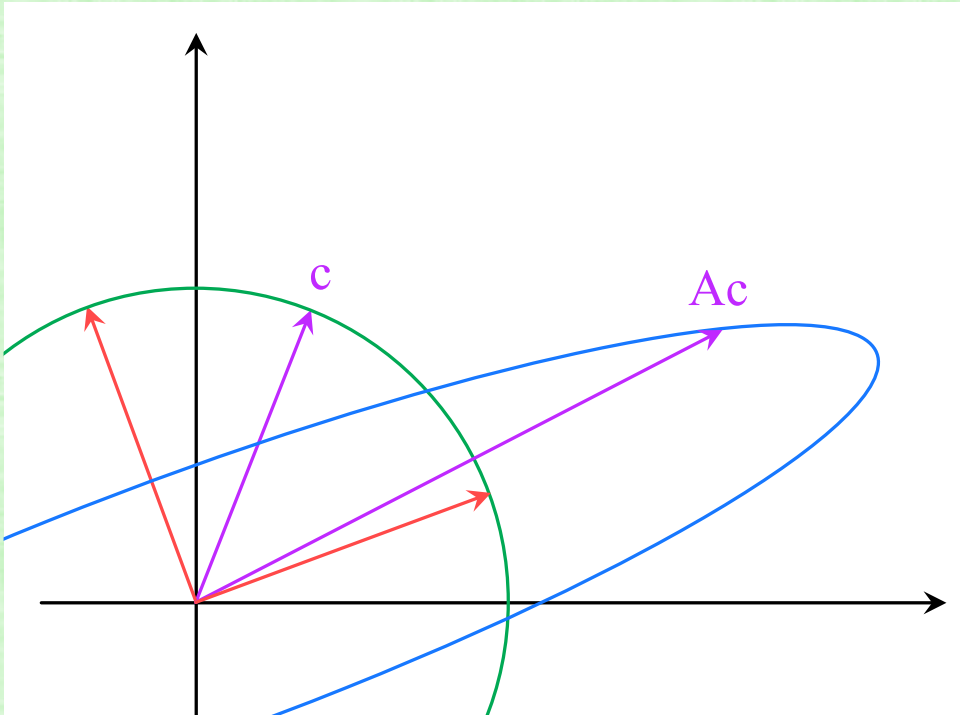
- Let  $c = \sum_k \alpha_k v_k$ , so that  $Ac = \sum_k \alpha_k A v_k = \sum_k (\alpha_k \lambda_k) v_k$
- $A$  tilts  $c$  away from directions with smaller eigenvalues and towards directions with larger eigenvalues



- Large  $\lambda_k$  stretch in their associated  $v_k$  directions
- Small  $\lambda_k$  squish in their associated  $v_k$  directions
- Negative  $\lambda_k$  flip the sign (i.e. direction) in their associated  $v_k$  directions

# Spatial Deformation

- Consider every point on the unit circle (green) as a vector  $c = \sum_k \alpha_k v_k$ , and remap each point via  $Ac = \sum_k (\alpha_k \lambda_k) v_k$

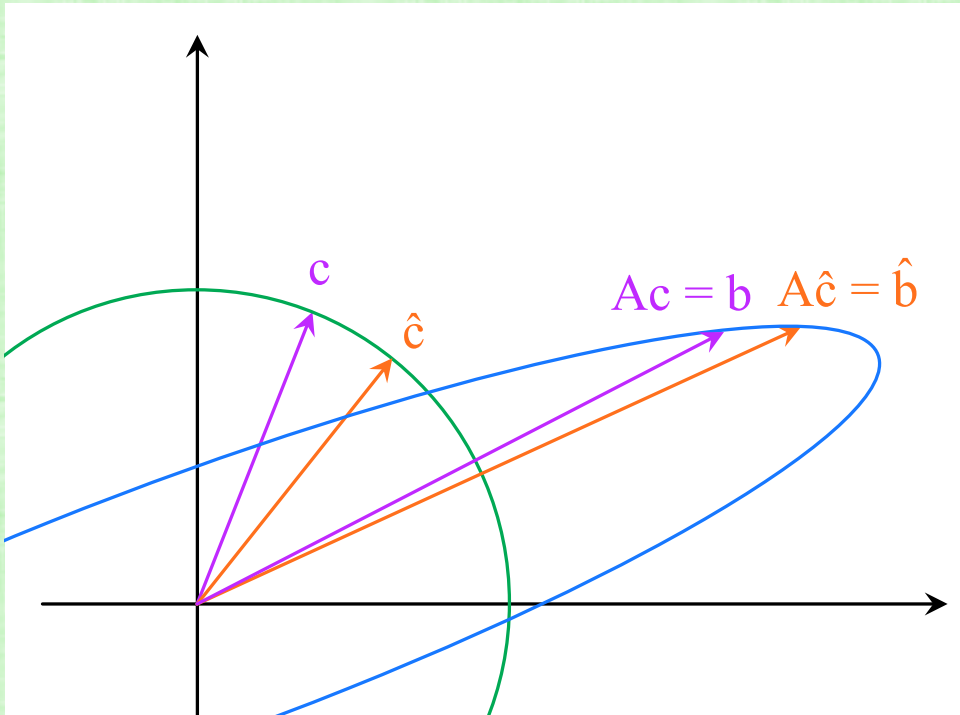


- The remapped shape (blue) is more elliptical than the original circle (green)
- The circle is stretched/compressed along the (red) axis with the larger/smaller eigenvalue, respectively
- The larger the ratio of eigenvalues, the more elliptical the new shape becomes
- This is true for all circles (and thus all points in the plane)



# Implications for Solving Linear Systems

- Perturb the right-hand side from  $b$  to  $\hat{b}$ , and solve  $A\hat{c} = \hat{b}$  to find  $\hat{c}$
- Note:  $c$  and  $\hat{c}$  are more separated than  $b$  and  $\hat{b}$ , i.e. the solution is perturbed more than the right-hand side was perturbed



- Small changes in  $b$  lead to larger changes in the solution
- **Small algorithmic errors are also amplified:** they change  $A^{-1}b$  to  $\hat{A}^{-1}b$ , which is similar to changing  $A^{-1}b$  to  $A^{-1}\hat{b}$
- The amount of amplification is proportional to the ratio of the eigenvalues



# Aside: Preconditioning

- Suppose  $A$  has large eigenvalue ratios, making  $Ac = b$  difficult to solve
- Let  $\hat{A}^{-1} \approx A^{-1}$  be an approximate guess for the inverse
- Transform  $Ac = b$  into  $\hat{A}^{-1}Ac = \hat{A}^{-1}b$  or  $\hat{I}c = \tilde{b}$ 
  - Typically, a bit more involved than this (but conceptually similar)
- $\hat{I}$  is not the identity, so more computation is required to find  $c$
- But,  $\hat{I}$  has similar magnitude eigenvalues (clusters work too), making  $\hat{I}c = \tilde{b}$  far easier to solve than a poorly conditioned  $Ac = b$

Preconditioning works GREAT!

- It is best to re-scale stretched ellipsoids along eigenvector axes, but scaling along coordinate axes (diagonal/Jacobi preconditioning) can work well too

# Rectangular Matrices (Rank)

- An  $m \times n$  rectangular matrix has  $m$  rows and  $n$  columns
  - Note: these comments also hold for square matrices with  $m = n$
- The columns span a space, and the unknowns are weights on each column (recall  $Ac = \sum_k c_k a_k$  )
- A matrix with  $n$  columns has maximum rank  $n$
- The actual rank depends on how many of the columns are linearly independent from one another
- Each column has length  $m$  (the number of rows)
- Since the columns live in an  $m$  dimensional space, they can at best span that whole space
- Thus, there is a maximum of  $m$  independent columns (that could exist)
- Overall, a matrix **at most** has rank equal to the minimum of  $m$  and  $n$
- Both considerations are based on looking at the columns

# Rows vs. Columns

- One can find discussions on rows, row spaces, etc. that are used for various purposes
- Although these are fine mathematical discussions, they are unnecessary for an intuitive understanding of high dimensional vector spaces (so, we'll ignore them)
- The number of columns is equal to the number of variables, which depends on the parameters of the problem
  - E.g. the unknown parameters that govern a neural network architecture
- The number of rows depends on the amount of data used, and adding/removing data does not intrinsically affect the nature of the problem
  - E. g. it does not change the network architecture, but merely perturbs the ascertained values of the unknown parameters



# Singular Value Decomposition (SVD)

- Factorization of any (size  $m \times n$ ) matrix:  $A = U \Sigma V^T$
- $\Sigma$  is  $m \times n$  diagonal with non-negative diagonal entries (called singular values)
- $U$  is  $m \times m$  orthogonal,  $V$  is  $n \times n$  orthogonal (their columns are called singular vectors)
  - Orthogonal matrices have orthonormal columns (an orthonormal basis), so their transpose is their inverse. They preserve inner products and thus are rotations, reflections, and combinations thereof
  - If  $A$  has complex entries, then  $U$  and  $V$  are unitary (conjugate transpose is their inverse)
- Introduced and rediscovered many times: Beltrami 1873, Jordan 1875, Sylvester 1889, Autonne 1913, Eckart and Young 1936. Pearson introduced principal component analysis (PCA) in 1901, which uses SVD. Numerical methods by Chan, Businger, Golub, Kahan, etc.



# (Rectangular) Diagonal Matrices

- All off-diagonal entries are 0
  - Diagonal entries are  $a_{kk}$ , and off diagonal entries are  $a_{ik}$  with  $i \neq k$
- E.g.  $\begin{pmatrix} 5 & 0 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 10 \\ -1 \\ \alpha \end{pmatrix}$  has  $5c_1 = 10$  and  $2c_2 = -1$ , so  $c_1 = 2$  and  $c_2 = -.5$ 
  - Note:  $\alpha \neq 0$  imposes a “no solution” condition (even though  $c_1$  and  $c_2$  are well-specified)
- E.g.  $\begin{pmatrix} 5 & 0 & 0 \\ 0 & 2 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 10 \\ -1 \end{pmatrix}$  has  $5c_1 = 10$  and  $2c_2 = -1$ , so  $c_1 = 2$  and  $c_2 = -.5$ 
  - Note: there are “infinite solutions” for  $c_3$  (even though  $c_1$  and  $c_2$  are well-specified)
- A zero on the diagonal indicates a singular system, leading to “no solution” (e.g.  $0c_1 = 10$ ) or “infinite solutions” (e.g.  $0c_1 = 0$ )

# Singular Value Decomposition (SVD)

- $A^T A = V \Sigma^T U^T U \Sigma V^T = V(\Sigma^T \Sigma) V^T$ , so  $(A^T A)v = \lambda v$  gives  $(\Sigma^T \Sigma)(V^T v) = \lambda(V^T v)$
- $\Sigma^T \Sigma$  is  $n \times n$  diagonal with eigenvectors  $\hat{e}_k$ , so  $\hat{e}_k = V^T v$  and  $v = V \hat{e}_k$
- That is, **the columns of  $V$  are the eigenvectors of the square  $A^T A$**
- $AA^T = U \Sigma V^T V \Sigma^T U^T = U(\Sigma \Sigma^T) U^T$ , so  $(AA^T)v = \lambda v$  gives  $(\Sigma \Sigma^T)(U^T v) = \lambda(U^T v)$
- $\Sigma \Sigma^T$  is  $m \times m$  diagonal with eigenvectors  $\hat{e}_k$ , so  $\hat{e}_k = U^T v$  and  $v = U \hat{e}_k$
- That is, **the columns of  $U$  are the eigenvectors of the square  $AA^T$**
- When  $m \neq n$ , either  $\Sigma^T \Sigma$  or  $\Sigma \Sigma^T$  is larger and contains extra zeros on the diagonal
- Their other diagonal entries are the squares of the singular values
- That is, **the singular values are the (non-negative) square roots of the non-extra eigenvalues of  $A^T A$  and  $AA^T$**
- Both  $A^T A$  and  $AA^T$  are symmetric positive semi-definite, and thus easy to work with
- E.g. symmetry means their eigensystem (and thus the SVD) has no complex numbers when  $A$  has none

## Example (Tall Matrix)

- Consider size  $4 \times 3$  matrix  $A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix}$
- Label the columns  $a_1 = \begin{pmatrix} 1 \\ 4 \\ 7 \\ 10 \end{pmatrix}$ ,  $a_2 = \begin{pmatrix} 2 \\ 5 \\ 8 \\ 11 \end{pmatrix}$ ,  $a_3 = \begin{pmatrix} 3 \\ 6 \\ 9 \\ 12 \end{pmatrix}$
- Since  $a_1$  and  $a_2$  point in different directions,  $A$  is at least rank 2
- Since  $a_3 = 2a_2 - a_1$ , the third column is in the span of the first two columns
- Thus,  $A$  is only rank 2 (not rank 3)

## Example (SVD)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix}$$

- Singular values are 25.5, 1.29, and 0
- Singular value of 0 indicates that the matrix is rank deficient (only rank 2, not rank 3)
- The rank of a matrix is equal to its number of nonzero singular values



# Derivation from $A^T A$ and $AA^T$

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix}$$

- $A^T A$  is size  $3 \times 3$  and has 3 eigenvectors (columns of  $V$ )
- The square roots of the 3 eigenvalues of  $A^T A$  are in  $\Sigma$  (color coded to the eigenvectors)
- $AA^T$  is size  $4 \times 4$  and has 4 eigenvectors (columns of  $U$ )
- The square roots of 3 of the eigenvalues of  $AA^T$  are in  $\Sigma$ 
  - The 4<sup>th</sup> eigenvalue of  $AA^T$  is an extra eigenvalue of 0

# Understanding $Ac$

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix}$$

- $A$  maps from  $R^3$  to  $R^4$
- $Ac$  first projects  $c \in R^3$  onto the 3 basis vectors in  $V$
- Then, the associated singular values (diagonally) scale the results
- Lastly, those scaled results are used as weights on the basis vectors in  $U$

# Understanding $Ac$

$$\begin{aligned}
 Ac &= \begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} \\
 &= \begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} v_1^T c \\ v_2^T c \\ v_3^T c \end{pmatrix} \\
 &= \begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} \sigma_1 v_1^T c \\ \sigma_2 v_2^T c \\ \sigma_3 v_3^T c \\ 0 \end{pmatrix} \\
 &= u_1 \sigma_1 v_1^T c + u_2 \sigma_2 v_2^T c + u_3 \sigma_3 v_3^T c + u_4 0
 \end{aligned}$$

- $Ac$  projects  $c$  onto the basis vectors in  $V$ , scales by the associated singular values, and uses those results as weights on the basis vectors in  $U$

# Extra Dimensions

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.420 & -.51 \\ .344 & .426 & .298 & .78 \\ .547 & .028 & .644 & -.59 \\ .750 & -.371 & -.542 & .09 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ \text{[redacted]} \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix}$$

- The 3D space of vector inputs can only span a 3D subspace of  $R^4$
- The last (green) column of  $U$  represents the unreachable dimension, orthogonal to the range of  $A$ , and is always multiplied by 0
- One can delete this column and the associated portion of  $\Sigma$  (and still obtain a valid factorization)



# Zero Singular Values

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.410 & -.551 \\ .344 & .426 & .218 & .781 \\ .547 & .028 & .646 & -.519 \\ .750 & -.371 & -.512 & .019 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 & 0 \\ 0 & 1.29 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .133 & .133 & .133 \\ .133 & .133 & .133 \end{pmatrix}$$

- The 3<sup>rd</sup> singular value is 0, so  $A$  has a 1D null space that reduces the 3D input vectors to only 2 dimensions
- The associated (pink) terms make no contribution to the final result, and can also be deleted (still obtaining a valid factorization)
- The first 2 columns of  $U$  span the 2D subset of  $R^4$  that comprises the range of  $A$

# Approximating $A$

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} \approx$$

$$\begin{pmatrix} .141 & .815 & -.400 & -.551 \\ .344 & .416 & .213 & .781 \\ .547 & .013 & .641 & -.519 \\ .750 & -.371 & -.512 & .019 \end{pmatrix} \begin{pmatrix} 25.5 & & & \\ & & & \\ & & & \\ & & & \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.781 & .037 & .016 \\ .103 & .010 & .103 \end{pmatrix}$$

- The first singular value is much bigger than the second, and so represents the vast majority of what  $A$  does (note, the vectors in  $U$  and  $V$  are unit length)
- Thus, one could approximate  $A$  quite well by only using the terms associated with the largest singular value
- This is not a valid factorization, but an **approximation** (and the idea behind PCA)

# Summary

- The columns of  $V$  that do not correspond to “nonzero” singular values form an orthonormal basis for the null space of  $A$
- The remaining columns of  $V$  form an orthonormal basis for the space perpendicular to the null space of  $A$  (parameterizing meaningful inputs)
- The columns of  $U$  corresponding to “nonzero” singular values form an orthonormal basis for the range of  $A$
- The remaining columns of  $U$  form an orthonormal basis for the (unattainable) space perpendicular to the range of  $A$
- One can drop the columns of  $U$  and  $V$  that do not correspond to “nonzero” singular values and still obtain a valid factorization of  $A$
- One can drop the columns of  $U$  and  $V$  that correspond to “smaller” singular values and still obtain a reasonable approximation of  $A$

## Example (Wide Matrix)

$$A = \begin{pmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{pmatrix} = \begin{pmatrix} .504 & -.761 & .408 \\ .574 & -.057 & -.816 \\ .644 & .646 & .408 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 & 0 \\ 0 & 1.29 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .141 & .344 & .547 & .750 \\ .825 & .426 & .028 & -.371 \\ -.420 & .298 & .644 & -.542 \\ -.351 & .782 & -.509 & .079 \end{pmatrix}$$

- $A$  maps from  $R^4$  to  $R^3$  and so has at least a 1D null space (green)
- The 3<sup>rd</sup> singular value is 0, and the associated (pink) terms make no contribution to the final result (so the null space is actually 2D)



# Example (Wide Matrix)

$$A = \begin{pmatrix} 1 & 4 & 7 & 10 \\ 2 & 5 & 8 & 11 \\ 3 & 6 & 9 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .504 & -.761 & .418 & .141 \\ .574 & -.057 & -.816 & .344 \\ .644 & .646 & .418 & .547 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 & 0 \\ 0 & 1.29 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .141 & .344 & .547 & .750 \\ .825 & .426 & .028 & -.371 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

- Only a 2D subspace (of inputs) in  $R^4$  matters, with the rest of  $R^4$  in the null space of  $A$
- Only a 2D subspace (of outputs) in  $R^3$  is in the range of  $A$

# More Theoretical than Practical

- The SVD is often unwieldy for computational purposes
- However, replacing matrices by their SVD can be quite useful/enlightening for theoretical pursuits
- Moreover, its theoretical underpinnings are often used to devise robust and computational algorithms

# SVD Uniqueness

- Mandating that all the  $\sigma_k \geq 0$  be listed in descending order makes  $\Sigma$  unique
- Non-repeated  $\sigma_k$  have unique singular vectors in both  $U$  and  $V$ 
  - up to a  $\pm 1$  choice of sign (as discussed in the next two slides)
- Repeated  $\sigma_k$  lead to non-unique singular vectors, since there are an infinite number of ways to span the associated subspace
  - This non-uniqueness stems from the non-uniqueness in eigensystems with repeated eigenvalues
  - This non-uniqueness wreaks havoc on differentiability of the SVD

# SVD Construction (an important detail)

- Let  $A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$  so that  $A^T A = A A^T = I$ , and thus  $U = V = \Sigma = I$
- But  $A \neq U \Sigma V^T = I$  **What's wrong?**
- Given a column vector  $v_k$  of  $V$ ,  $A v_k = U \Sigma V^T v_k = U \Sigma \hat{e}_k = U \sigma_k \hat{e}_k = \sigma_k u_k$  where  $u_k$  is the corresponding column of  $U$
- $A v_1 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = u_1$  but  $A v_2 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \neq \begin{pmatrix} 0 \\ 1 \end{pmatrix} = u_2$
- Since  $U$  and  $V$  are orthonormal, their columns are unit length; however, there are still **two choices for the direction of each column**
- Multiplying  $u_2$  by  $-1$  to get  $u_2 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$  makes  $U = A$ , and thus  $A = U \Sigma V^T$  as desired



# SVD Construction Strategy

- An orthogonal matrix has determinant equal to  $\pm 1$ , where  $-1$  indicates a reflection of the coordinate system
- If  $\det V = -1$ , flip the direction of any column to make  $\det V = 1$  (so  $V$  does not contain a reflection)
  - Regarding uniqueness: subsequently flipping any even number of columns preserves  $\det V = 1$
- Then, for each  $v_k$ , compare  $Av_k$  to  $\sigma_k u_k$  and flip the direction of  $u_k$  if necessary to make  $Av_k = \sigma_k u_k$
- When  $\det U = -1$ , it contains a reflection
  - When  $m > n$ , this can be fixed by flipping a column of  $U$  that does not correspond to a singular value
  - When  $m < n$ , flip a column of  $U$  to make  $\det U = 1$  along with the corresponding column  $V$ ; then, flip a column of  $V$  that does not correspond to a singular value
  - When  $m = n$ , column flips cannot remove the intrinsic reflection
- When  $\det U = -1$  with  $m = n$ , one can flip the sign of the smallest singular value in  $\Sigma$  to be negative, while also flipping the direction of the corresponding column in  $U$ 
  - This moves the reflection from  $U$  to  $\Sigma$  and is called the polar-SVD
  - See "Invertible Finite Elements For Robust Simulation of Large Deformation", Irving et al. 2004

# Solving Linear Systems

- $Ac = b$  becomes  $U\Sigma V^T c = b$  or  $\Sigma(V^T c) = (U^T b)$  or  $\Sigma\hat{c} = \hat{b}$ 
  - The unknowns  $c$  are remapped into the space spanned by  $V$
  - The right-hand side  $b$  is remapped into the space spanned by  $U$
- Every matrix is a diagonal matrix, when viewed in the right space
- Solve the diagonal system  $\Sigma\hat{c} = \hat{b}$  by dividing the entries of  $\hat{b}$  by the singular values  $\sigma_k$ ; then,  $c = V\hat{c}$
- The SVD transforms the problem into an inherently diagonal space with eigenvectors along the coordinate axes
- Circles becoming ellipses (discussed earlier) is still problematic
  - Eccentricity is caused by ratios of singular values (since  $U$  and  $V$  are orthogonal)

# Condition Number

- The condition number of  $A$  is  $\frac{\sigma_{max}}{\sigma_{min}}$ , and it measures closeness to being singular
- For a square  $A$ , it measures the difficulty in solving  $Ac = b$
- For a rectangular (and square)  $A$ , it measures how close the columns are to being linearly dependent
  - For a wide (rectangular)  $A$ , it ignores the extra columns that must be linearly dependent (which is fine, because the associated variables lack any data)
- The condition number does not depend on the right-hand side
- The condition number is always bigger than 1, and approaches  $\infty$  for nearly singular matrices
- The condition number is  $\infty$  for singular matrices, since  $\sigma_{min} = 0$



# Understanding Singular Matrices

- Diagonalize  $Ac = b$  to  $\Sigma(V^T c) = (U^T b)$ , e.g.  $\begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{pmatrix} \begin{pmatrix} \hat{c}_1 \\ \hat{c}_2 \end{pmatrix} = \begin{pmatrix} \hat{b}_1 \\ \hat{b}_2 \end{pmatrix}$  with  $\hat{c}_1 = \frac{\hat{b}_1}{\sigma_1}$ ,  $\hat{c}_2 = \frac{\hat{b}_2}{\sigma_2}$
- When  $\sigma_1 \neq 0$  and  $\sigma_2 = 0$ ; then, there is no unique solution:
  - When  $\hat{b}_2 = 0$ , there are infinite solutions for  $\hat{c}_2$  (but  $\hat{c}_1$  is well determined)
  - When  $\hat{b}_2 \neq 0$ , there is no solution for  $\hat{c}_2$ , and  $b$  is not in the range of  $A$  (but  $\hat{c}_1$  is well determined)
- Consider:  $\begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \hat{c}_1 \\ \hat{c}_2 \end{pmatrix} = \begin{pmatrix} \hat{b}_1 \\ \hat{b}_2 \\ \hat{b}_3 \end{pmatrix}$  with  $\hat{c}_1 = \frac{\hat{b}_1}{\sigma_1}$ ,  $\hat{c}_2 = \frac{\hat{b}_2}{\sigma_2}$ 
  - When  $\hat{b}_3 = 0$ , the last row adds no new information (it's extra redundant data)
  - When  $\hat{b}_3 \neq 0$ , the last row is false and there is no solution (but  $\hat{c}_1$  and  $\hat{c}_2$  are well determined)
- Consider:  $\begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{pmatrix} \begin{pmatrix} \hat{c}_1 \\ \hat{c}_2 \\ \hat{c}_3 \end{pmatrix} = \begin{pmatrix} \hat{b}_1 \\ \hat{b}_2 \end{pmatrix}$  with  $\hat{c}_1 = \frac{\hat{b}_1}{\sigma_1}$ ,  $\hat{c}_2 = \frac{\hat{b}_2}{\sigma_2}$ 
  - Infinite solutions work for  $\hat{c}_3$  (but  $\hat{c}_1$  and  $\hat{c}_2$  are well determined)



# Understanding Variables

Consider any column  $k$  of  $\Sigma$ :

- When  $\sigma_k \neq 0$ , a unique value can be determined for  $\hat{c}_k$
- When  $\sigma_k = 0$  or there is no  $\sigma_k$  (wide matrices), the data contains no information for  $\hat{c}_k$ 
  - This does not mean that other variables cannot be adequately determined!

Consider a row  $i$  of  $\Sigma$  that is identically zero:

- When  $\hat{b}_i = 0$ , this row indicates that there is extra (redundant) data
- When  $\hat{b}_i \neq 0$ , this row indicates that the data contains conflicting information
  - Conflicting information doesn't necessarily imply that all is lost, i.e. "no solution"; rather, it might merely mean that the data contains a bit of noise
  - Regardless, in spite of any conflicting information, the determinable  $\hat{c}_k$  represent the "best" that one can do for those variables

# Norms

- Common norms:  $\|c\|_1 = \sum_k |c_k|$ ,  $\|c\|_2 = \sqrt{\sum_k c_k^2}$ ,  $\|c\|_\infty = \max_k |c_k|$
- "All norms are interchangeable" is **only** valid **theoretically**
- In practice, the "worst case scenario" ( $L^\infty$ ) and the "average" ( $L^1$ ,  $L^2$ , etc.) are not interchangeable
- For example:  
$$(100 \text{ people} * 98.6^\circ + 1 \text{ person} * 105^\circ) / (101 \text{ people}) = 98.66^\circ$$
- Their average temperature is  $98.66^\circ$ , but everything is not "ok"

# Matrix Norms

- The norm of a matrix is defined via  $\|A\| = \max_{c \neq 0} \frac{\|Ac\|}{\|c\|}$ , so:
  - $\|A\|_1$  is the maximum absolute value column sum
  - $\|A\|_\infty$  is the maximum absolute value row sum
  - $\|A\|_2$  is the square root of the maximum eigenvalue of  $A^T A$ , i.e. the maximum singular value  $\sigma_{max}$  of  $A$
  - Since  $A^{-1} = V\Sigma^{-1}U^T$  and  $\Sigma^{-1}$  has diagonal entries  $\frac{1}{\sigma_k}$ ,  $\|A^{-1}\|_2 = \frac{1}{\sigma_{min}}$
- The condition number for solving  $Ac = b$  for a square matrix  $A$  is:

$$\|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{max}}{\sigma_{min}}$$

# Unit 4



# Special Matrices

# (Strict) Diagonal Dominance

- The magnitude of each diagonal element is (either):
  - strictly larger than the sum of the magnitudes of all the other elements in its row
  - strictly larger than the sum of the magnitudes of all the other elements in its column
- One may row/column scale and permute rows/columns to achieve diagonal dominance (since it's just a rewriting of the equations)
  - Recall: **choosing the form of the equations wisely is important**

- E.g. consider  $\begin{pmatrix} 3 & -2 \\ 5 & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 9 \\ 4 \end{pmatrix}$ 
  - Switch rows  $\begin{pmatrix} 5 & 1 \\ 3 & -2 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 9 \end{pmatrix}$  and column scale  $\begin{pmatrix} 5 & -2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} c_1 \\ -.5c_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 9 \end{pmatrix}$

# (Strict) Diagonal Dominance

- Strictly diagonally dominant (square) matrices are guaranteed to be non-singular
  - Since  $\det(A) = \det(A^T)$ , either row or column diagonal dominance is enough
- Column diagonal dominance guarantees that pivoting is not required during  $LU$  factorization
  - However, pivoting still improves robustness
  - E.g. consider  $\begin{pmatrix} 4 & 3 \\ -2 & 50 \end{pmatrix}$  where 50 is more desirable than 4 for  $a_{11}$

## Recall: SVD Construction (Unit 3)

- Let  $A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$  so that  $A^T A = A A^T = I$ , and thus  $U = V = \Sigma = I$
- But  $A \neq U \Sigma V^T = I$  What's wrong?
- Given a column vector  $v_k$  of  $V$ ,  $A v_k = U \Sigma V^T v_k = U \Sigma \hat{e}_k = U \sigma_k \hat{e}_k = \sigma_k u_k$  where  $u_k$  is the corresponding column of  $U$
- $A v_1 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = u_1$  but  $A v_2 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ -1 \end{pmatrix} \neq \begin{pmatrix} 0 \\ 1 \end{pmatrix} = u_2$
- Since  $U$  and  $V$  are orthonormal, their columns are unit length; however, there are still two choices for the direction of each column
- Multiplying  $u_2$  by  $-1$  to get  $u_2 = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$  makes  $U = A$ , and thus  $A = U \Sigma V^T$  as desired



# Symmetric Matrices

- Since  $A^T A = A A^T = A^2$ , both the columns of  $U$  and the columns of  $V$  are eigenvectors of  $A^2$
- They have identical (but **potentially opposite**) directions:  $u_k = \pm v_k$
- Thus,  $A v_k = \sigma_k u_k$  implies  $A v_k = \pm \sigma_k v_k$
- That is, the  $v_k$  (and  $u_k$ ) are eigenvectors of  $A$  with eigenvalues  $\pm \sigma_k$
- Pulling negative signs out of the columns of  $U$  and into the  $\sigma_k$  (similar to the polar-SVD) leads to  $U = V$  and  $A = V \Lambda V^T$  as a modified SVD
- $A = V \Lambda V^T$  implies  $AV = V \Lambda$ , which is the matrix form of the eigensystem of  $A$ 
  - $\Lambda$  contains the positive and negative eigenvalues of  $A$

# Making/Breaking Symmetry

- Row/column scaling can make or break symmetry:
  - Row scaling  $\begin{pmatrix} 5 & 3 \\ 3 & -4 \end{pmatrix}$  by 2 gives a non-symmetric  $\begin{pmatrix} 5 & 3 \\ \textcolor{red}{6} & -8 \end{pmatrix}$
  - Additional column scaling by 2 gives a symmetric  $\begin{pmatrix} 5 & \textcolor{teal}{6} \\ \textcolor{teal}{6} & -16 \end{pmatrix}$
- Scaling corresponding rows/columns together preserves symmetry
- Important: a nonsymmetric matrix might be inherently symmetric when properly rescaled/rearranged

# Symmetric Approximation

- A non-symmetric  $A$  can be approximated by a symmetric  $\hat{A} = \frac{1}{2}(A + A^T)$ , which averages off-diagonal components
- Solving the symmetric  $\hat{A}c = b$  instead of the non-symmetric  $Ac = b$  gives a faster/easier (but erroneous) approximation to  $c$ 
  - This is useful when  $c$  might not require too much accuracy
- The inverse of a symmetric  $\hat{A}$  (or the notion of the inverse) may be used to devise a preconditioner for  $Ac = b$

# Inner Product

- The dot/inner product of two vectors (of the same length) is  $u \cdot v = \sum_i u_i v_i$
- The magnitude of a vector is  $\|v\|_2 = \sqrt{v \cdot v} \ (\geq 0)$
- Alternative notations:  $\langle u, v \rangle = u \cdot v = u^T v$
- Weighted inner product defined via an  $m \times n$  matrix  $A$
- $\langle u, v \rangle_A = u \cdot Av = u^T Av$
- Since  $\langle u, v \rangle_A = u^T Av = v^T A^T u = \langle v, u \rangle_{A^T}$ , weighted inner products commute when  $A$  is symmetric
- The standard dot product uses identity matrix weighting:  $\langle u, v \rangle = \langle u, v \rangle_I$



# Definiteness

- Assume  $A$  is symmetric so that  $\langle u, v \rangle_A = \langle v, u \rangle_A$
- $A$  is positive definite if and only if  $\langle v, v \rangle_A = v^T A v > 0$  for  $\forall v \neq 0$
- $A$  is positive semi-definite if and only if  $\langle v, v \rangle_A = v^T A v \geq 0$  for  $\forall v \neq 0$
- We abbreviate with SPD and SP(S)D
- $A$  is negative definite if and only if  $\langle v, v \rangle_A = v^T A v < 0$  for  $\forall v \neq 0$
- $A$  is negative semi-definite if and only if  $\langle v, v \rangle_A = v^T A v \leq 0$  for  $\forall v \neq 0$
- If  $A$  is negative (semi) definite, then  $-A$  is positive (semi) definite (and vice versa)
- Thus, can convert such problems to SPD or SP(S)D
- $A$  is considered indefinite when it is neither positive/negative semi-definite

# Eigenvalues

- SPD matrices have all eigenvalues  $> 0$
- SP(S)D matrices have all eigenvalues  $\geq 0$
- Symmetric negative definite matrices have all eigenvalues  $< 0$
- Symmetric negative semi-definite matrices have all eigenvalues  $\leq 0$
- Indefinite matrices have both positive and negative eigenvalues

# SPD Matrices

- When  $A$  is SPD or SP(S)D,  $\Lambda = \Sigma$  and the standard SVD is  $A = V\Sigma V^T$  (i.e.  $U = V$ )

SPD matrices:

- The singular values are the (all positive) eigenvalues of  $A$
- Construct  $V$  with  $\det V = 1$  (as usual); then, all  $\sigma_k > 0$  implies that there are no reflections
- Since all  $\sigma_k > 0$ , SPD matrices have full rank and are invertible

SP(S)D matrices:

- Has at least one  $\sigma_k = 0$  and a null space
- Often, one can slightly modify SPD techniques for SP(S)D matrices
- Significantly easier to treat than indefinite matrices!

# Cholesky Factorization

- SPD matrices have an  $LU$  factorization of  $LL^T$ , which can be computed without elimination

- Consider  $\begin{pmatrix} a_{11} & a_{21} \\ a_{21} & a_{22} \end{pmatrix} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix}$

- So  $l_{11} = \sqrt{a_{11}}$  and  $l_{21} = \frac{a_{21}}{l_{11}}$  and  $l_{22} = \sqrt{a_{22} - l_{21}^2}$

for(j=1,n){

for(k=1,j-1) for(i=j,n)  $a_{ij} -= a_{ik}a_{jk};$

$a_{jj} = \sqrt{a_{jj}}; \text{ for}(k=j+1,n) a_{kj}/= a_{jj};\}$

\\ For each column j of the matrix

\\ Loop over all previous columns k, and subtract a multiple of column k from the current column j

\\ Take the square root of the diagonal entry, and scale column j by that value

\\ This factors the matrix “in place” replacing A with L



# Incomplete Cholesky Preconditioner

- Cholesky factorization can be used to construct a preconditioner for a sparse matrix
- The full Cholesky factorization would fill in too many non-zero entries
- So, incomplete Cholesky preconditioning uses Cholesky factorization with the **caveat** that only the nonzero entries are modified
  - That is, all zeros remain zeros

# Rules Galore

- There are many rules/theorems regarding special matrices (e.g. for SPD)
- It is important to be aware of reference material (and to look things up)
- Examples:
  - SPD matrices don't require pivoting during  $LU$  factorization
  - A symmetric (strictly) diagonally dominant matrix with positive diagonal entries is positive definite
  - Jacobi and Gauss-Seidel iteration converge when a matrix is strictly (or irreducibly) diagonally dominant
  - Etc.

# Unit 5

# Iterative Solvers



# Iterative vs. Direct Solvers

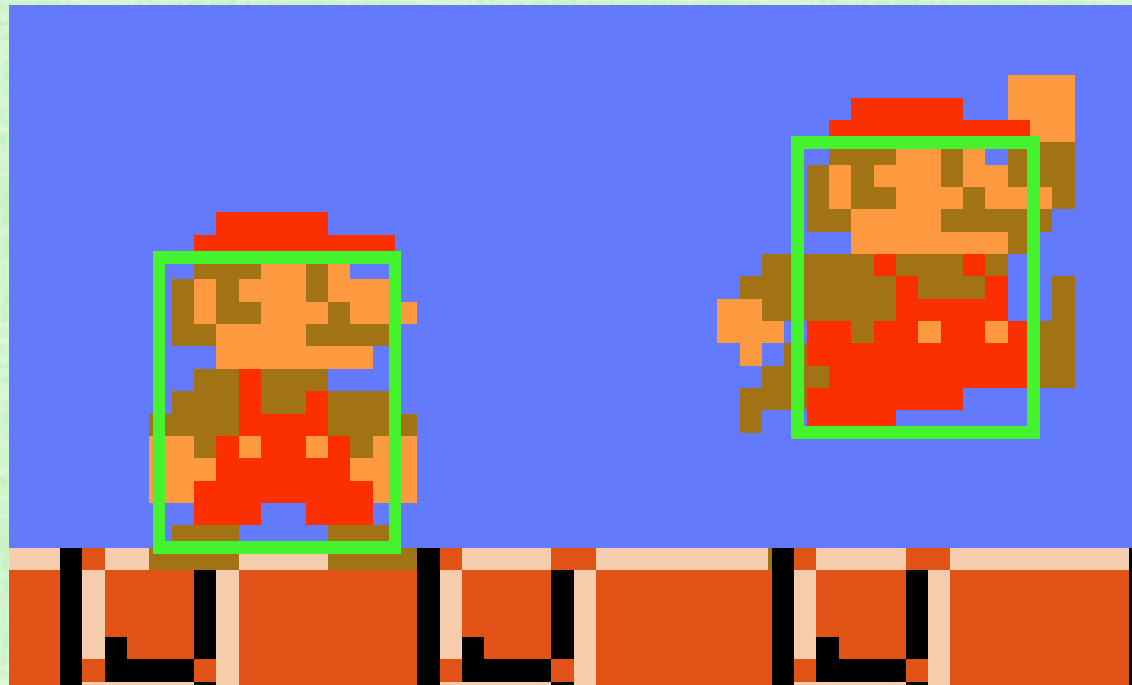
- Direct Solver/Method – closed form strategy, e.g. quadratic/Cardano formulas, Gaussian Elimination for  $LU$ , Cholesky factorization for  $LL^T$ , etc.
- Iterative Solver/Method
  - start with an initial guess:  $c^1$
  - use a recursive approach to improve that guess:  $c^2, c^3, c^4, \dots$
  - terminate based on stopping criteria, e.g. when error is small  $\|c^q - c^{exact}\| \leq \epsilon$
- A direct method can be used to obtain an initial guess
- Iterative methods are great for sparse matrices, as they often can ignore 0 entries
  - E.g. by formulating the method via the matrix's action (multiplication) on a vector
- Direct solvers are more commonly used on dense matrices
- Iterative solvers are used for training Neural Networks!

# Issues with Direct Methods

- (Recall) Quadratic formula loses precision, and can fail, when  $-b \pm \sqrt{b^2 - 4ac}$  has catastrophic cancellation
  - The de-rationalized quadratic formula instead uses  $-b \mp \sqrt{b^2 - 4ac}$
  - Using one formula for each root avoids catastrophic cancellation
- Cardano's formula for the roots of a cubic equation suffers from similar issues, but there is no straightforward fix
  - The computed roots too often have unacceptably high error
  - To highlight why one might need accurate cubic roots, let's consider collision detection...

# Hit Box

- In order to detect interactions between objects in video games, they were assigned hit boxes
- Anything inside an object's hit box can potentially interact with (i.e. hit) it



# Better Hit Boxes

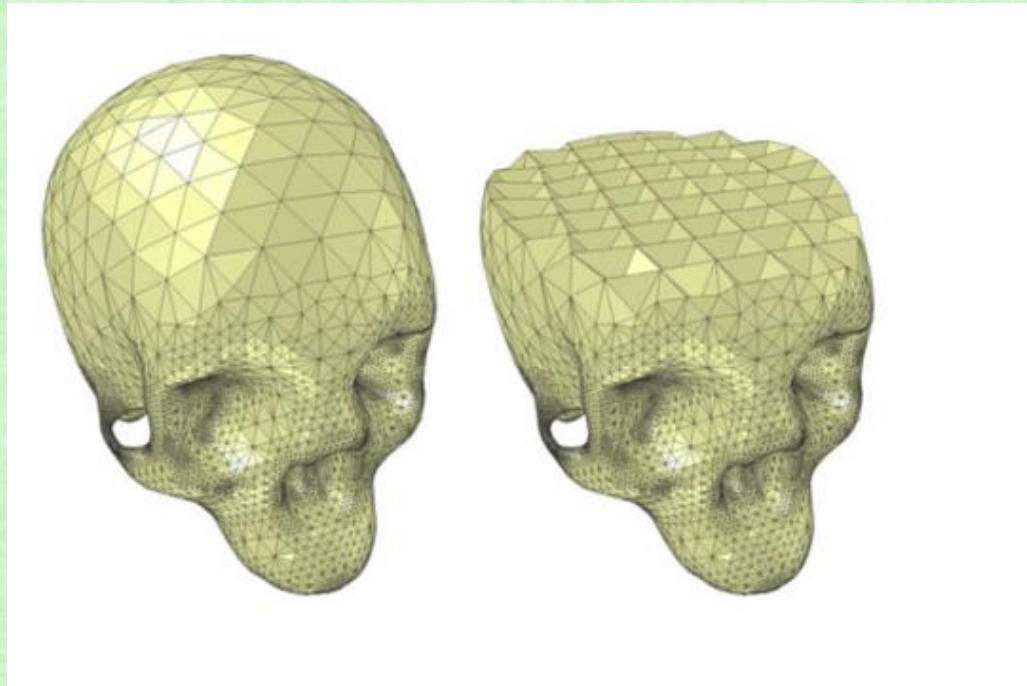
- These evolved over time to more complicated shapes in both 2D and 3D
  - e.g. spheres, ellipsoids, capsules, etc.
- Anything inside any of an object's hit boxes can potentially interact with it





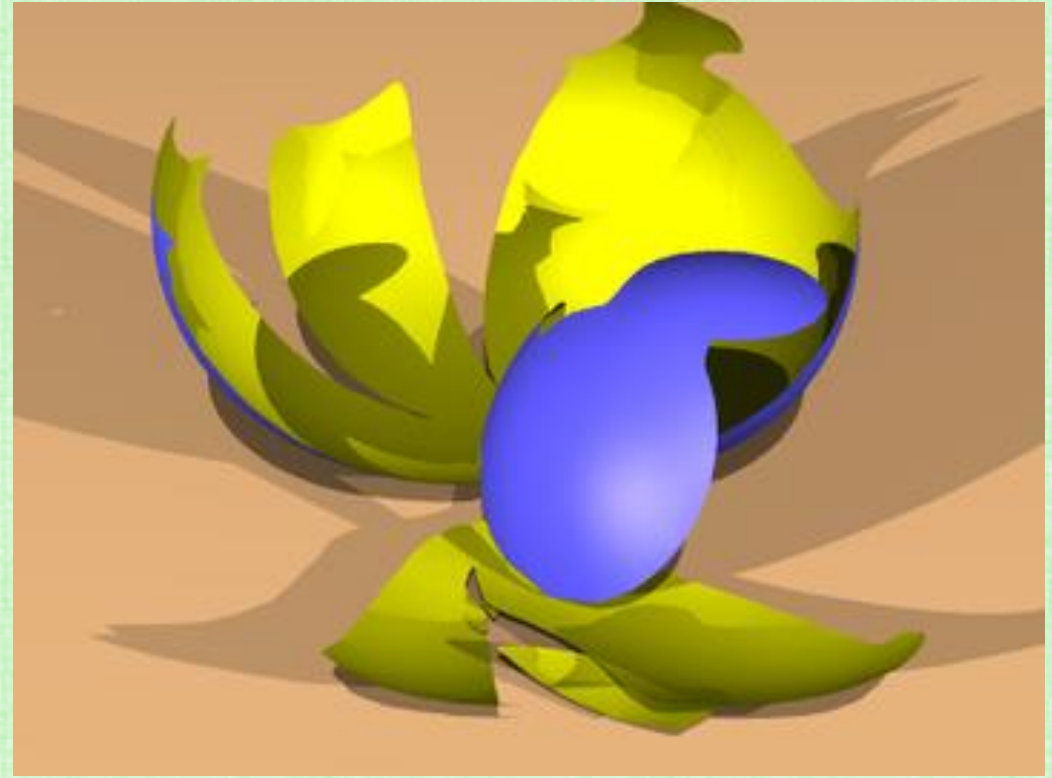
# Accurate Collision Detection

- More complex objects are often modeled by a triangulated surface mesh
- The interior can be filled with tetrahedra, or approximated with other objects
- Anything inside any of an object's interior structures can potentially interact with it



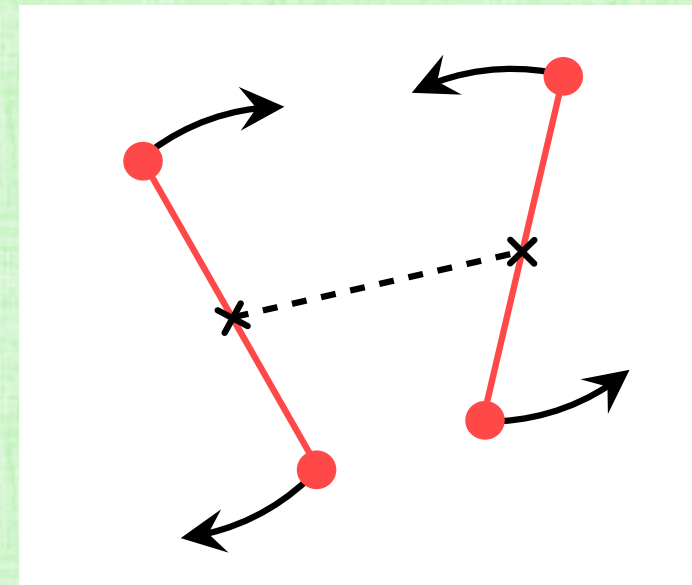
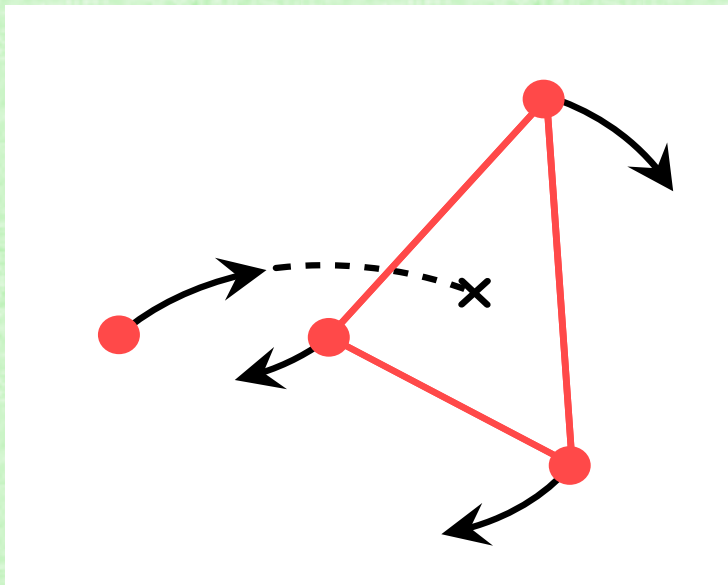
# Objects Without Interiors

- Very thin objects, such as cloth/shells, do not have an interior region
- One cannot use the same concept of inside to detect potential interactions



# Continuous Collision Detection (CCD)

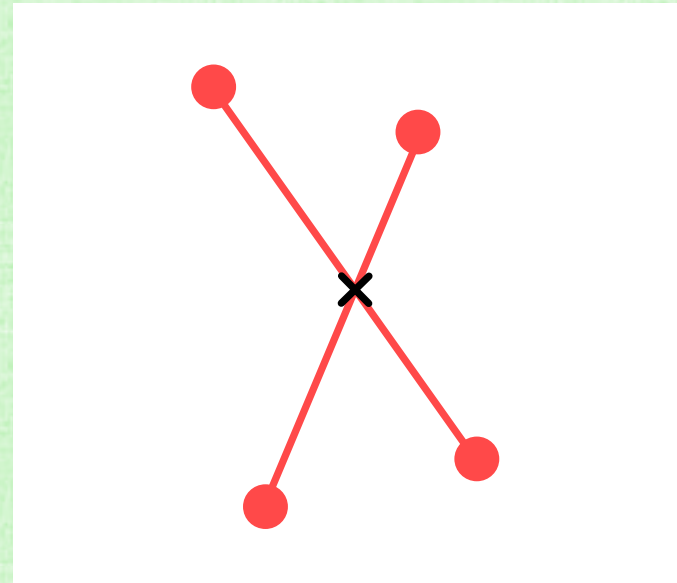
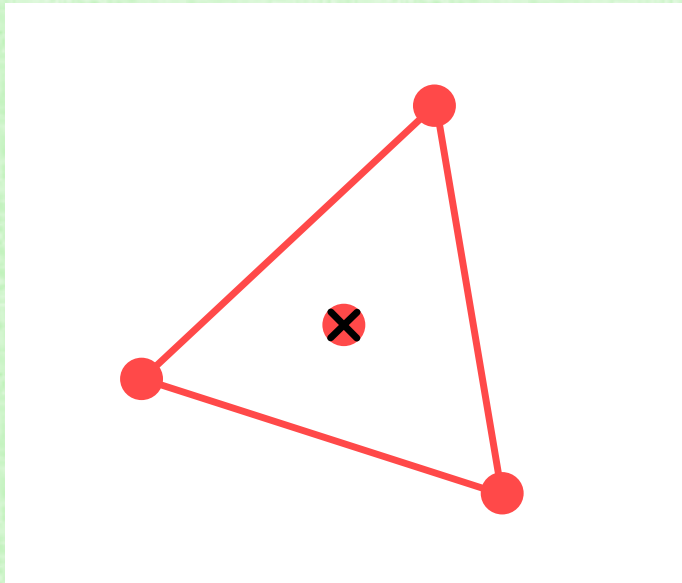
- Model the time varying trajectories of surface triangle vertices to see if/when triangles collide with each other
- Doesn't depend on the existence of an interior region
- There are two cases to consider: (1) Point-Face, (2) Edge-Edge





# Continuous Collision Detection (CCD)

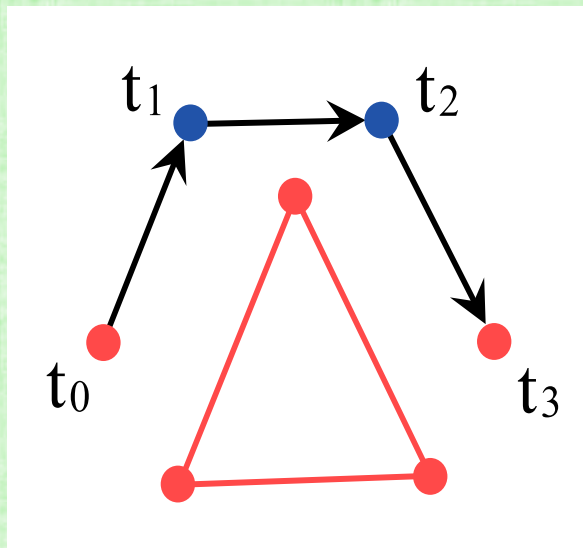
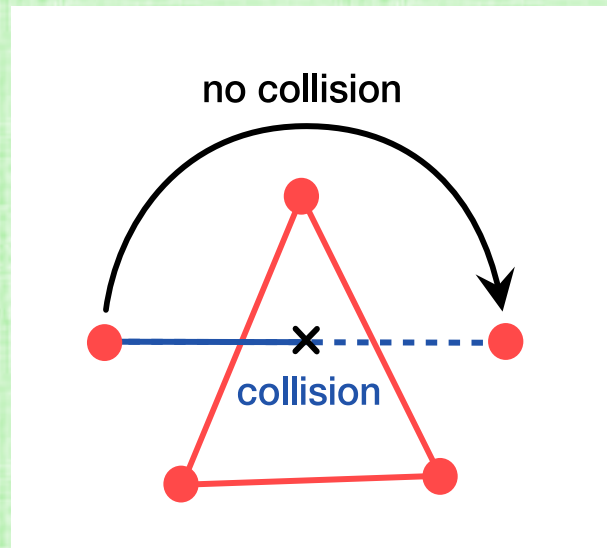
- In both cases, the 4 relevant points become coplanar when they collide
- Once coplanar, a second check determines whether: the point is inside the triangle (for Point-Face) or the two edges intersect (for Edge-Edge)





# Continuous Collision Detection (CCD)

- Consider time  $t_o$  to time  $t_f$ , and assume that the points have constant velocities during the time interval:  $V_i(t_o)$  for  $i = 1, 2, 3, 4$
- The time evolving positions are:  $X_i(t) = X_i(t_o) + V_i(t_o)(t - t_o)$  for  $t \in [t_o, t_f]$
- Although their paths are (generally) curved, considering piecewise linear increments is sufficient for preventing self-intersection



# Continuous Collision Detection (CCD)

- Coplanarity occurs when  $X_4(t) - X_1(t)$ ,  $X_3(t) - X_1(t)$ , and  $X_2(t) - X_1(t)$  are not a basis for  $R^3$ , which can be checked by making them the columns of a 3x3 matrix and setting the determinant to zero (leading to a cubic equation in  $t$ )
- Find the first root of this cubic equation in the interval  $[t_o, t_f]$
- Cubic equation solvers are so error prone that collisions are (very) often missed, and the cloth/shell ends up in a spurious self-intersecting state
- A very carefully devised/implemented iterative solver for cubic equations was able to detect all collisions:
  - It requires double precision (and fails often in single precision)
  - See Bridson et al. “Robust Treatment of Collisions, Contact, and Friction for Cloth Animation” (2002)

# Residual and Solution Error

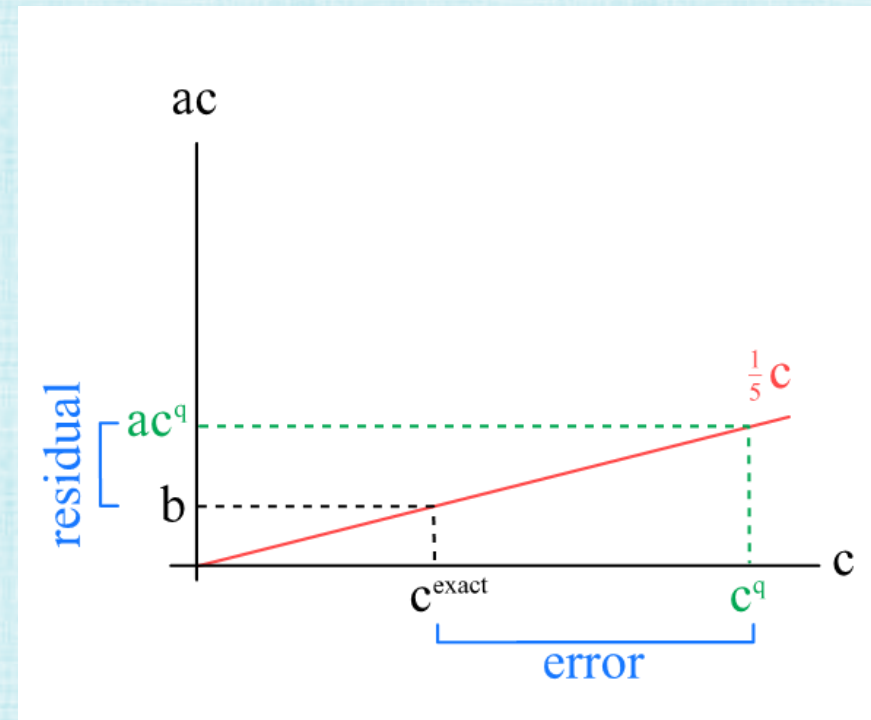
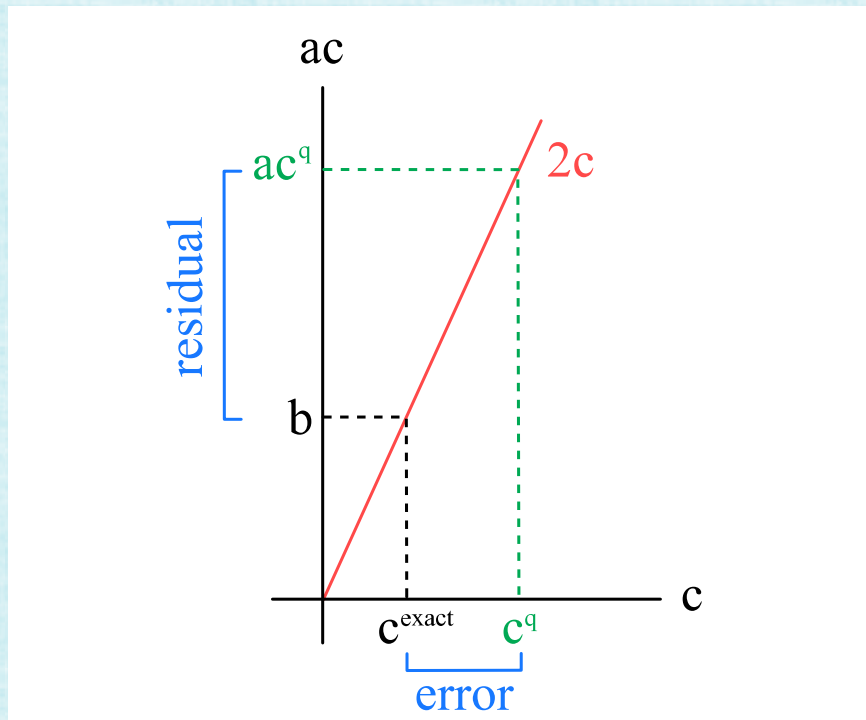
- When solving  $Ac = b$ , a current guess  $c^q$  has residual  $r^q = b - Ac^q$
- The residual measures the errors in the equations, not the error in the solution
- The error in the solution  $e^q = c^q - c^{exact}$  relates to the residual via:

$$r^q = b - Ac^q = Ac^{exact} - Ac^q = A(c^{exact} - c^q) = -Ae^q$$

- That is, the residual is the solution error transformed into the space that  $b$  lives in (the range of  $A$ )

# 1D example

- Consider a simple size  $1 \times 1$  matrix, i.e.  $[a]c = b$  with exact solution  $c = \frac{b}{a}$
- Since  $r^q = -ae^q$ , smaller  $a$  values lead to deceptively small residuals even when the error is large





# Diagonalizing the Residual/Error Equation

- "All matrices are diagonal matrices"
- And, diagonal matrices represent decoupled 1D scalar problems
- Using the SVD,  $r^q = -Ae^q$  becomes  $(U^T r^q) = -\Sigma(V^T e^q)$  which is a decoupled set of diagonal equations
- Each decoupled equation has the form  $\hat{r}_k^q = -\sigma_k \hat{e}_k^q$  (seen on the previous slide)
- Small  $\sigma_k$  lead to deceptively small residuals even when the error is large
- A small residual indicates a small error for larger singular values, but not for smaller singular values

# Line Search

- Choose a search direction  $s^q$  and move some distance  $\alpha^q$  in that direction to update the current guess to the next guess:  $c^{q+1} = c^q + \alpha^q s^q$ 
  - There are various strategies for choosing  $\alpha^q$ , including the notion of safe sets that clamp its maximum magnitude
  - Subtract  $c^{exact}$  from both sides of this recursion to get  $e^{q+1} = e^q + \alpha^q s^q$
  - Multiply through by  $-A$  to get  $r^{q+1} = r^q - \alpha^q A s^q$
- Optimally, one would follow  $s^q$  until all the error in that direction was eliminated
  - That is, until the remaining error is orthogonal to  $s^q$ , i.e.  $e^{q+1} \cdot s^q = 0$
  - However, the error is unknown (otherwise, the solution would be known)
- Instead, follow  $s^q$  until the residual is orthogonal to  $s^q$ , i.e.  $r^{q+1} \cdot s^q = 0$ 
  - Plugging in the recursion for  $r^{q+1}$  gives  $\alpha^q = \frac{s^q \cdot r^q}{s^q \cdot A s^q}$

# Steepest Descent

- Steepest Descent chooses the steepest downhill direction as the search direction
  - That turns out to be the residual, i.e. choose  $s^q = r^q$
- Iterate:  $r^q = b - Ac^q$ ,  $\alpha^q = \frac{r^q \cdot r^q}{r^q \cdot Ar^q}$ ,  $c^{q+1} = c^q + \alpha^q r^q$ , until  $r^q$  is considered small enough
- Can replace  $r^q = b - Ac^q$  with  $r^q = r^{q-1} - \alpha^{q-1} Ar^{q-1}$ 
  - Since  $Ar^{q-1}$  had already been computed to find  $\alpha^{q-1}$ , this eliminates one of the two (possibly expensive) multiplications by  $A$
- Drawback: Steepest Descent repeatedly searches in overlapping (non-orthogonal) directions, especially for higher condition number matrices (more on this later)



# Conjugate Gradients (CG)

- A very efficient and robust method for SPD systems
- Converges (theoretically) in at most  $n$ -steps for an  $n \times n$  matrix
  - Theoretically, only need one step for each distinct eigenvalue
  - Almost converged when taking one step for each eigenvalue cluster
  - Thus, preconditioning makes a big difference (assuming it clusters eigenvalues)
- Motivation: choosing **orthogonal** search directions would preclude repeatedly searching in overlapping directions (in contrast to Steepest Descent)
  - But, can't properly eliminate the error via  $e^{q+1} \cdot s^q = 0$  in each search direction
- Trick: use **A-orthogonal** search directions
  - Instead of  $\langle s^q, s^{\hat{q}} \rangle = 0$ , use  $\langle s^q, s^{\hat{q}} \rangle_A = 0$  for  $q \neq \hat{q}$



# Error Analysis for CG

- In the A-orthogonal basis of search directions, the initial error is  $e^1 = \sum_{\hat{q}=1}^n \beta^{\hat{q}} s^{\hat{q}}$
- Error recursion (i.e.,  $e^{q+1} = e^q + \alpha^q s^q$ ) gives  $e^q = e^1 + \sum_{\hat{q}=1}^{q-1} \alpha^{\hat{q}} s^{\hat{q}}$
- Thus,  $e^q = \sum_{\hat{q}=q}^n \beta^{\hat{q}} s^{\hat{q}} + \sum_{\hat{q}=1}^{q-1} (\beta^{\hat{q}} + \alpha^{\hat{q}}) s^{\hat{q}}$
- Progressing until  $r^{q+1} \cdot s^q = 0$  gives  $\alpha^q = \frac{s^q \cdot r^q}{s^q \cdot A s^q} = \frac{-\langle s^q, e^q \rangle_A}{\langle s^q, s^q \rangle_A} = \frac{-\beta^q \langle s^q, s^q \rangle_A}{\langle s^q, s^q \rangle_A} = -\beta^q$
- Thus,  $e^q = \sum_{\hat{q}=q}^n \beta^{\hat{q}} s^{\hat{q}}$
- This proves that the error is indeed cancelled out in  $n$  steps, i.e.  $e^{n+1} = 0$
- Aside: If  $q > \tilde{q}$ , then  $r^q \cdot s^{\tilde{q}} = -\langle s^{\tilde{q}}, e^q \rangle_A = 0$ ; so, **each residual is orthogonal to all previous search directions** (not just the previous one)

# Gram-Schmidt

- Orthogonalizes a set of vectors
- For each new vector, subtract its (weighted) dot product overlap with all prior vectors, making it orthogonal to them
- A-orthogonal Gram-Schmidt uses an A-weighted dot/inner product
- Given candidate vector  $\bar{s}^q$ , subtract out the A-overlap with  $s^1$  to  $s^{q-1}$  so that the resulting vector  $s^q$  has  $\langle s^q, s^{\hat{q}} \rangle_A = 0$  for  $\hat{q} \in \{1, 2, \dots, q-1\}$
- That is,  $s^q = \bar{s}^q - \sum_{\hat{q}=1}^{q-1} \frac{\langle \bar{s}^q, s^{\hat{q}} \rangle_A}{\langle s^{\hat{q}}, s^{\hat{q}} \rangle_A} s^{\hat{q}}$  where the two non-normalized  $s^{\hat{q}}$  both require division by their norm
  - Note:  $\langle s^{\hat{q}}, s^{\hat{q}} \rangle_A = \|s^{\hat{q}}\|_A^2$
  - Proof:  $\langle s^q, s^{\tilde{q}} \rangle_A = \langle \bar{s}^q, s^{\tilde{q}} \rangle_A - \frac{\langle \bar{s}^q, s^{\tilde{q}} \rangle_A}{\langle s^{\tilde{q}}, s^{\tilde{q}} \rangle_A} \langle s^{\tilde{q}}, s^{\tilde{q}} \rangle_A = 0$  for  $\tilde{q} \in \{1, 2, \dots, q-1\}$

# Gram-Schmidt for CG

- Choose candidate search directions  $r^q$  to make A-orthogonal via Gram-Schmidt:

$$s^q = r^q - \sum_{\hat{q}=1}^{q-1} \frac{\langle r^q, s^{\hat{q}} \rangle_A}{\langle s^{\hat{q}}, s^{\hat{q}} \rangle_A} s^{\hat{q}}$$

- Dot product with  $r^{\tilde{q}}$  to get:  $s^q \cdot r^{\tilde{q}} = r^q \cdot r^{\tilde{q}} - \sum_{\hat{q}=1}^{q-1} \frac{\langle r^q, s^{\hat{q}} \rangle_A}{\langle s^{\hat{q}}, s^{\hat{q}} \rangle_A} s^{\hat{q}} \cdot r^{\tilde{q}}$ 
  - If  $\tilde{q} > q$ , then  $0 = r^q \cdot r^{\tilde{q}} + 0$  implies that all the residuals are orthogonal to each other

- If  $\tilde{q} = q$ , then  $s^q \cdot r^q = r^q \cdot r^q + 0$  implies  $\alpha^q = \frac{r^q \cdot r^q}{\langle s^q, s^q \rangle_A}$

- Dot product  $r^q = r^{q-1} - \alpha^{q-1} A s^{q-1}$  with  $r^{\tilde{q}}$  to get:

$$r^{\tilde{q}} \cdot r^q = r^{\tilde{q}} \cdot r^{q-1} - \alpha^{q-1} \langle r^{\tilde{q}}, s^{q-1} \rangle_A$$

- If  $\tilde{q} = q$ , then  $r^q \cdot r^q = 0 - \alpha^{q-1} \langle r^q, s^{q-1} \rangle_A$  for the last term in the Gram-Schmidt sum
- If  $\tilde{q} > q$ , then  $0 = 0 - \alpha^{q-1} \langle r^{\tilde{q}}, s^{q-1} \rangle_A$  implies that all the other terms are zero

- Thus,  $s^q = r^q + \frac{r^q \cdot r^q}{\alpha^{q-1} \langle s^{q-1}, s^{q-1} \rangle_A} s^{q-1} = r^q + \frac{r^q \cdot r^q}{r^{q-1} \cdot r^{q-1}} s^{q-1}$

# Conjugate Gradients Method

- Start with:  $s^1 = r^1 = b - Ac^1$
- Iterate:
  - $\alpha^q = \frac{r^q \cdot r^q}{\langle s^q, s^q \rangle_A}$
  - $c^{q+1} = c^q + \alpha^q s^q$  and  $r^{q+1} = r^q - \alpha^q A s^q$  (both as usual for line search)
  - $s^{q+1} = r^{q+1} + \frac{r^{q+1} \cdot r^{q+1}}{r^q \cdot r^q} s^q$
- Note: Gram-Schmidt drifts, making search directions less A-orthogonal over time; thus, for computational robustness, occasionally throw out all search directions and start over with  $s^1 = r^1 = b - Ac^1$



# Non-Symmetric and/or Indefinite

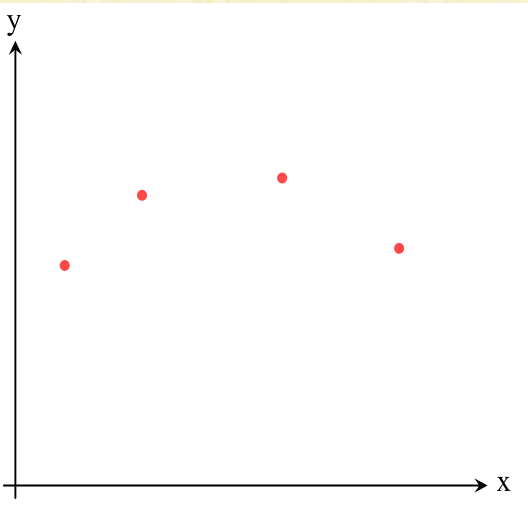
- GMRES, MINRES, BiCGSTAB, etc...
- Generally speaking, iterative methods for non-symmetric and/or indefinite matrices are less stable, more error prone, and slower than CG on an SPD matrix

# Unit 6

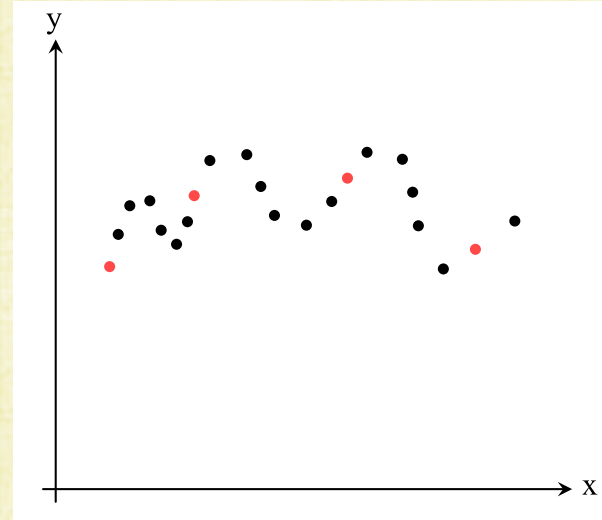
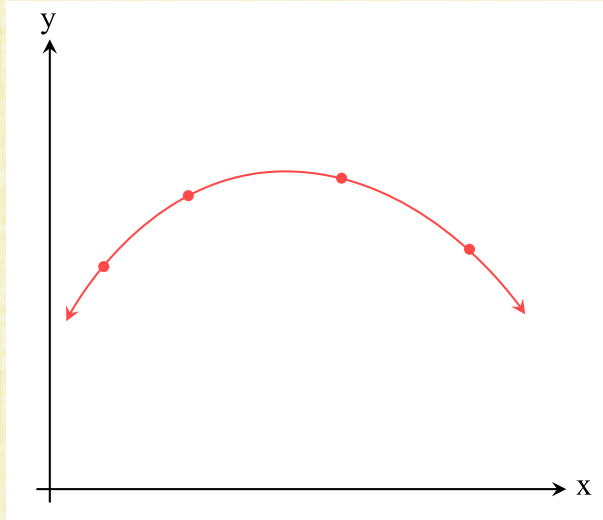
# Local Approximations

# Sampling

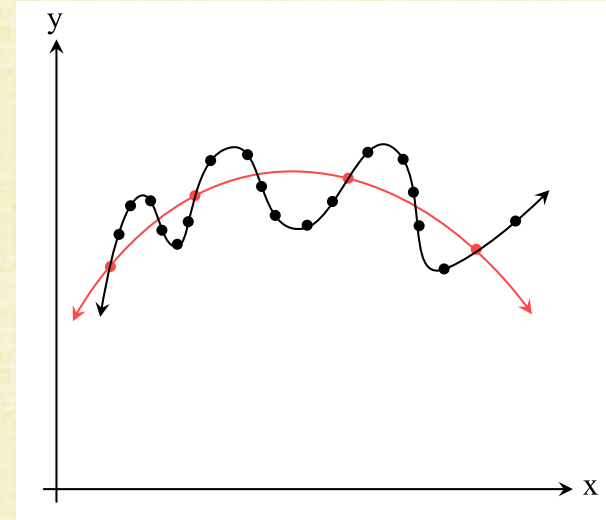
- Accurate approximation of a function is often limited by the amount of available data
- Given too few samples (left), one may “hallucinate” an incorrect function
- Adding more data allows for more accurate feature resolution (right)
- Given “enough” sample points, a function will eventually be well-represented by them



under-resolved



resolved better with more data





# Taylor Expansion

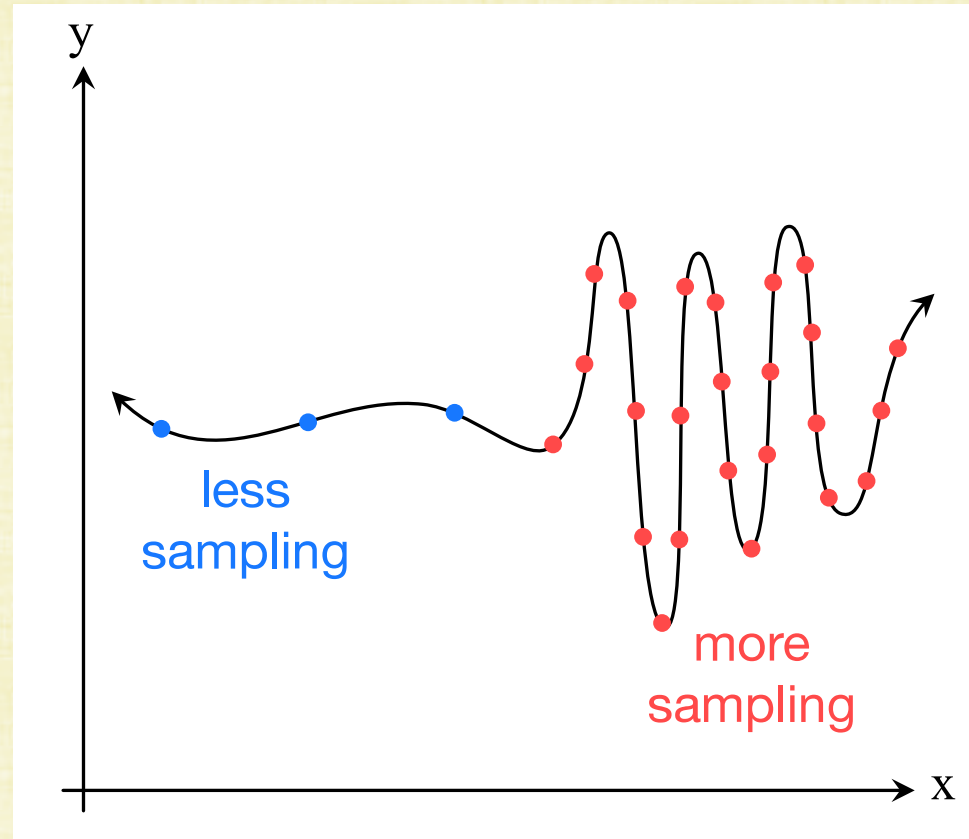
- $f(x + h) = \sum_{p=0}^{\infty} \frac{h^p}{p!} f^{(p)}(x) = \sum_{p=0}^{\hat{p}} \frac{h^p}{p!} f^{(p)}(x) + O(h^{\hat{p}+1})$
- Bounded derivatives would indicate that  $O(h^{\hat{p}+1}) \rightarrow 0$  as  $h \rightarrow 0$
- Examples:
  - $f(x + h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x) + O(h^4)$  looking forward
  - $f(x - h) = f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(x) + O(h^4)$  looking backward
- Truncated Taylor expansions become more valid approximations as  $h \rightarrow 0$ 
  - $f(x + h) \approx f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x)$  looking forward
  - $f(x - h) \approx f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(x)$  looking backward

# Well-Resolved Functions

- The Taylor expansion approximates a function  $f$  at a new location  $x + h$  based on known information at a nearby point  $x$
- When the sample points are “closely” spaced, new locations are “close” to known sample points making  $h$  “small” enough
- However, large derivatives can overwhelm even a small  $h$
- Thus, functions with more variation need higher sampling rates
  - Similarly, smoother functions can utilize lower sampling rates
- Well-resolved functions have vanishing high order terms in their Taylor expansion making truncated Taylor expansions more valid

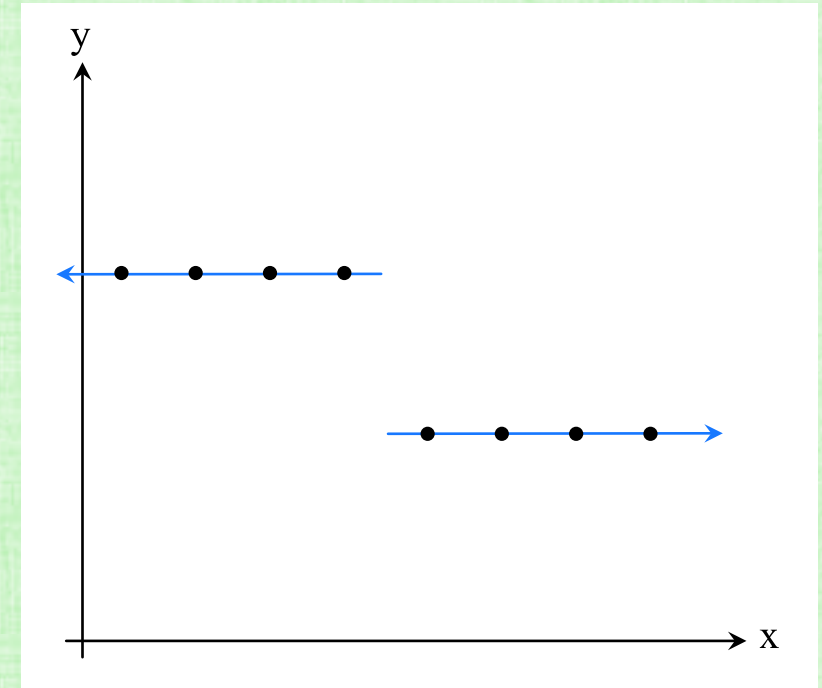
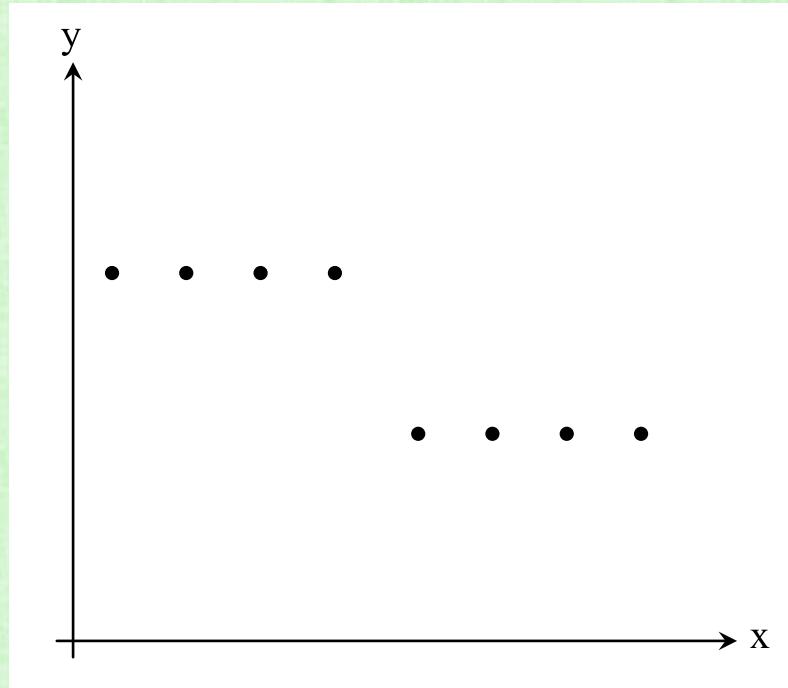
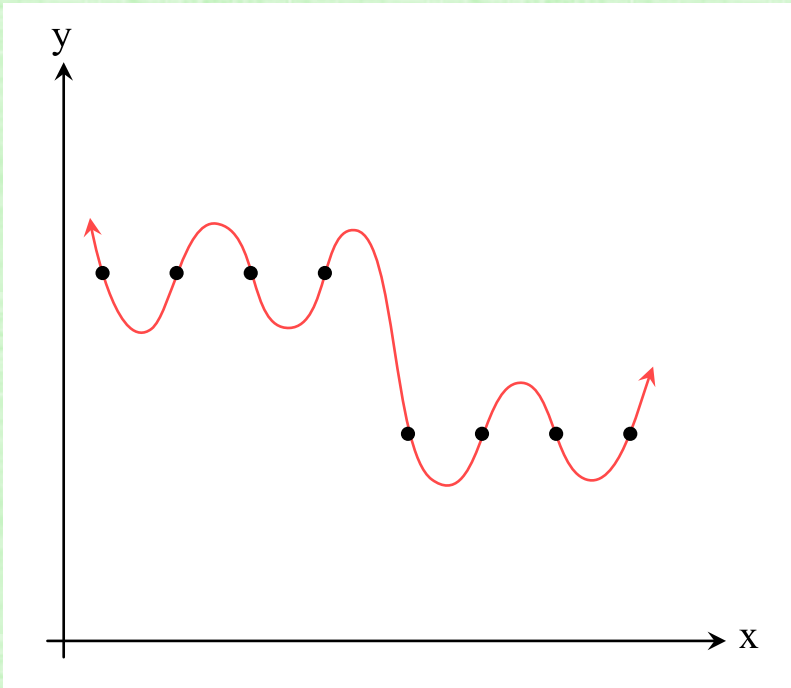
# Well-Resolved Functions

- Regions of a function with less/more variation require lower/higher sampling rates



# Piecewise Approximation

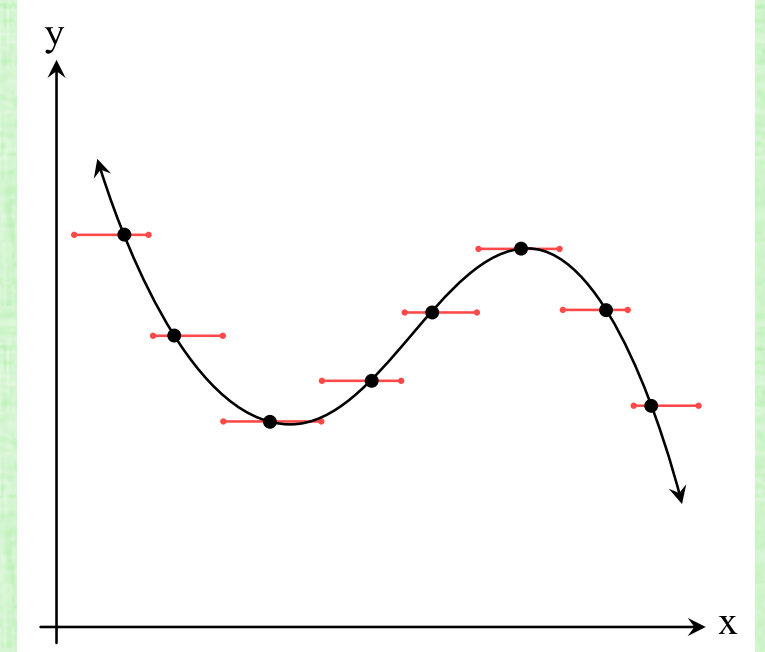
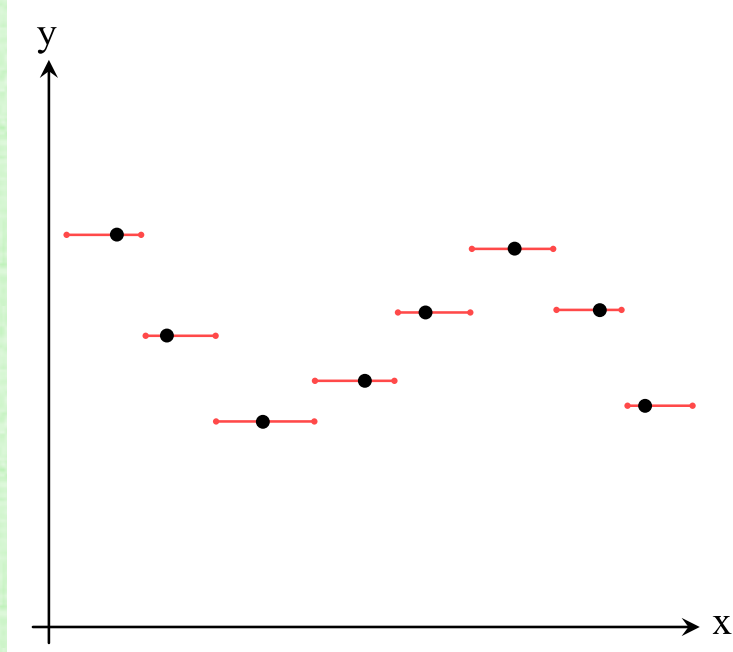
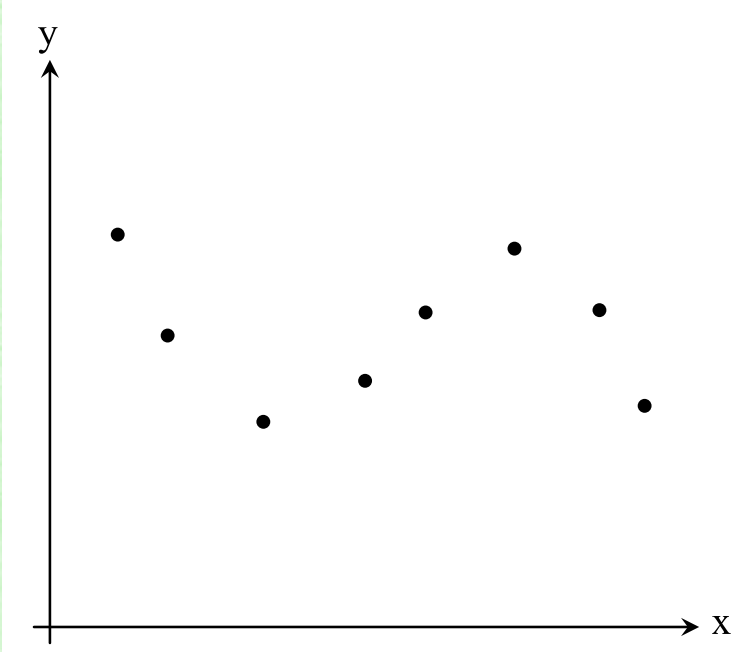
- Piecewise approximation enables the use of simpler models to approximate (potentially disjoint) subsets of data
  - In ML/DL, “sub-manifold” often refers to a coherent subset of data





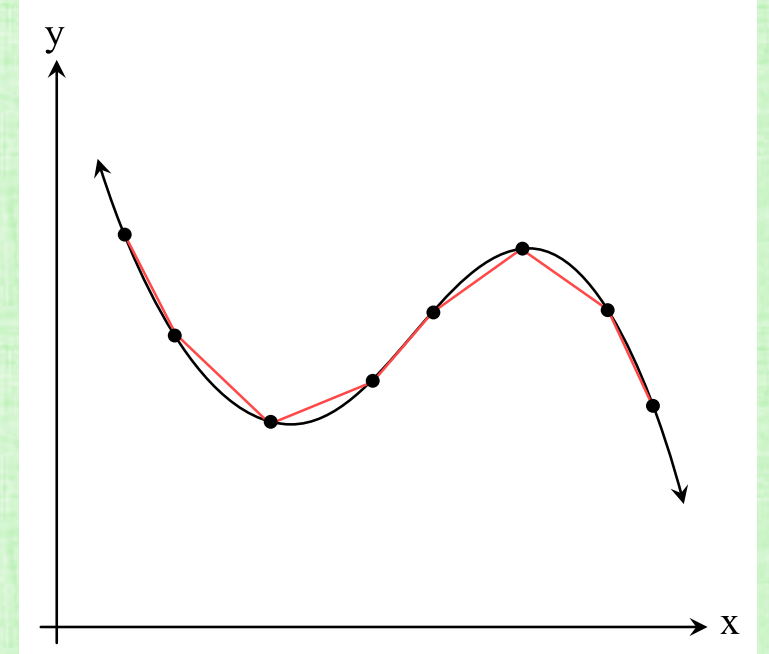
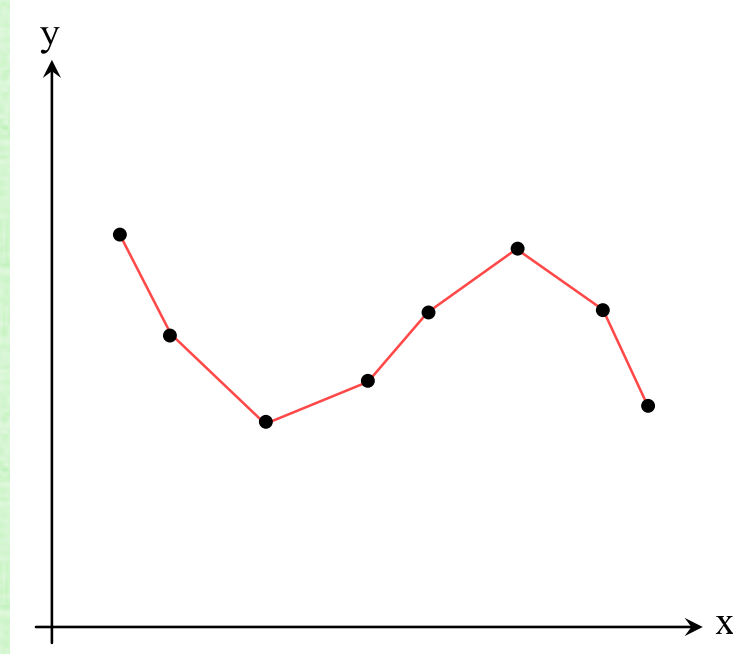
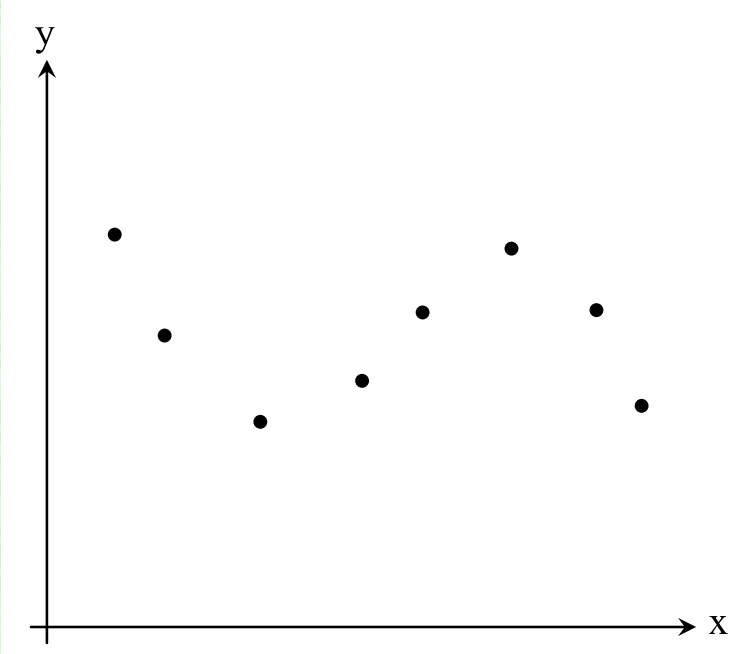
# Piecewise Constant Interpolation

- Use the first term in the Taylor expansion (only):  $f(x + h) \approx f(x)$
- Errors are  $O(h)$ , since  $f(x + h) = f(x) + O(h)$
- Recall: nearest neighbor is piecewise constant



# Piecewise Linear Interpolation

- Use the first two terms in the Taylor expansion:  $f(x + h) \approx f(x) + hf'(x)$
- Errors are  $O(h^2)$ , since  $f(x + h) = f(x) + hf'(x) + O(h^2)$

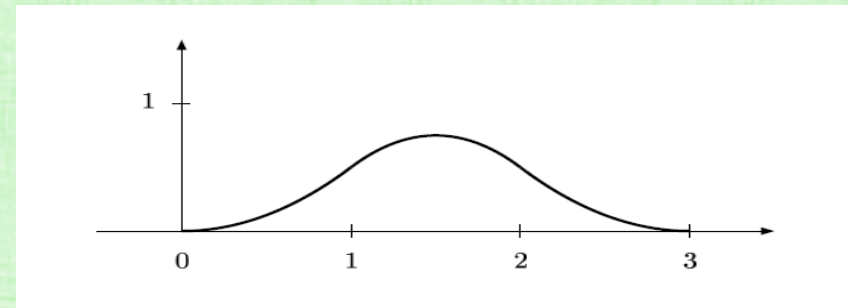
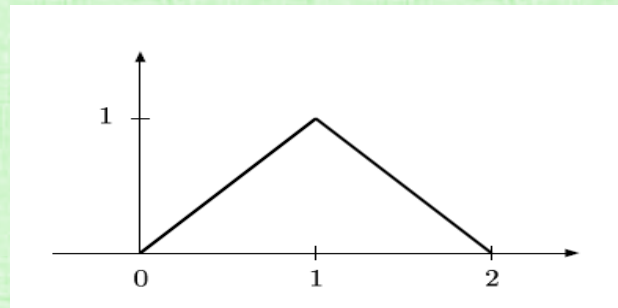
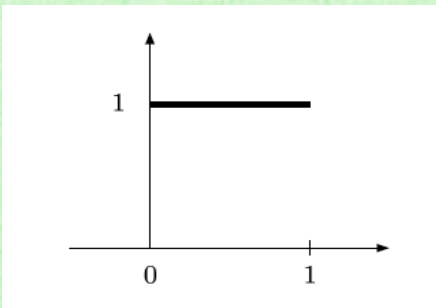


# Higher Order Piecewise Interpolation

- Piecewise quadratic interpolation uses the first three terms in the Taylor expansion and has  $O(h^3)$  errors
- Piecewise cubic interpolation uses the first four terms in the Taylor expansion and has  $O(h^4)$  errors
- Recall: higher order interpolation becomes more oscillatory (i.e. overfitting)
  - These oscillations are sometimes referred to as Gibbs phenomena

# Piecewise Cubic Interpolation (B-Splines)

- Piecewise cubic splines are quite popular because of their ability to match derivatives across approximation boundaries
- B-splines – hierarchical family:  $\phi_i^p$  is a piecewise polynomial of degree  $p$ 
  - Piecewise constant:  $\phi_i^0(x) = 1$  for  $x \in [x_i, x_{i+1}]$  and 0 otherwise
  - Recursively:  $\phi_i^{p+1}(x) = w_i^p(x)\phi_i^p(x) + (1 - w_{i+1}^p(x))\phi_{i+1}^p(x)$ 
    - A linear  $w_i^p(x) = \frac{x-x_i}{x_{i+p+1}-x_i}$  increases the polynomial degree of  $\phi_i^p$  by one
  - Piecewise linear  $\phi_i^1$ , piecewise quadratic  $\phi_i^2$ , piecewise cubic  $\phi_i^3$ , etc.





# Image Segmentation

- The goal is to divide image pixels into separate regions, each representing separate objects or groups of objects
- Before neural networks: Various methods relied on clustering in color and/or space, graph-cuts, edge detection, etc.
- Since humans do well on this problem, can use neural networks to aim to mimic human perception/semantics
- Training examples:
  - Input: an image (all the pixel RGB values)
  - Output: labels on all the pixels, indicating what group each pixel is in

# Bool Output Labels

- Binary segmentation of an image
- E.g. true = dog, false = not dog



Input



Output

# Integer Output Labels

- Multi-object segmentation with an integer for each object
- E.g. 1=cat, 2=dog, 3=human, 4=mug, 5=couch, 6=everything else



Input



Output

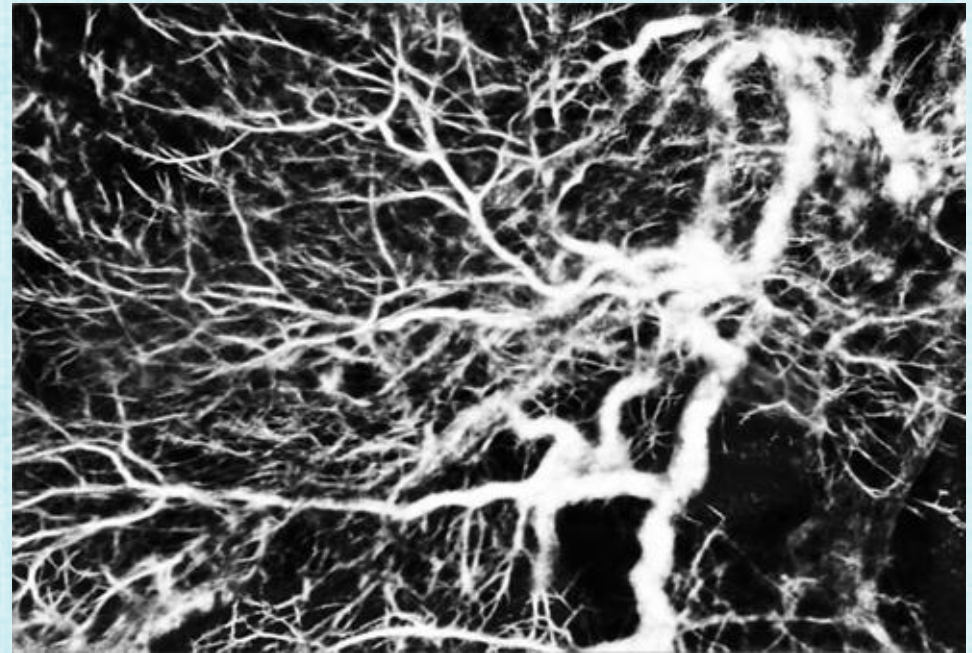


# Real Number Output Labels

- Probabilistic segmentation with real number values in  $[0,1]$
- E.g. 1=tree branch, .8=probably a branch, .2=probably not a branch, etc.



Input



Output



# Segmenting Botanical Trees

## Difficult Problem:

- Trees are large-scale and geometrically-complex structures
  - Branches severely occlude each other
  - The images have limited pixel resolution of individual branches
- 
- Even humans have a hard time ascertaining the correct topological structure from a single image/view
  - Can we train a neural network to help?

# Constructing Training Data

- Begin with a dataset of labels (tediously) created by hand
- Draw lines and thicknesses on top of branches; then, use this information to create a binary mask for the image



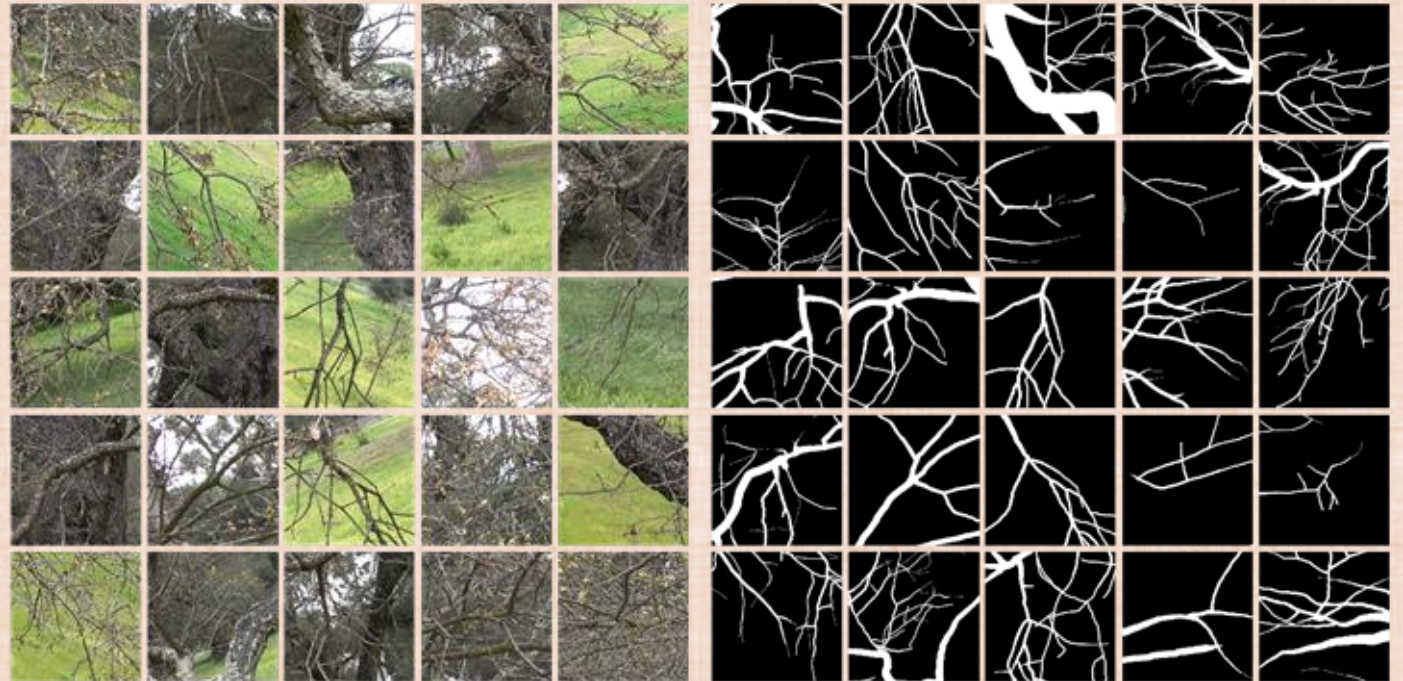
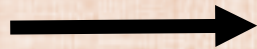


# Artificially Augmenting Training Data

- Artificially increase the amount of training data by taking various image subsets
  - This also helps to avoid down-sampling (networks use low-resolution images)



3840 pixels wide, 2160 pixels tall

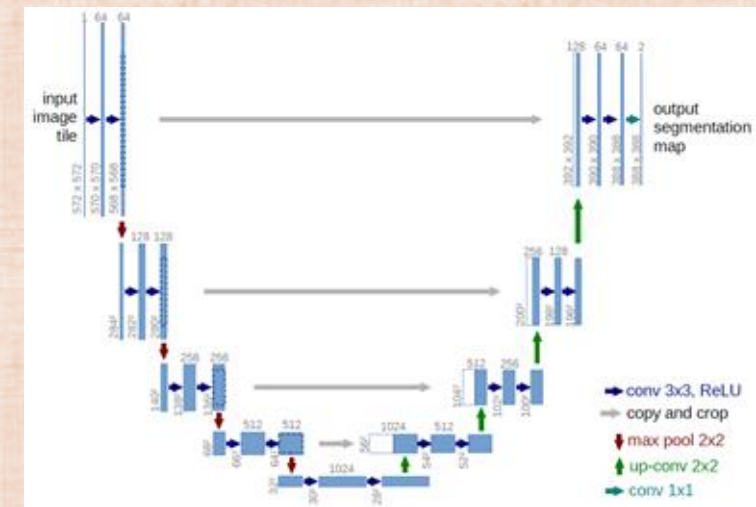


each image: 512 pixels wide, 512 pixels tall



# Training the Neural Network

- Find function parameters  $c$  such that the network function  $f_c(x)$  gives minimal error on the training data (i.e. minimize a “loss” function)
- The network should predict the known target labels (or close to it) from the input images

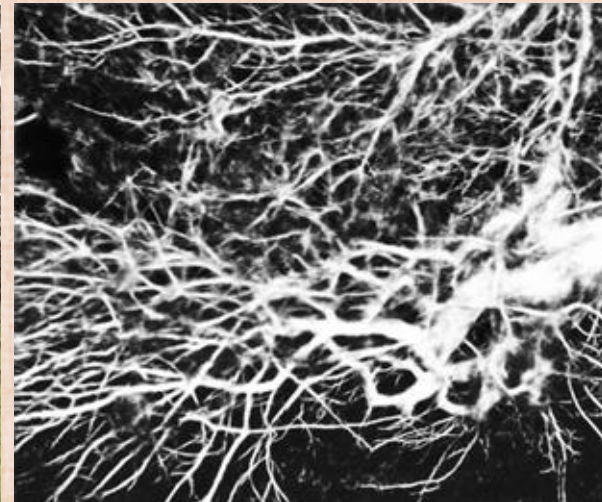
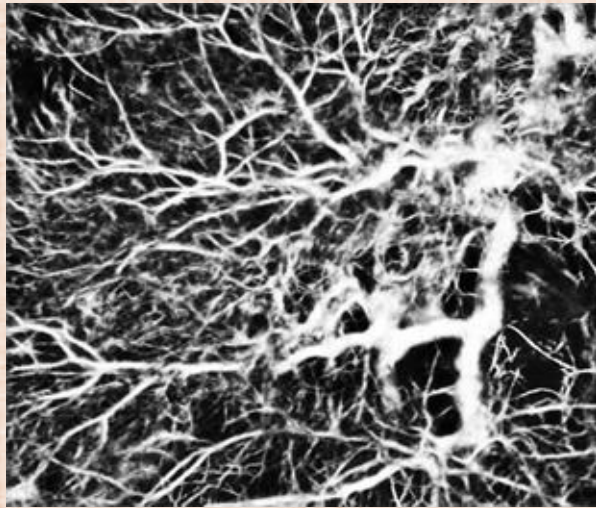


network architecture



# Network Inference/Prediction

- After training, use the resulting network function  $f_{c_{trained}}(x)$  to infer/predict labels for new images (not previously hand-labeled)





# Local Approximations

- Roughly speaking, input images mostly seem to be of two different types: either (1) branches over green grass or (2) clusters of brown branches



# Train 2 Neural Networks

- Divide the training data into these two disparate sets:
  - k-means clustering on hue/saturation can be used to divide the training images into two separate clusters
- Train a separate network on each of the two sets of training data:
  - separate architecture, separate trainable parameters, etc.
- Note: this approach works for more than 2 clusters/networks too!



# Combining the Network Inferred Outputs

- Given an input image, inference it (separately) on each of the two networks
- Then, combine the two predictions, using the network that makes the most sense locally in each part of the image (blend predictions when appropriate)

To inference each pixel:

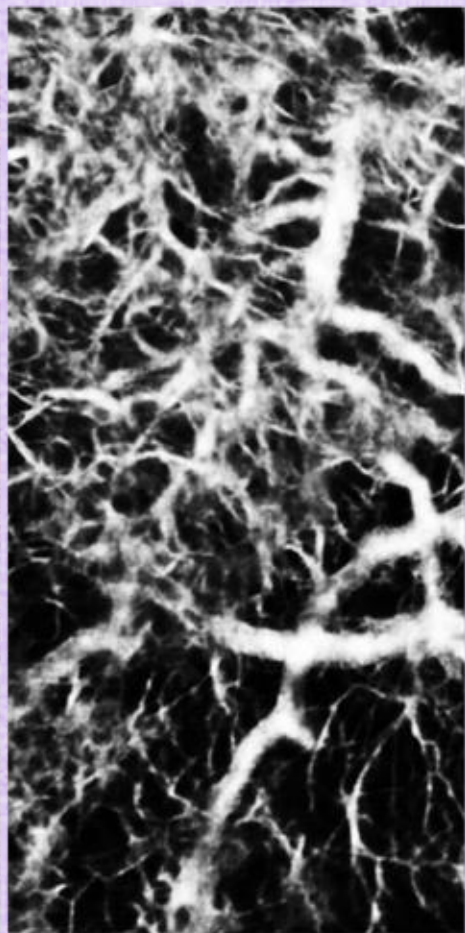
- Compute hue/saturation values on a small patch around the pixel
- Find the distances from the patch hue/saturation values to the two cluster centers
- Interpolate the outputs from the two networks using those distances
  - The closer a pixel is to a k-means cluster, the more weight is given to that cluster's network inference/prediction
- Note: this approach works for more than 2 clusters/networks too!



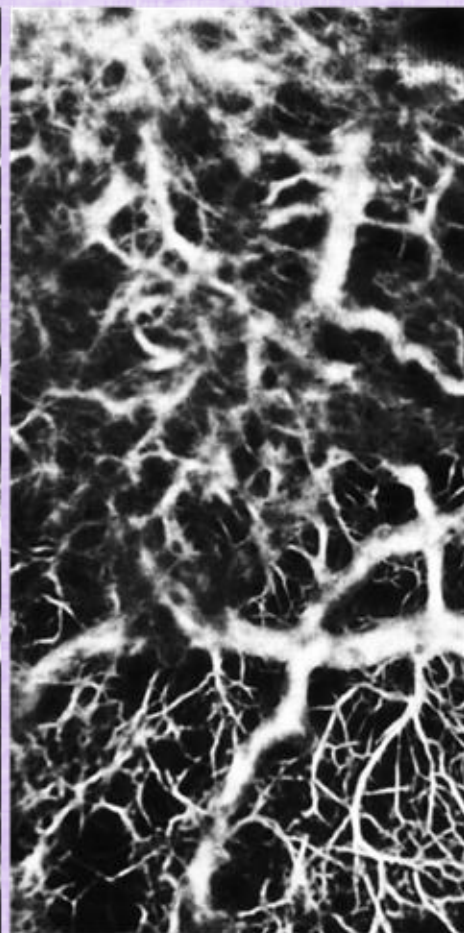
# Example



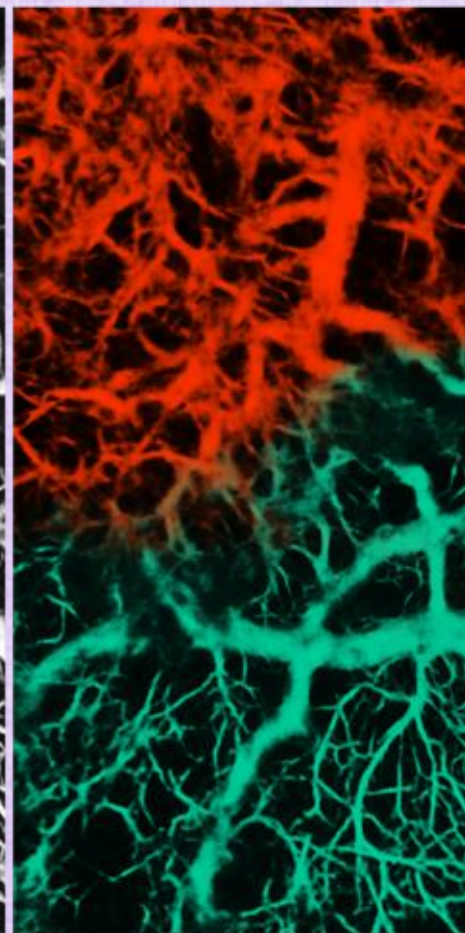
Input



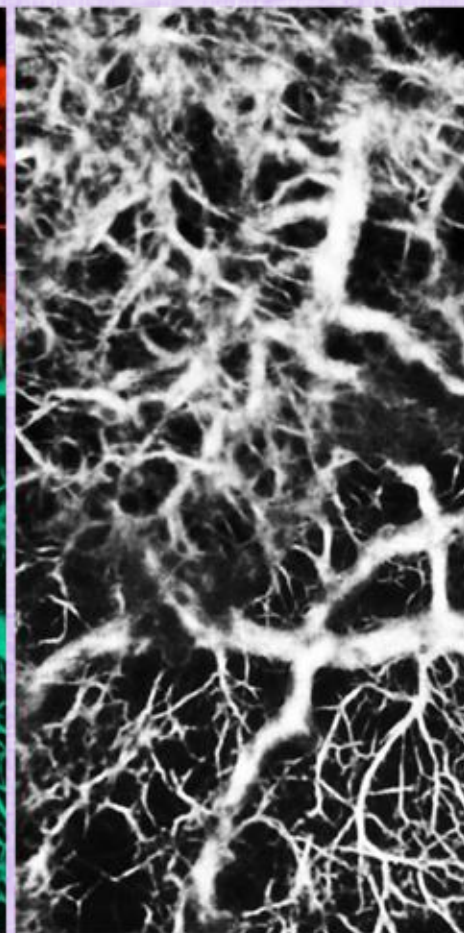
Network 1



Network 2



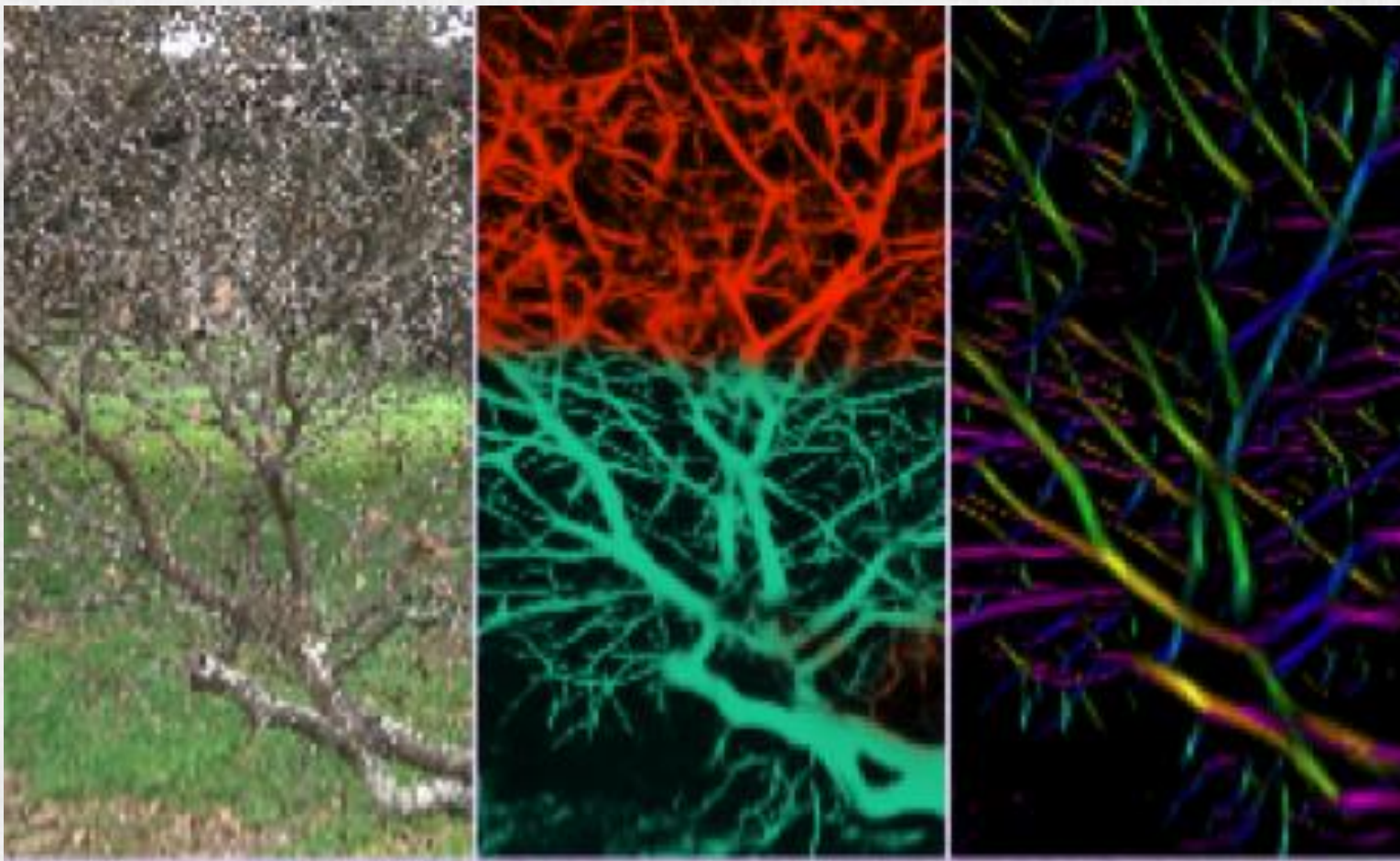
Combine



Final Result



## Aside: Branch Estimation







# Unit 7



# Curse of Dimensionality

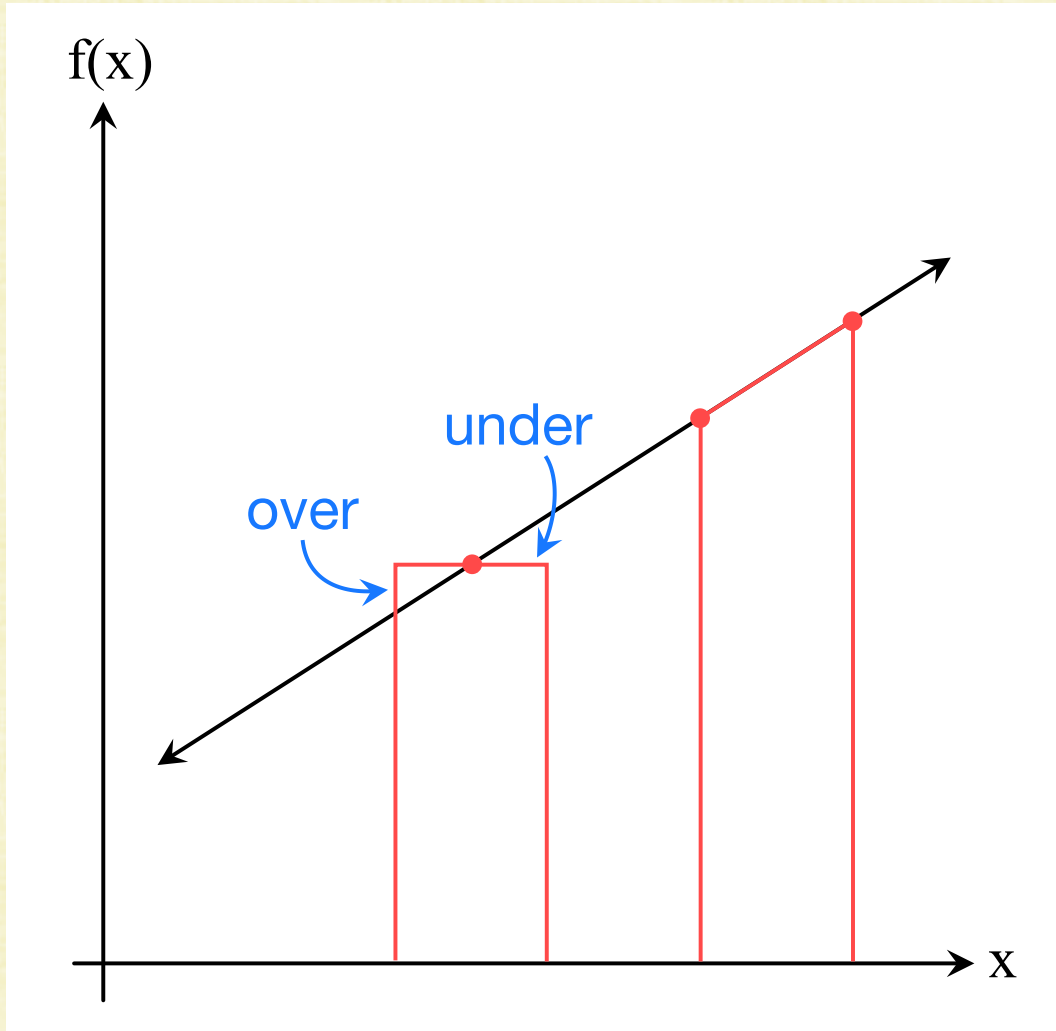
# Numerical Integration (i.e., Quadrature)

- Approximate  $\int_{x_L}^{x_R} f(x)dx$  numerically
- Break up  $[x_L, x_R]$  into subintervals, and consider each subinterval separately
- On each subinterval:
  - Reconstruct the function
  - Analytically find the area under the reconstructed curve
  - (These two steps can be combined in various ways, for efficiency)
- $f$  is often not explicitly known
- I.e., often only have access to output values  $f(x_i)$  given input values  $x_i$
- In addition, it could be expensive to “evaluate”  $f(x_i)$ , especially when it requires performing experiments, gathering data, hand labeling, running code, etc.

# Newton-Cotes Quadrature

- Uses a local approximation on each subinterval
- On each subinterval, choose  $p$  equally spaced points and use  $p - 1$  degree polynomial interpolation to reconstruct the function and approximate the area under the curve
- This gives the exact solution when  $f$  is a degree  $p - 1$  polynomial (as expected)
- When the number of points  $p$  is odd, symmetric cancellation gives the exact solution on a degree  $p$  polynomial (one degree higher than expected)

# Symmetric Cancellation



- When  $p = 2$  points, the 1<sup>st</sup> degree piecewise linear approximation integrates piecewise linear functions exactly (right interval in the figure)
- When  $p = 1$  point, the 0<sup>th</sup> degree piecewise constant approximation (also) integrates piecewise linear functions exactly (left interval in the figure)
  - Note the cancellation of under/over approximations (in the left interval)



# Newton-Cotes Quadrature

- Consider a total of  $m$  subintervals
- Piecewise constant approximation ( $p = 1$  point) uses  $m$  points to integrate piecewise linear functions exactly
- Piecewise linear approximation ( $p = 2$  points) uses  $m + 1$  points to integrate piecewise linear functions exactly
  - points on the boundary between intervals are used for both intervals
- Piecewise quadratic approximation ( $p = 3$  points) uses  $2m + 1$  points to integrate piecewise cubic functions exactly
  - points on the boundary between intervals are used for both intervals
- Piecewise cubic approximation ( $p = 4$  points) uses  $3m + 1$  points to integrate piecewise cubic functions exactly
  - points on the boundary between intervals are used for both intervals

# Recall: Taylor Expansion

- $f(x + h) = \sum_{p=0}^{\infty} \frac{h^p}{p!} f^{(p)}(x) = \sum_{p=0}^{\hat{p}} \frac{h^p}{p!} f^{(p)}(x) + O(h^{\hat{p}+1})$
- Bounded derivatives would indicate that  $O(h^{\hat{p}+1}) \rightarrow 0$  as  $h \rightarrow 0$
- Examples:
  - $f(x + h) = f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x) + O(h^4)$  looking forward
  - $f(x - h) = f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(x) + O(h^4)$  looking backward
- Truncated Taylor expansions become more valid approximations as  $h \rightarrow 0$ 
  - $f(x + h) \approx f(x) + hf'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x)$  looking forward
  - $f(x - h) \approx f(x) - hf'(x) + \frac{h^2}{2} f''(x) - \frac{h^3}{6} f'''(x)$  looking backward

# Local and Global Error

- Degree  $p$  polynomial reconstruction captures the Taylor expansion terms up to and including  $\frac{h^p}{p!} f^{(p)}(x)$ , with  $O(h^{p+1})$  errors
- This  $O(h^{p+1})$  error in the height of the function multiplied times the  $O(h)$  width of the interval gives a per interval area error (local error) of  $O(h^{p+2})$
- The total number of intervals is  $\frac{x_R - x_L}{O(h)} = O\left(\frac{1}{h}\right)$ , so the total error (global error) is  $O\left(\frac{1}{h}\right) O(h^{p+2}) = O(h^{p+1})$
- Doubling the number of intervals halves their size leading to  $O\left(\left(\frac{h}{2}\right)^{p+1}\right) = \left(\frac{1}{2}\right)^{p+1} O(h^{p+1})$  total/global error, or  $\left(\frac{1}{2}\right)^{p+1}$  as much total/global error
- This is denoted by an order of accuracy of  $p + 1$



# Newton-Cotes Quadrature (Methods)

- Midpoint Rule:  $\sum_i h_i f(x_i^{mid})$ 
  - 1 point, piecewise constant approximation, exact for piecewise linear functions,  $O(h^2)$  error, 2<sup>nd</sup> order accurate
- Trapezoidal Rule:  $\sum_i h_i \frac{f(x_i^{left}) + f(x_i^{right})}{2}$ 
  - 2 points, piecewise linear approximation, exact for piecewise linear functions,  $O(h^2)$  error, 2<sup>nd</sup> order accurate
- Simpson's Rule:  $\sum_i h_i \frac{f(x_i^{left}) + 4f(x_i^{mid}) + f(x_i^{right})}{6}$ 
  - 3 points, piecewise quadratic approximation, exact for piecewise cubic functions,  $O(h^4)$  error, 4<sup>th</sup> order accurate



# Gaussian Quadrature

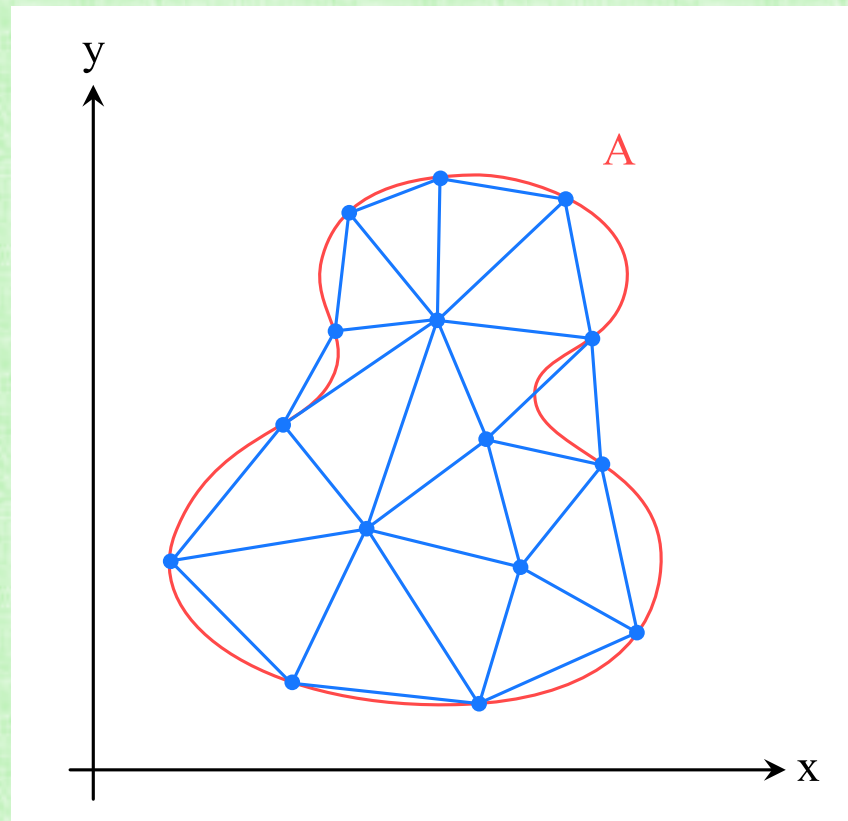
- Use  $p$  optimally chosen points to obtain a method that is exact on degree  $2p - 1$  polynomials, and thus has an order of accuracy of  $2p$
- For example: 
$$\sum_i h_i \frac{f\left(x_i^{mid} - \frac{h_i}{2\sqrt{3}}\right) + f\left(x_i^{mid} + \frac{h_i}{2\sqrt{3}}\right)}{2}$$
- 2 points, piecewise cubic approximation, exact for piecewise cubic functions,  $O(h^4)$  error, 4<sup>th</sup> order accurate
- Same accuracy as the 3 point Simpson's Rule
  - Simpson has 1 point on shared boundaries, so only  $2m + 1$  total points are required
  - That is, Gaussian quadrature only saves 1 point in total ( $2m$  total points)

# Two Dimensions

- $\iint_A f(x, y) dA$  where each sub-region  $dA$  of area  $A$  is approximated locally
- When  $A$  is rectangular, it can be broken into sub-rectangles and addressed dimension-by-dimension using 1D techniques
- When  $A$  is more interesting, triangle sub-regions can be used to approximate it
- The difference between  $A$  and its approximation leads to a new source of error not seen in 1D (where interval boundaries are just points instead of curves)

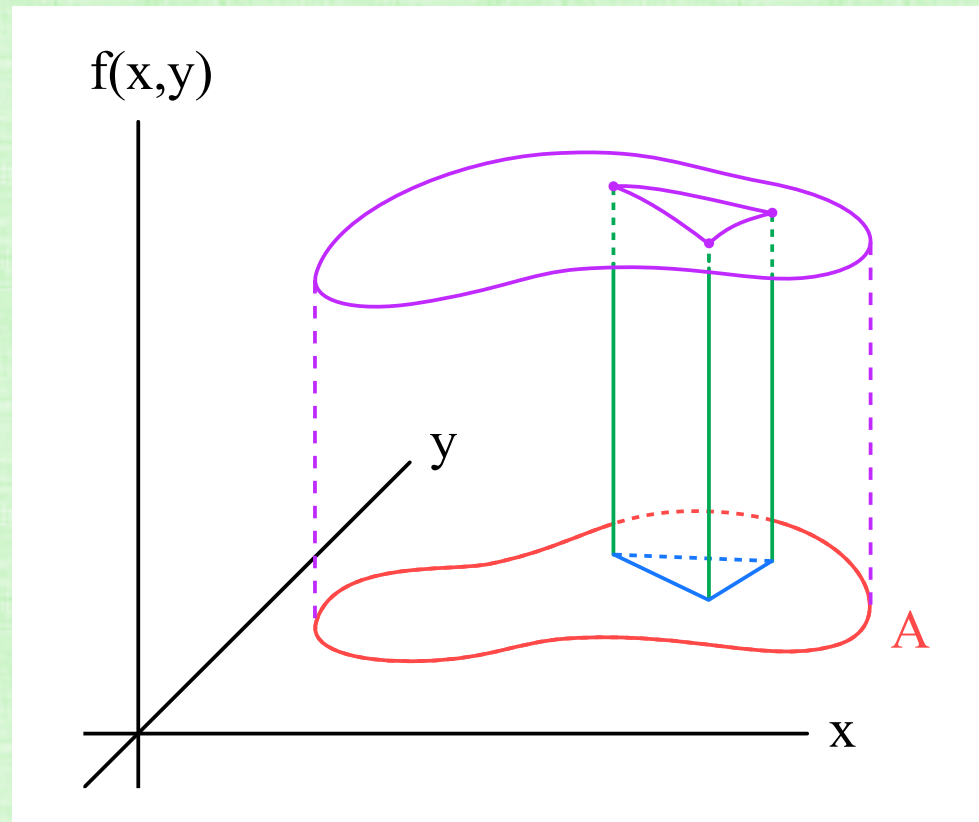
# Domain Approximation Errors

- The difference between  $A$  and its approximation (via triangles here) leads to a new source of error in the integral (both missing and extra areas)



# Integrating over Sub-regions

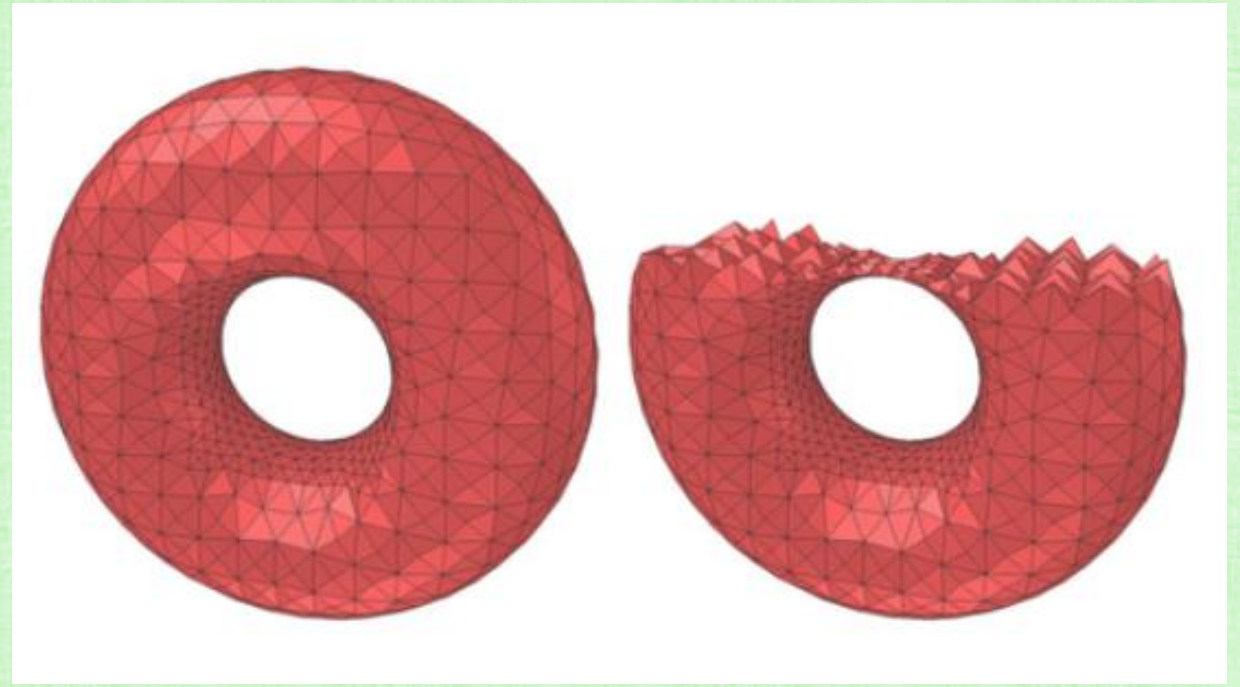
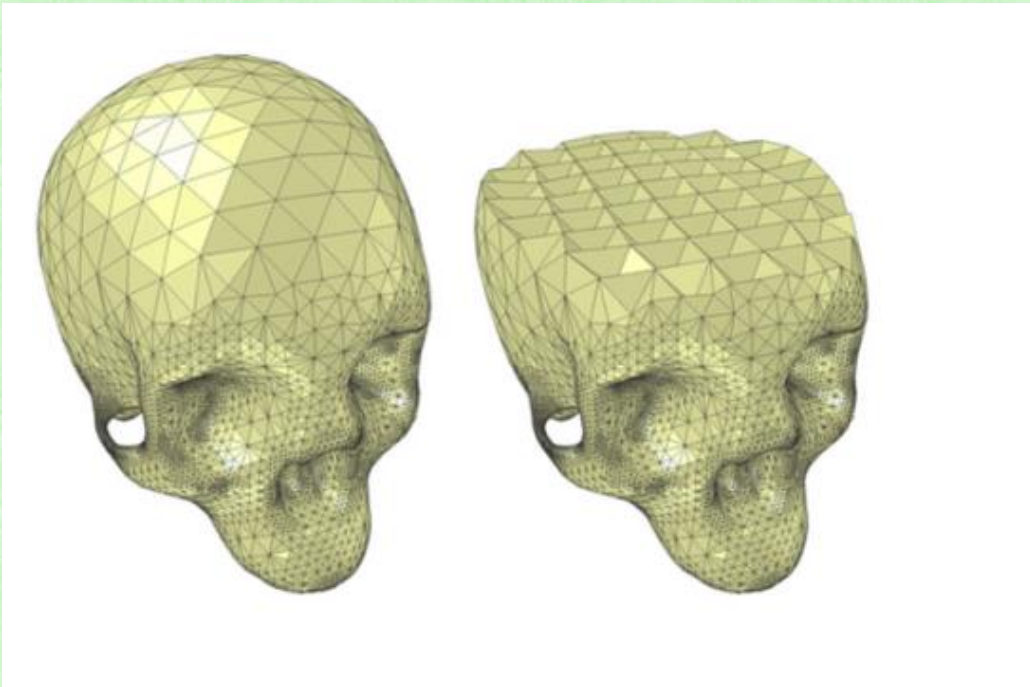
- Each triangle sub-region utilizes optimally chosen Gaussian quadrature points to compute sub-volumes





# Three Dimensions

- $\iiint_V f(x, y, z) dV$  where each tetrahedral sub-region  $dV$  of volume  $V$  is approximated locally (with Gaussian quadrature points)



# Curse of Dimensionality

- Consider a 1<sup>st</sup> order accurate method with  $O(h)$  error
- 1D: doubling the number of intervals cuts the error in half (2x work =  $\frac{1}{2}$  error)
- 2D: halving interval size requires 4 times the rectangles/triangles (4x work =  $\frac{1}{2}$  error)
- 3D: halving interval size requires 8 times the cubes/boxes/tets (8x work =  $\frac{1}{2}$  error)
- 4D: 16x work =  $\frac{1}{2}$  error, 5D: 32x work =  $\frac{1}{2}$  error, etc.
- Cutting error by a factor of 4 in 5D takes  $32^2=1024$ x work
- Cutting error by a factor of 8 in 5D takes  $32^3=32,768$ x work
- If the original code took 1 sec to run in 5D, cutting error by a factor of 8 takes 9 hours
- And cutting error by a factor of 16 takes 12 days
- And cutting the error by a factor of 32 takes over a year....

**Yep, you're cursed**

# Curse of Dimensionality

- Consider a 2<sup>nd</sup> order accurate method with  $O(h^2)$  error
- In 1D/2D/3D/4D/5D/etc. halving the interval size gives 4 times less error
- Cutting error by a factor of 4 in 5D takes 32x work
- If the original code took 1 sec to run in 5D, cutting error by a factor of 16 takes only 17 min (much faster than the 12 days for the 1<sup>st</sup> order accurate method)
- Cutting error by a factor of 1024 (3 decimal places more accuracy) takes over a year...
- In 10D, cutting error by a factor of 4 takes 1024x work
- Second order is better than first, but still intractable in higher dimensions
- Moreover, it's difficult (or impossible) to construct higher order methods in higher dimensions (and overfitting is a concern too)



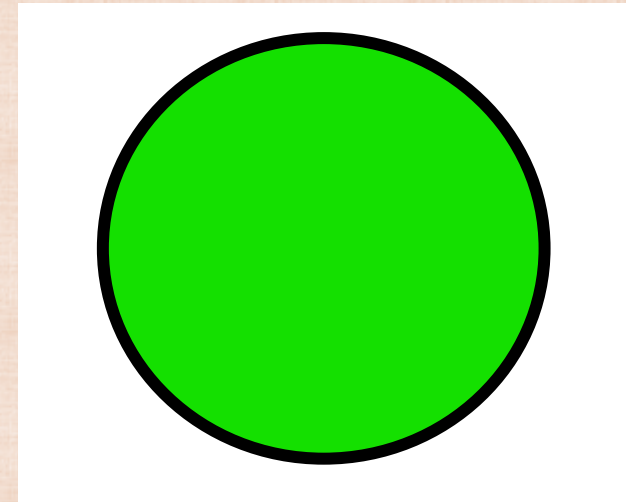
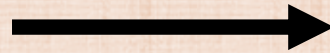
# Conclusion

- Newton-Cotes style approaches are only practical for 1D/2D/3D
  - or 1D/2D/3D + time
- Sometimes they can work ok in 4D
- They are never a good idea in 5D+



# A Simple Example

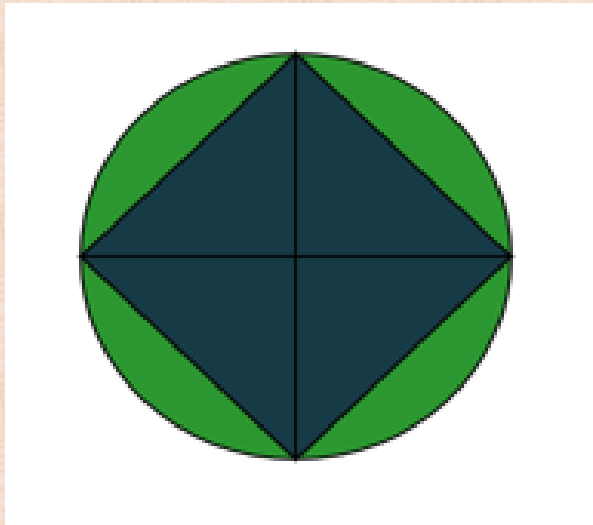
- Consider approximating  $\pi = 3.1415926535 \dots$
- Use a compass to construct a circle with radius = 1
- Since  $A = \pi r^2$ , the area of this unit circle is  $\pi$
- Integrate  $f(x, y) = 1$  over the unit circle to obtain  $\iint_A f(x, y) dA = \pi$



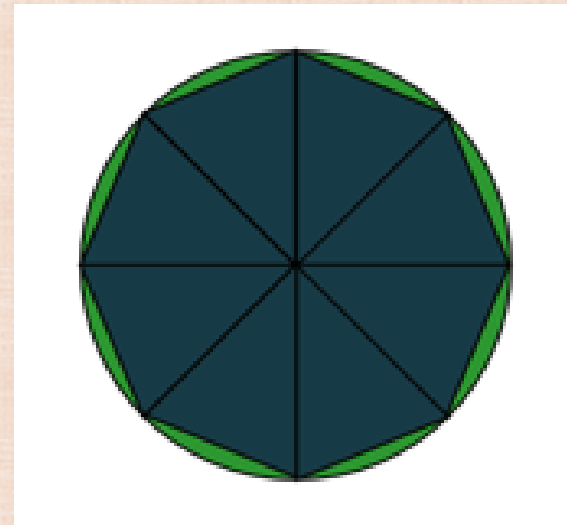
*Area =  $\pi$*

# Newton-Cotes Approach

- Inscribe triangles inside the circle
- Sum the area of all the triangles (no need to trivially multiply by the height = 1)
- The difference between the area  $A$  and its approximation with triangles leads to errors



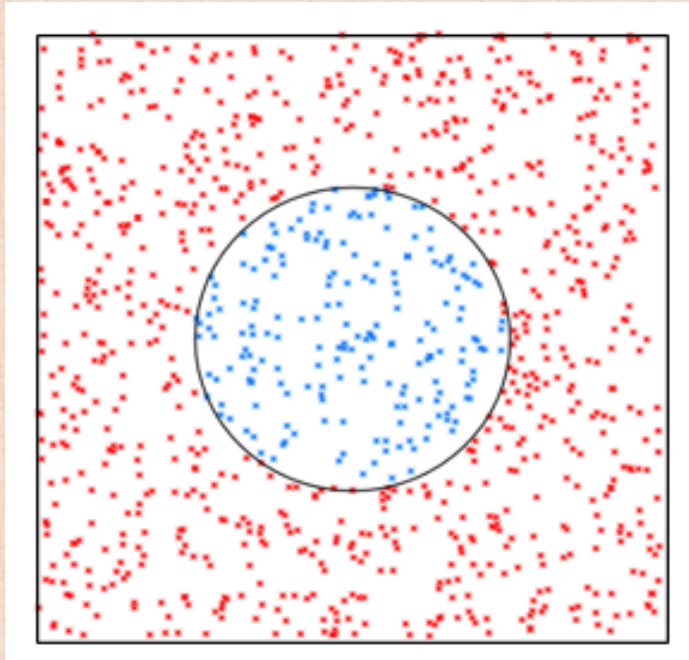
$$\pi \approx 2$$



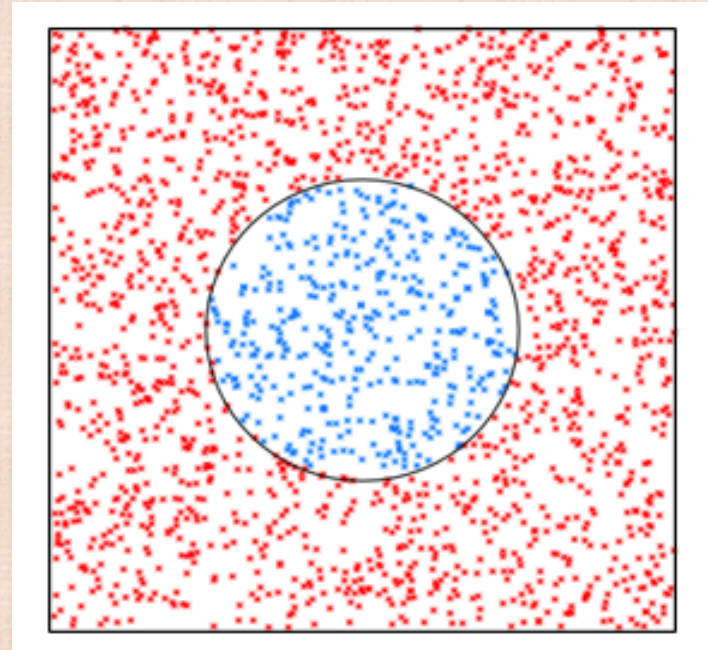
$$\pi \approx 2.8284$$

# Monte Carlo Approach

- Construct a square with side length 4 containing the circle
- Randomly generate  $N$  points in the square (color points inside the circle blue)
- Since  $\frac{A_{circle}}{A_{box}} = \frac{\pi}{16}$ , can approximate  $\pi \approx 16 \left( \frac{N_{blue}}{N_{blue} + N_{red}} \right)$



$$\pi \approx 3.136$$



$$\pi \approx 3.1440$$

# Monte Carlo Integration

- Typically used in higher dimensions (5D or more)
- Random (pseudo-random) numbers generate sample points that are multiplied by “element size” (e.g. length, area, volume, etc.)
- Error decreases like  $\frac{1}{\sqrt{N}}$  where N is the number of samples (only ½ order accurate)
  - E.g. 4 times more sample points are needed for ½ the error
  - E.g. 100 times more sample points are needed to gain one more digit of accuracy
- **Very slow convergence, but independent of the number of dimensions!**
- Not competitive for lower dimensional problems (i.e., 1D, 2D, 3D), but the only tractable approach for high dimensional problems



# Machine Learning Implications

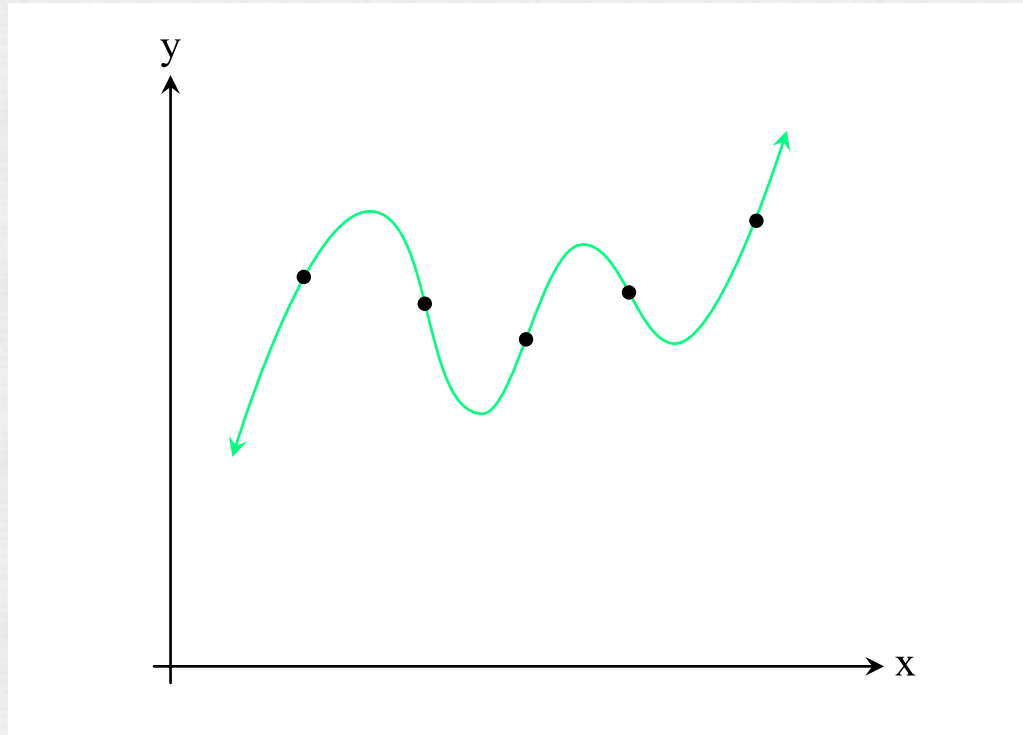
- Consider  $y = f(x)$  where  $x \in R^n$  with large  $n$
- Newton-Cotes style approaches would first do polynomial interpolation, and then analytically integrate the result
- An intractable number of points and control volumes is required to construct polynomial functions in higher dimensions (the curse of dimensionality)
- The same is true when constructing an approximation to  $y = f(x)$  for function interpolation for the sake of neural network inference, i.e. it is intractable in higher dimensions (too many control volumes, too much data, etc.)
  - However, it's still a good way to visualize or think about things to build intuition, especially since intuition is best built in  $R^1$ ,  $R^2$ , and  $R^3$
- Monte Carlo approaches are far more efficient in higher dimensions
  - This is a major reason for the close collaborations between CS ML/DL folks and Statistics departments
  - Classical engineering disciplines tend to operate in a lower dimensional 3D model of the physical world, and thus they typically have closer ties to Applied Mathematics (than to Statistics)

# Unit 8

# Least Squares

# Recall: Polynomial Interpolation (Unit 1)

- Given  $m$  data points, one can (at best) draw a unique  $m - 1$  degree polynomial that goes through all of them
  - As long as they're not degenerate (e.g. 3 points on a line)



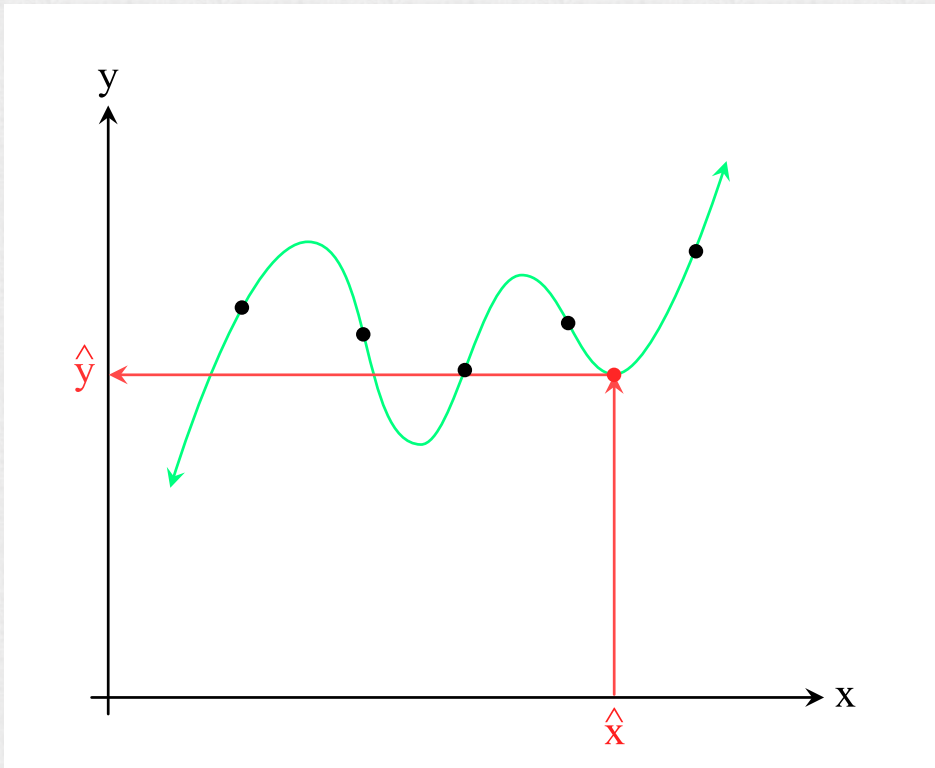


# Recall: Basis Functions (Unit 1)

- Given basis functions  $\phi$  and unknown parameters  $c$ :
$$y = c_1\phi_1 + c_2\phi_2 + \cdots + c_n\phi_n$$
- Monomial basis:  $\phi_k(x) = x^{k-1}$
- Lagrange basis:  $\phi_k(x) = \frac{\prod_{i \neq k} x - x_i}{\prod_{i \neq k} x_k - x_i}$
- Newton basis:  $\phi_k(x) = \prod_{i=1}^{k-1} x - x_i$
- Write a (linear) equation for each data point, and put into matrix form:  $Ac = y$
- Monomial/Lagrange/Newton basis all give the same polynomial, but different matrices need to be inverted in each case

# Recall: Overfitting (Unit 1)

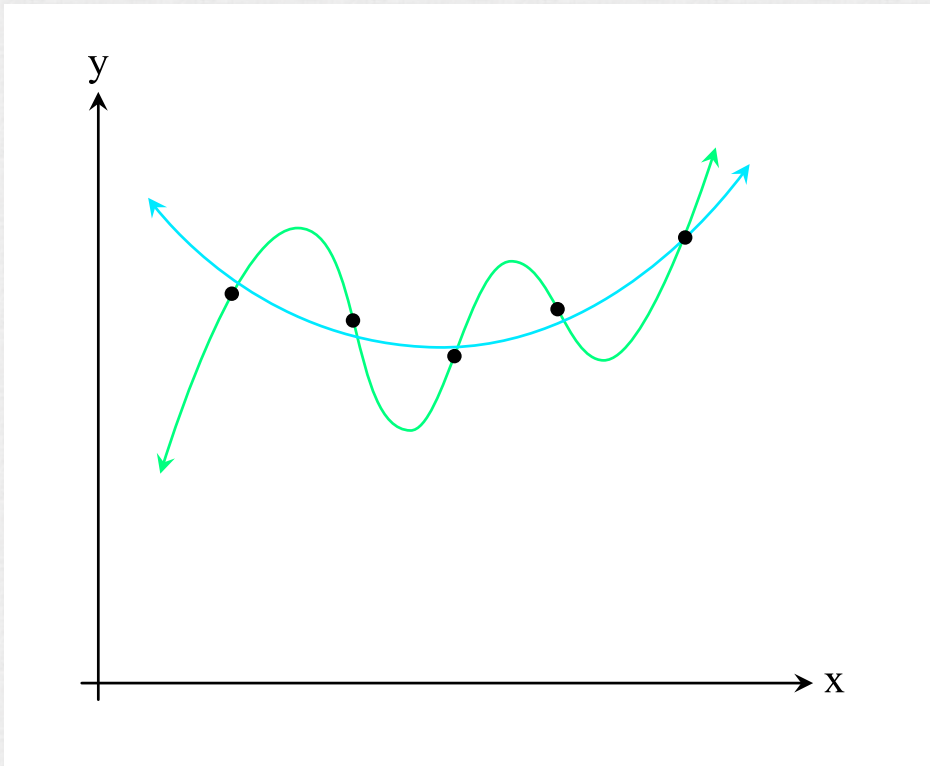
- Given a new input  $\hat{x}$ , the interpolating polynomial infers/predicts an output  $\hat{y}$  that may be far from what one may expect



- Interpolating polynomials are smooth (continuous function and derivatives)
- Thus, they wiggle/overshoot in between data points (so that they can smoothly turn back and hit the next point)
- Overly forcing polynomials to exactly hit every data point is called overfitting (overly fitting to the data)
- Overfitting leads to inference/predictions that can vary wildly from the training data

# Recall: Regularization (Unit 1)

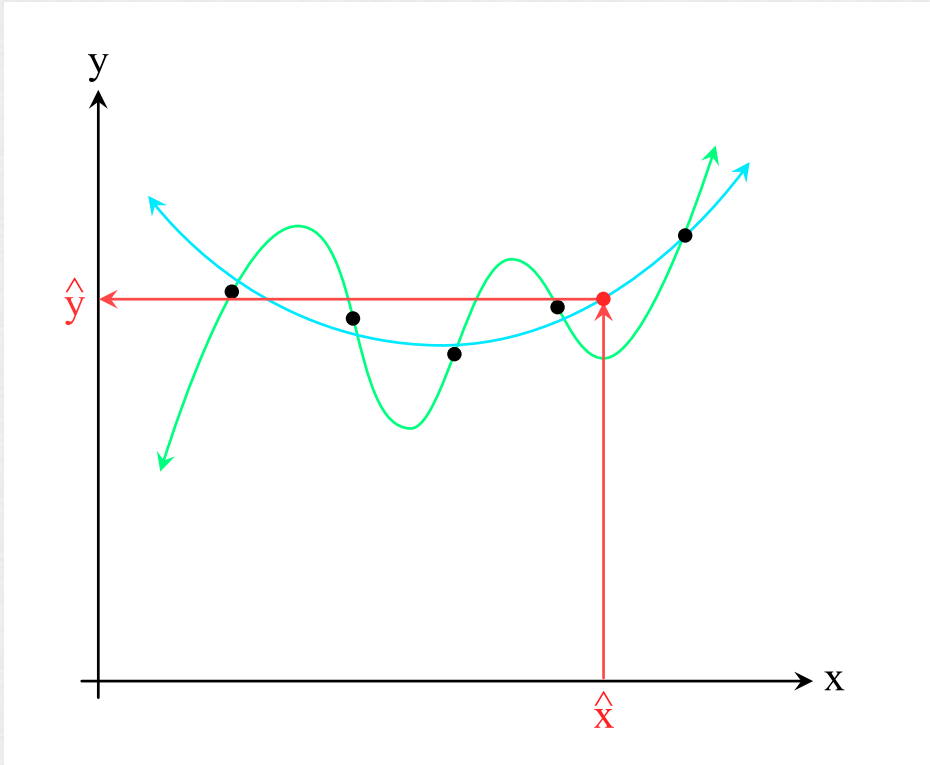
- Using a **lower order polynomial** that doesn't (can't) exactly fit the data points provides regularization



- A regularized interpolant contains intentional error, missing some/all of the data points
- However, this hopefully makes the function more predictable/smooth in between the data points
- The data points themselves may contain noise/error, so it is not clear whether they should be interpolated exactly anyways

# Recall: Regularization (Unit 1)

- Given  $\hat{x}$ , the regularized interpolant infers/predicts a more reasonable  $\hat{y}$

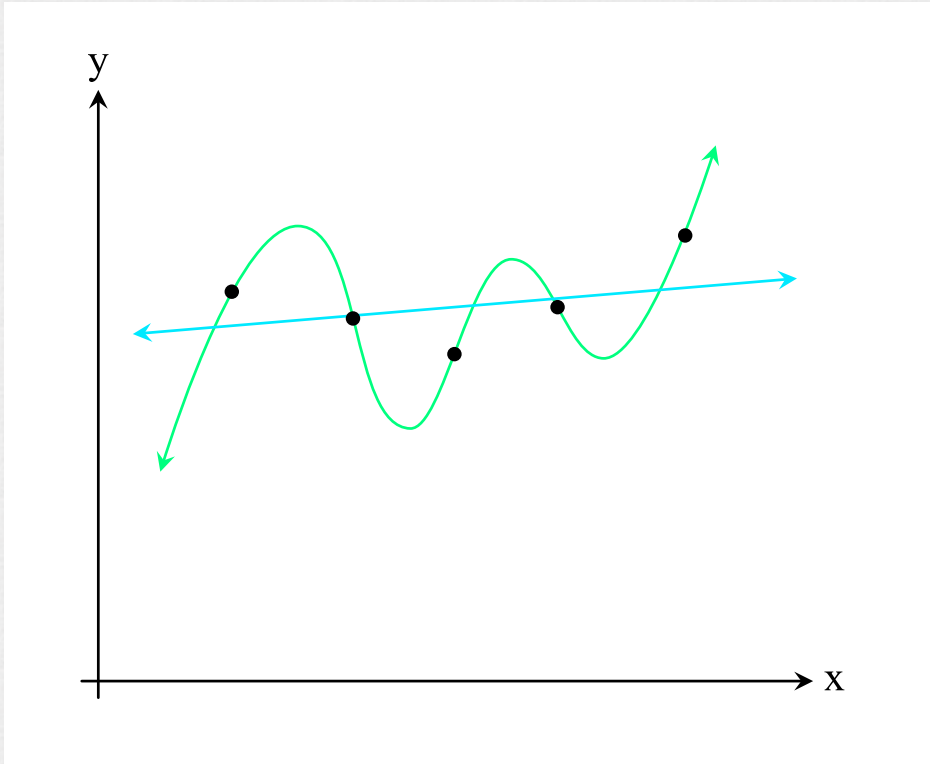


- There is a trade-off between sacrificing accuracy on fitting the original input data, and obtaining better accuracy on inference/prediction for new inputs



# Recall: Underfitting (Unit 1)

- Using too low of an order polynomial causes it to miss the data by too much



- A linear function doesn't capture the essence of this data as well as a quadratic function does
- Choosing too simple of a model function or regularizing too much prevents one from properly representing the data

# Eliminating Basis Functions

- Consider  $Ac = y$ :
  - Each row of  $A$  evaluates all  $n$  basis functions  $\phi_k$  on a single data point  $x_i$
  - Each column of  $A$  evaluates all  $m$  data points  $x_i$  on a single basis function  $\phi_k$
- Regularize by reducing the number of basis functions (**remove columns**)
  - This reduces the degree of the polynomial
  - Still write an equation for each point, and put into matrix form  $Ac = y$  (as usual)
- When there are more points than basis functions, there will be more rows than columns (the matrix will be tall/rectangular)
  - This tall matrix has full (column) rank when the columns are linearly independent
  - This requires both well-chosen basis functions and non-degenerate data

# Understanding Tall Full Rank Matrices

- Let  $A$  be a size  $m \times n$  tall (i.e.  $m \geq n$ ) matrix with full (column) rank (i.e. rank  $n$ )
- Since there are  $n$  entries in each row, the rows span at most an  $n$  dimensional space; thus, at least  $m - n$  rows are linear combinations of others
  - That is,  $A$  contains at least  $m - n$  extra/unnecessary equations that are linear combinations of other equations
  - Thus,  $A$  could be reduced to  $n$  equations (and size  $n \times n$ ) without losing any information
- The SVD ( $A = U\Sigma V^T$ ) illustrates this with: the last  $m - n$  rows of  $\Sigma$  are all zeros
  - The last  $m - n$  columns in  $U$  are hit by these zeros, and thus not in the range of  $A$

## Recall: Example (Unit 3)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix}$$

- Singular values are 25.5, 1.29, and 0
- Singular value of 0 indicates that the matrix is rank deficient (only rank 2, not rank 3)
- The rank of a matrix is equal to its number of nonzero singular values



# Recall: Extra Dimensions (Unit 3)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} =$$

$$\begin{pmatrix} .141 & .825 & -.420 & -.51 \\ .344 & .426 & .298 & .78 \\ .547 & .028 & .644 & -.59 \\ .750 & -.371 & -.542 & .09 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ \text{[redacted]} \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix}$$

- The 3D space of vector inputs can only span a 3D subspace of  $R^4$
- The last (**green**) column of  $U$  represents the unreachable dimension, orthogonal to the range of  $A$ , and is always multiplied by 0
- One can delete this column and the associated portion of  $\Sigma$  (and still obtain a valid factorization)

# Understanding Tall **Full Rank** Linear Systems

- $Ac = b$  becomes  $U\Sigma V^T c = b$  or  $\Sigma(V^T c) = (U^T b)$  or  $\Sigma \hat{c} = \hat{b}$
- Solve  $\Sigma \hat{c} = \hat{b}$  by dividing the entries of  $\hat{b}$  by the all nonzero (because full rank) singular values  $\sigma_k$ ; then,  $c = V \hat{c}$
- The last  $m - n$  equations are identically zero on the left, and need to be identically zero on the right as well in order for a solution to exist
  - E.g.  $\begin{pmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \hat{c}_1 \\ \hat{c}_2 \end{pmatrix} = \begin{pmatrix} \hat{b}_1 \\ \hat{b}_2 \\ \hat{b}_3 \end{pmatrix}$  requires  $\hat{b}_3 = 0$  in order for a solution to exist
- The last  $m - n$  columns in  $U$  are not in the range of  $A$ , so  $b$  must be in the span of the first  $n$  columns of  $U$  in order for a solution to exist

# False Statements

- Reasoning with a false statement leads to infinitely more false statements:

$$\begin{aligned}a &= b \\a^2 &= ab \\a^2 - b^2 &= ab - b^2 \\(a + b)(a - b) &= b(a - b) \\a + b &= b \\b + b &= b \\b(1 + 1) &= b(1) \\2 &= 1\end{aligned}$$

- Don't make false statements!

# False Statements

- Reasoning with a false statement leads to infinitely more false statements:

$$\begin{aligned}Ac &= b \\ A^T A c &= A^T b \\ c &= (A^T A)^{-1} (A^T b)\end{aligned}$$


Is it? Is it really?

- Don't make false statements!
- A mix of false/true statements makes it difficult to keep track of what is and what is not true



# False Statements

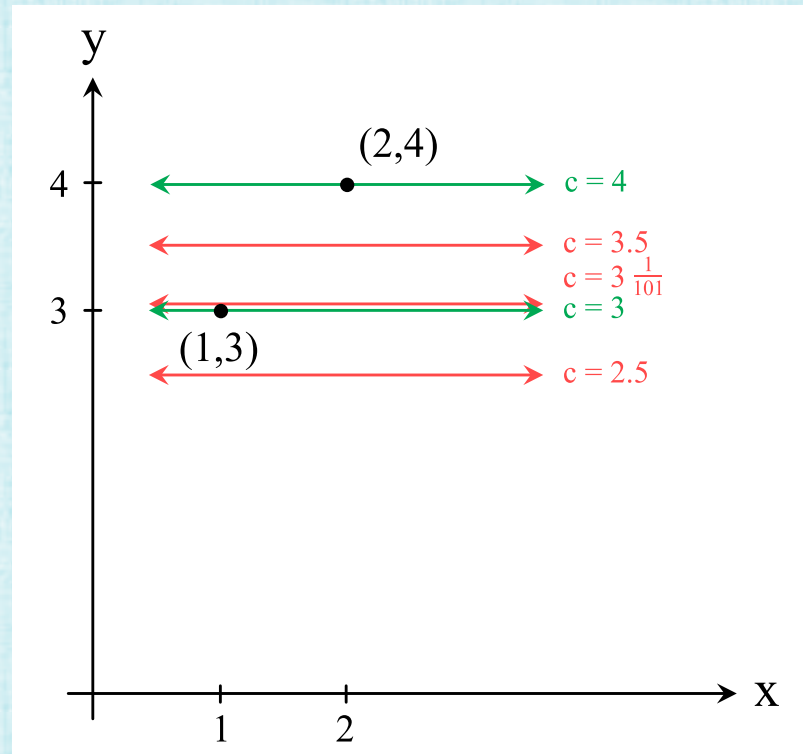
- Consider a very simple  $Ac = b$  given by:  $\begin{pmatrix} 1 \\ 1 \end{pmatrix} (c) = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$
- This contains the equations  $c = 3$  and  $c = 4$ , and as such is a false statement
- Solve via  $\begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} (c) = \begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ , so  $2c = 7$  or  $c = 3.5$
- Row scale the first equation by 10 to obtain:  $\begin{pmatrix} 10 \\ 1 \end{pmatrix} (c) = \begin{pmatrix} 30 \\ 4 \end{pmatrix}$
- Solve via  $\begin{pmatrix} 10 & -1 \end{pmatrix} \begin{pmatrix} 10 \\ 1 \end{pmatrix} (c) = \begin{pmatrix} 10 & -1 \end{pmatrix} \begin{pmatrix} 30 \\ 4 \end{pmatrix}$ , so  $101c = 304$  or  $c = 3 \frac{1}{101} \approx 3.01$
- Perfectly valid row scaling leads to a different answer

# False Statements

- Again, starting with the same:  $\begin{pmatrix} 1 \\ 1 \end{pmatrix} (c) = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$
- Subtract  $2 \times (\text{row 1})$  from  $\text{row 2}$  to obtain  $\begin{pmatrix} 1 \\ -1 \end{pmatrix} (c) = \begin{pmatrix} 3 \\ -2 \end{pmatrix}$
- Solve via  $(1 \quad -1) \begin{pmatrix} 1 \\ -1 \end{pmatrix} (c) = (1 \quad -1) \begin{pmatrix} 3 \\ -2 \end{pmatrix}$ , so  $2c = 5$  or  $c = 2.5$
- A perfectly valid row operation again leads to a different answer
  - Note that  $2.5 \notin [3,4]$  either!
- Issue:  $\begin{pmatrix} 3 \\ 4 \end{pmatrix}$  is not in the range of  $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ , so  $\begin{pmatrix} 1 \\ 1 \end{pmatrix} (c) \neq \begin{pmatrix} 3 \\ 4 \end{pmatrix}$  for  $\forall c \in \mathcal{R}$

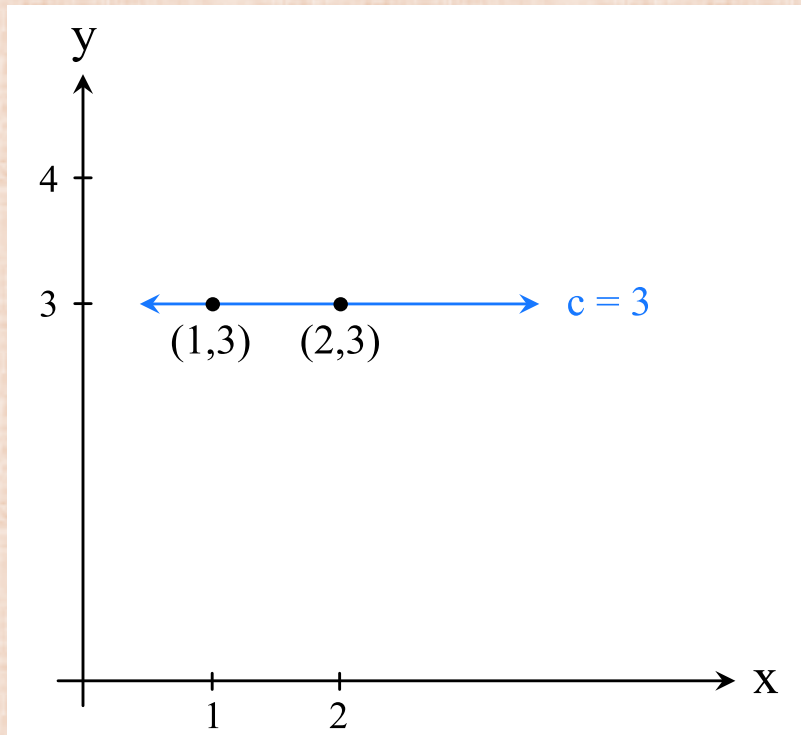
# False Statements

- Consider  $y = c_1 \phi_1$  with monomial  $\phi_1 = 1$ , and data points  $(1,3)$  and  $(2,4)$
- This leads to the same  $\begin{pmatrix} 1 \\ 1 \end{pmatrix} (c_1) = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$



# True Statements

- Consider  $y = c_1 \phi_1$  with monomial  $\phi_1 = 1$ , and data points  $(1,3)$  and  $(2,3)$
- This leads instead to  $\begin{pmatrix} 1 \\ 1 \end{pmatrix} (c_1) = \begin{pmatrix} 3 \\ 3 \end{pmatrix}$  which is valid and has solution  $c_1 = 3$





# True Statements

- When  $b$  is in the range of  $A$ , then  $Ac = b$  is a true statement
  - There exists at least one  $c$  that satisfies this statement
- When  $b$  is in not the range of  $A$ , then  $Ac \neq b$  is the true statement
  - In this case,  $Ac \neq b$  is true for all  $c$
- The equation for the residual  $r = b - Ac$  is always true (it's a definition)
  - When  $b$  is in the range of  $A$ , there exists a  $c$  with  $Ac = b$  and  $r = 0$
  - When  $b$  is not in the range of  $A$ , then  $Ac \neq b$  and  $r \neq 0$  for all  $c$
- The goal in both cases is to minimize the residual  $r = b - Ac$

# Norm Matters

- Consider  $y = c_1 \phi_1$  where  $\phi_1 = 1$  along with data points (1,3), (2,3), and (3,4)
- This leads to  $r = \begin{pmatrix} 3 \\ 3 \\ 4 \end{pmatrix} - \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} (c_1)$
- Setting  $c_1 = 3.5$  minimizes  $\|r\|_\infty$  with  $r = \begin{pmatrix} -.5 \\ -.5 \\ .5 \end{pmatrix}$ ,  $\|r\|_\infty = .5$ ,  $\|r\|_2 = \frac{\sqrt{3}}{2}$
- Setting  $c_1 = 3\frac{1}{3}$  minimizes  $\|r\|_2$  with  $r = \begin{pmatrix} -1/3 \\ -1/3 \\ 2/3 \end{pmatrix}$ ,  $\|r\|_\infty = \frac{2}{3}$ ,  $\|r\|_2 = \frac{\sqrt{6}}{3}$

# Row Operations Matter

- Given a set of equations, they can be manipulated in various ways
- These manipulations typically change the answer
- Thus, **one should carefully choose the residual they want to minimize**
- Equivalent sets of equations lead to different answers when minimizing the corresponding residuals

# Weighted Minimization

- Given  $r = b - Ac$ , some equations may be deemed more important than others
- Scaling entries in the residual (before taking the norm) changes the relative importance of various equations
- This is accomplished by minimizing  $\|Dr\|$  for a diagonal matrix  $D$  with non-zero diagonal entries
- This is equivalent to row scaling:  $Dr = Db - DAc$
- Column scaling doesn't affect the residual, e.g.  $Dr = Db - DA\hat{D}^{-1}(\hat{D}c)$
- So, it can be used to preserve symmetry:  $Dr = Db - (DAD^T)(D^{-T}c)$ 
  - when  $A$  is square and symmetric



# Least Squares

- Minimizing  $\|r\|_2$  is referred to as least squares, and the resulting solution is referred to as **the** least squares solution (it's really **a** least squares solution)
  - A least squares solution **is** the unique solution when  $\|r\|_2 = 0$
- Minimizing  $\|Dr\|_2$  is referred to as weighted least squares
- $\|r\|_2$  is minimized when  $\|r\|_2^2$  is minimized
- And  $\|r\|_2^2 = r \cdot r = (b - Ac) \cdot (b - Ac) = c^T A^T A c - 2b^T A c + b^T b$  is minimized when  $c^T A^T A c - 2b^T A c$  is minimized
- Thus, **minimize  $c^T A^T A c - 2b^T A c$**
- For weighted least squares, **minimize  $c^T A^T D^2 A c - 2b^T D^2 A c$**

# Unit 9

# Basic Optimization

# Jacobian

- The Jacobian of  $F(c) = \begin{pmatrix} F_1(c) \\ F_2(c) \\ \vdots \\ F_m(c) \end{pmatrix}$  has entries  $J_{ik} = \frac{\partial F_i}{\partial c_k}(c)$

- Thus, the Jacobian  $J(c) = F'(c) = \begin{pmatrix} \frac{\partial F_1}{\partial c_1}(c) & \frac{\partial F_1}{\partial c_2}(c) & \cdots & \frac{\partial F_1}{\partial c_n}(c) \\ \frac{\partial F_2}{\partial c_1}(c) & \frac{\partial F_2}{\partial c_2}(c) & \cdots & \frac{\partial F_2}{\partial c_n}(c) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial c_1}(c) & \frac{\partial F_m}{\partial c_2}(c) & \cdots & \frac{\partial F_m}{\partial c_n}(c) \end{pmatrix}$



# Gradient

- Consider the scalar (output) function  $f(c)$  with multi-dimensional input  $c$
- The Jacobian of  $f(c)$  is  $J(c) = \left( \frac{\partial f}{\partial c_1}(c) \quad \frac{\partial f}{\partial c_2}(c) \quad \cdots \quad \frac{\partial f}{\partial c_n}(c) \right)$
- The gradient of  $f(c)$  is  $\nabla f(c) = J^T(c) = \begin{pmatrix} \frac{\partial f}{\partial c_1}(c) \\ \frac{\partial f}{\partial c_2}(c) \\ \vdots \\ \frac{\partial f}{\partial c_n}(c) \end{pmatrix}$
- In 1D, both  $J(c)$  and  $\nabla f(c) = J^T(c)$  are the usual  $f'(c)$

# Critical Points

- To identify critical points of  $f(c)$ , set the gradient to zero:  $\nabla f(c) = 0$

- This is a system of equations: 
$$\begin{pmatrix} \frac{\partial f}{\partial c_1}(c) \\ \frac{\partial f}{\partial c_2}(c) \\ \vdots \\ \frac{\partial f}{\partial c_n}(c) \end{pmatrix} = 0 \quad \text{or} \quad \begin{pmatrix} \frac{\partial f}{\partial c_1}(c) = 0 \\ \frac{\partial f}{\partial c_2}(c) = 0 \\ \vdots \\ \frac{\partial f}{\partial c_n}(c) = 0 \end{pmatrix}$$

- Any  $c$  that simultaneously solves all the equations is a critical point
- In 1D, this is the usual  $f'(c) = 0$

# Jacobian of the Gradient

- Taking the **Jacobian** of the column vector **gradient** gives:

- The  $J(\nabla f(c)) = \begin{pmatrix} \frac{\partial^2 f}{\partial c_1 \partial c_1}(c) & \frac{\partial^2 f}{\partial c_2 \partial c_1}(c) & \cdots & \frac{\partial^2 f}{\partial c_n \partial c_1}(c) \\ \frac{\partial^2 f}{\partial c_1 \partial c_2}(c) & \frac{\partial^2 f}{\partial c_2 \partial c_2}(c) & \cdots & \frac{\partial^2 f}{\partial c_n \partial c_2}(c) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial c_1 \partial c_n}(c) & \frac{\partial^2 f}{\partial c_2 \partial c_n}(c) & \cdots & \frac{\partial^2 f}{\partial c_n \partial c_n}(c) \end{pmatrix}$

- Note:  $\frac{\partial^2 f}{\partial c_2 \partial c_1} = \frac{\partial}{\partial c_2} \left( \frac{\partial f}{\partial c_1} \right) = (f_{c_1})_{c_2} = f_{c_1 c_2}$

# Hessian

- The Hessian of  $f(c)$  is  $H(c) = J(\nabla f(c))^T$  and has entries  $H_{ik} = \frac{\partial^2 f}{\partial c_i \partial c_k}(c)$

- The Hessian is 
$$H(c) = \begin{pmatrix} \frac{\partial^2 f}{\partial c_1^2}(c) & \frac{\partial^2 f}{\partial c_1 \partial c_2}(c) & \cdots & \frac{\partial^2 f}{\partial c_1 \partial c_n}(c) \\ \frac{\partial^2 f}{\partial c_2 \partial c_1}(c) & \frac{\partial^2 f}{\partial c_2^2}(c) & \cdots & \frac{\partial^2 f}{\partial c_2 \partial c_n}(c) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial c_n \partial c_1}(c) & \frac{\partial^2 f}{\partial c_n \partial c_2}(c) & \cdots & \frac{\partial^2 f}{\partial c_n^2}(c) \end{pmatrix}$$

- When the order of differentiation doesn't matter,  $H(c)$  is symmetric
- In 1D, this is the usual  $f''(c)$



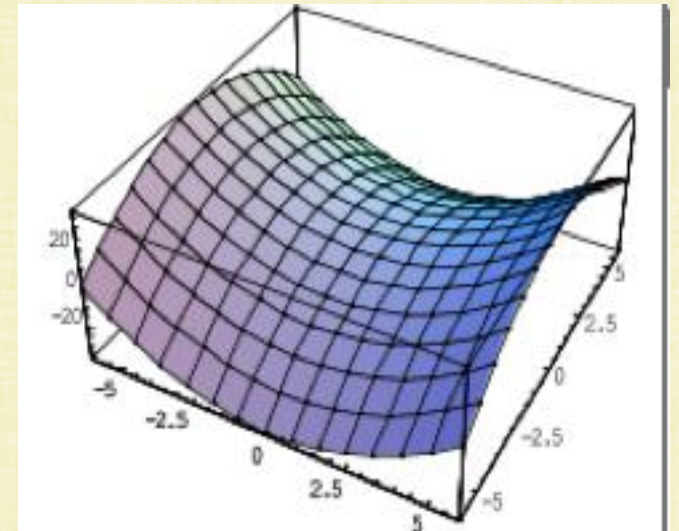
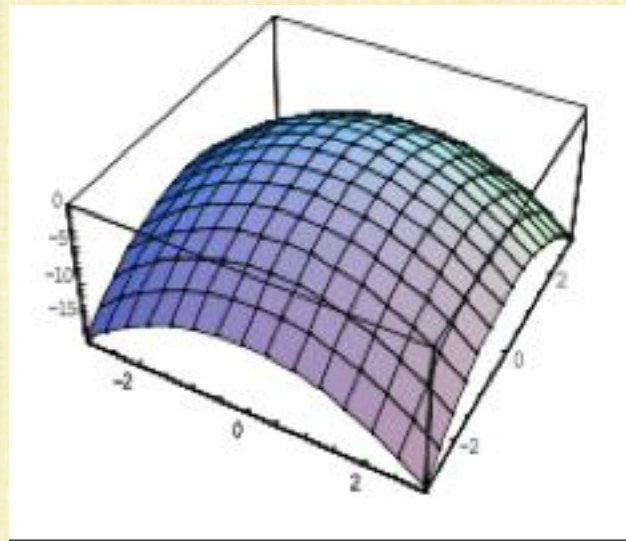
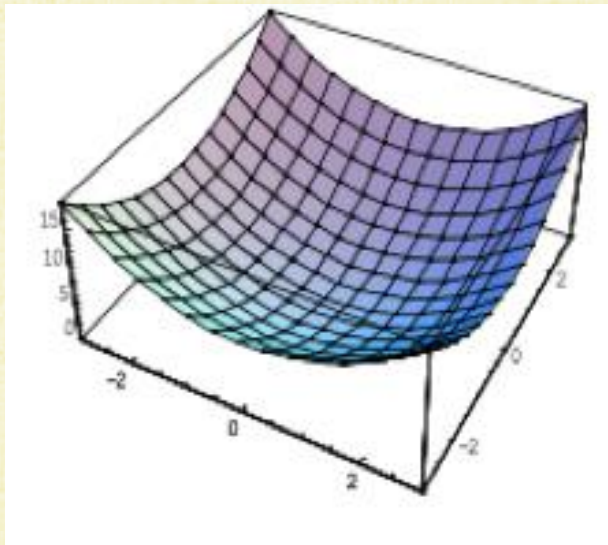
# Differential Forms

- Vector valued function:  $dF(c) = J(F(c))dc$
- Substitute  $\nabla f$  for  $F$  to get:  $d\nabla f(c) = J(\nabla f(c))dc = H^T(c)dc$
- Scalar valued function:  $df(c) = J(f(c))dc$
- Take the transpose:  $df(c) = dc^T \nabla f(c)$
- Take (another) differential:  $d^2 f(c) = J(dc^T \nabla f(c))dc$
- Some hand waving:  $d^2 f(c) = dc^T H^T(c)dc = dc \cdot H^T(c)dc$

# Classifying Critical Points

Given a critical point  $c^*$ , i.e. with  $\nabla f(c^*) = 0$ , the Hessian is used to classify it

- If  $H(c^*)$  is positive definite, then  $c^*$  is a local minimum
- If  $H(c^*)$  is negative definite, then  $c^*$  is a local maximum
- Otherwise,  $H(c^*)$  is indefinite, and  $c^*$  is a saddle point



# Classifying Critical Points (in 1D)

- In 1D, given critical point  $c^*$ , i.e. with  $\nabla f(c^*) = f'(c^*) = 0$ , the Hessian is used to classify it
- In 1D,  $H(c^*) = (f''(c^*))$  is a size  $1 \times 1$  diagonal matrix with eigenvalue  $f''(c^*)$
- If  $H(c^*)$  is positive definite with eigenvalue  $f''(c^*) > 0$ , then  $c^*$  is a local minimum
  - As usual,  $f''(c^*) > 0$  implies concave up and a local min
- If  $H(c^*)$  is negative definite with eigenvalue  $f''(c^*) < 0$ , then  $c^*$  is a local maximum
  - As usual,  $f''(c^*) < 0$  implies concave down and a local max
- Otherwise,  $H(c^*)$  is indefinite with eigenvalue  $f''(c^*) = 0$ , and  $c^*$  is a saddle point
  - As usual,  $f''(c^*) = 0$  implies an inflection point (not a local extrema)



# Quadratic Form

- The quadratic form of a square matrix  $\tilde{A}$  is  $f(c) = \frac{1}{2} c^T \tilde{A} c - \tilde{b}^T c + \tilde{c}$ 
  - In 1D,  $f(c) = \frac{1}{2} \tilde{a} c^2 - \tilde{b} c + \tilde{c}$
- Minimize  $f(c)$  by (first) finding critical points where  $\nabla f(c) = 0$
- Note  $\nabla f(c) = \frac{1}{2} \tilde{A} c + \frac{1}{2} \tilde{A}^T c - \tilde{b}$ , since  $J(c^T v) = J(v^T c) = v^T$  (the gradient is  $v$ )
  - Solve the symmetric system  $\frac{1}{2} (\tilde{A} + \tilde{A}^T) c = \tilde{b}$  to find critical points
- When  $\tilde{A}$  is symmetric,  $\nabla f(c) = \tilde{A} c - \tilde{b} = 0$  is satisfied when  $\tilde{A} c = \tilde{b}$ 
  - In 1D, the critical point is on the line of symmetry  $\tilde{c} = \frac{\tilde{b}}{\tilde{a}}$
- That is, solve  $\tilde{A} c = \tilde{b}$  to find the critical point



# Quadratic Form

The Hessian of  $f(c)$  is  $H = \frac{1}{2}(\tilde{A}^T + \tilde{A})$  or just  $\tilde{A}$  when  $\tilde{A}$  is symmetric

- When  $\tilde{A}$  is SPD, the solution to  $\tilde{A}c = \tilde{b}$  is a minimum
- When  $\tilde{A}$  is symmetric negative definite, the solution to  $\tilde{A}c = \tilde{b}$  is a maximum
- When  $\tilde{A}$  is indefinite, the solution to  $\tilde{A}c = \tilde{b}$  is a saddle point

In 1D,  $H = (\tilde{a})$  is a size  $1 \times 1$  diagonal matrix with eigenvalue  $\tilde{a}$

- As usual,  $\tilde{a} > 0$  implies concave up and a local min
- As usual,  $\tilde{a} < 0$  implies concave down and a local max
- As usual,  $\tilde{a} = 0$  implies an inflection point (not a local extrema)

# Recall: Least Squares (Unit 8)

- Minimizing  $\|r\|_2$  is referred to as least squares, and the resulting solution is referred to as the least squares solution (it's really a least squares solution)
  - A least squares solution is the unique solution when  $\|r\|_2 = 0$
- Minimizing  $\|Dr\|_2$  is referred to as weighted least squares
- $\|r\|_2$  is minimized when  $\|r\|_2^2$  is minimized
- And  $\|r\|_2^2 = r \cdot r = (b - Ac) \cdot (b - Ac) = c^T A^T A c - 2b^T A c + b^T b$  is minimized when  $c^T A^T A c - 2b^T A c$  is minimized
- Thus, minimize  $c^T A^T A c - 2b^T A c$
- For weighted least squares, minimize  $c^T A^T D^2 A c - 2b^T D^2 A c$

# Normal Equations

- $c^T A^T D^2 A c - 2b^T D^2 A c$  has the same minimum as  $\frac{1}{2} c^T A^T D^2 A c - b^T D^2 A c$
- This is a quadratic form with symmetric  $\tilde{A} = A^T D^2 A$  and  $\tilde{b} = A^T D^2 b$
- The critical point is found from solving  $\tilde{A}c = \tilde{b}$  or  $A^T D^2 A c = A^T D^2 b$
- For (unweighted) least squares where  $D = I$ , solve  $A^T A c = A^T b$
- These are called the normal equations

# Hessian

- Recall:  $A$  is a tall (or square) full rank matrix with size  $m \times n$  where  $m \geq n$
- The Hessian  $H = \tilde{A} = A^T A = V \Sigma^T U^T U \Sigma V^T = V \Sigma^T \Sigma V^T = V \Lambda V^T$ 
  - $\Lambda = \Sigma^T \Sigma$  is a size  $n \times n$  matrix of nonzero singular values squared
- $HV = V\Lambda$  illustrates that  $H$  has all positive eigenvalues; so,  $H$  is SPD
- That is, **the critical point is indeed a minimum** (as desired)

For weighted least squares:

- Nonzero diagonal elements in  $D$  implies that  $DAc = 0$  if and only if  $Ac = 0$ 
  - That is, a full column rank  $A$  implies a full column rank  $DA$
- So, the SVD of  $DA$  can be used to prove that  $H = (DA)^T (DA)$  is SPD



# Unit 10

# Solving Least Squares

# Normal Equations

- Let  $\tilde{A}$  have full column rank, and be size  $m \times n$  with  $m \geq n$
- Diagonal (nonzero) weighting  $A = D\tilde{A}$  does not change the rank/size
  - but it does change the answer when both  $D \neq I$  and  $m \neq n$  (when  $A$  is square with  $m = n$ , the unique solution doesn't change via row scaling)
- Minimizing  $\|r\|_2 = \|b - Ac\|_2$  leads to the normal equations  $A^T Ac = A^T b$  for the critical point
- Since  $A^T A$  is SPD,  $A^T Ac = A^T b$  has a unique solution obtainable via fast/efficient SPD solvers
- When  $b$  is in the range of  $A$ , the unique solution to  $A^T Ac = A^T b$  makes  $r = 0$ , and thus it is the unique solution to  $Ac = b$ 
  - Note: when  $A$  is square ( $m = n$ ) and full rank,  $b$  is always in the range of  $A$

# Condition Number for the Normal Equations

- Compare  $A = U\Sigma V^T$  and  $A^T A = V\Sigma^T \Sigma V^T = V\Lambda V^T$  where  $\Lambda = \Sigma^T \Sigma$  is a diagonal size  $n \times n$  matrix of singular values squared
- Since the singular values of  $A^T A$  are the square of those in  $A$ , the condition number  $\frac{\sigma_{\max}^2}{\sigma_{\min}^2}$  of  $A^T A$  is also squared as compared to  $\frac{\sigma_{\max}}{\sigma_{\min}}$  for  $A$ 
  - Thus, solving the normal equations requires twice the precision (e.g.  $(10^7)^2 = 10^{14}$ )
  - It takes twice as much precision to get the same number of significant digits!
- The normal equations are not the preferred approach (unless  $A$  is extremely well conditioned)
  - However, (like the SVD) it is a great for theoretical purposes
  - Can transform any full rank matrix into an SPD system



# Understanding the Normal Equations

- When  $A = U\Sigma V^T$  has full column rank,  $\Sigma = \begin{pmatrix} \hat{\Sigma} \\ 0 \end{pmatrix}$  with  $\hat{\Sigma}$  a size  $n \times n$  diagonal matrix of (strictly) positive singular values
  - The **0 submatrix** is size  $(m - n) \times n$  and doesn't exist when  $m = n$
- Note:  $A^T A = V \begin{pmatrix} \hat{\Sigma} & 0^T \\ 0 \end{pmatrix} \begin{pmatrix} \hat{\Sigma} \\ 0 \end{pmatrix} V^T = V \hat{\Sigma}^2 V^T$  and  $(A^T A)^{-1} = V \hat{\Sigma}^{-2} V^T$
- $c = (A^T A)^{-1} A^T b = V \hat{\Sigma}^{-2} V^T V \begin{pmatrix} \hat{\Sigma} & 0^T \\ 0 \end{pmatrix} U^T b = V \begin{pmatrix} \hat{\Sigma}^{-1} & 0^T \\ 0 & 0^T \end{pmatrix} U^T b$
- $Ac = U \begin{pmatrix} \hat{\Sigma} \\ 0 \end{pmatrix} V^T V \begin{pmatrix} \hat{\Sigma}^{-1} & 0^T \\ 0 & 0^T \end{pmatrix} U^T b = U \begin{pmatrix} I_{n \times n} & 0^T \\ 0 & 0^T \end{pmatrix} U^T b$
- $r = b - Ac = U I_{m \times m} U^T b - U \begin{pmatrix} I_{n \times n} & 0 \\ 0 & 0^T \end{pmatrix} U^T b = U \begin{pmatrix} 0^T 0 & 0^T \\ 0 & I_{(m-n) \times (m-n)} \end{pmatrix} U^T b$

# Recall: Summary (Unit 3)

- The columns of  $V$  that do not correspond to “nonzero” singular values form an orthonormal basis for the null space of  $A$
- The remaining columns of  $V$  form an orthonormal basis for the space perpendicular to the null space of  $A$  (parameterizing meaningful inputs)
- The columns of  $U$  corresponding to “nonzero” singular values form an orthonormal basis for the range of  $A$
- The remaining columns of  $U$  form an orthonormal basis for the (unattainable) space perpendicular to the range of  $A$
- One can drop the columns of  $U$  and  $V$  that do not correspond to “nonzero” singular values and still obtain a valid factorization of  $A$
- One can drop the columns of  $U$  and  $V$  that correspond to “smaller” singular values and still obtain a reasonable approximation of  $A$

# Understanding the Normal Equations

- $A$  has  $n$  singular values
- So, only the first  $n$  columns of  $U$  (which has  $m$  columns) span the range of  $A$
- Write  $\begin{pmatrix} \hat{b}_r \\ \hat{b}_z \end{pmatrix} = U^T b$
- $\hat{b}_r$  is size  $n \times 1$ , and it corresponds to the part of  $b$  in the range of  $A$
- $\hat{b}_z$  is size  $(m - n) \times 1$ , and it corresponds to the part of  $b$  orthogonal to the range of  $A$
- Then:  $c = V \hat{\Sigma}^{-1} \hat{b}_r$ ,  $Ac = U \begin{pmatrix} \hat{b}_r \\ 0 \end{pmatrix}$ , and  $r = U \begin{pmatrix} 0 \\ \hat{b}_z \end{pmatrix}$

# Recall: Singular Value Decomposition (Unit 3)

- Factorization of any (size  $m \times n$ ) matrix:  $A = U \Sigma V^T$
- $\Sigma$  is  $m \times n$  diagonal with non-negative diagonal entries (called singular values)
- $U$  is  $m \times m$  orthogonal,  $V$  is  $n \times n$  orthogonal (their columns are called singular vectors)
  - Orthogonal matrices have orthonormal columns (an orthonormal basis), so their transpose is their inverse. They preserve inner products and thus are rotations, reflections, and combinations thereof
  - If  $A$  has complex entries, then  $U$  and  $V$  are unitary (conjugate transpose is their inverse)
- Introduced and rediscovered many times: Beltrami 1873, Jordan 1875, Sylvester 1889, Autonne 1913, Eckart and Young 1936. Pearson introduced principal component analysis (PCA) in 1901, which uses SVD. Numerical methods by Chan, Businger, Golub, Kahan, etc.



## Aside: Orthogonal Matrices and the L2 norm

- An orthogonal  $\hat{Q}$  has  $\hat{Q}\hat{Q}^T = \hat{Q}^T\hat{Q} = I$

- So,

$$\|\hat{Q}r\|_2 = \sqrt{\hat{Q}r \cdot \hat{Q}r} = \sqrt{r^T \hat{Q}^T \hat{Q}r} = \sqrt{r^T r} = \|r\|_2$$

$$\|\hat{Q}^T r\|_2 = \sqrt{\hat{Q}^T r \cdot \hat{Q}^T r} = \sqrt{r^T \hat{Q} \hat{Q}^T r} = \sqrt{r^T r} = \|r\|_2$$

- That is, orthogonal transformations preserve Euclidean distance

# Understanding the Normal Equations

- Since  $U$  is orthogonal,  $\|r\|_2 = \left\| U \begin{pmatrix} 0 \\ \hat{b}_z \end{pmatrix} \right\|_2 = \left\| \begin{pmatrix} 0 \\ \hat{b}_z \end{pmatrix} \right\|_2 = \|\hat{b}_z\|_2$

Recall the SVD view of  $Ac = b$  in the case where  $A$  is full rank:

- Then,  $U\Sigma V^T c = b$  gives  $\begin{pmatrix} \hat{\Sigma} \\ 0 \end{pmatrix} \hat{c} = \begin{pmatrix} \hat{b}_r \\ \hat{b}_z \end{pmatrix}$
- The first block row leads to  $c = V\hat{\Sigma}^{-1}\hat{b}_r$ , just like for the normal equations
- The norm of the residual is  $\left\| \begin{pmatrix} \hat{b}_r \\ \hat{b}_z \end{pmatrix} - \begin{pmatrix} \hat{\Sigma} \\ 0 \end{pmatrix} \hat{c} \right\|_2 = \left\| \begin{pmatrix} 0 \\ \hat{b}_z \end{pmatrix} \right\|_2 = \|\hat{b}_z\|_2$ , just like for the normal equations
- The normal equations approach (derived from minimizing  $\|r\|_2$ ) agrees with the solution obtained via the SVD

# Summary

- Minimizing  $\|r\|_2$  leads to the normal equations
- The SVD leads to the same solution (and residual) as the normal equations
- Neither approach is a good idea
- Instead, we will use a new matrix factorization
- The  $QR$  factorization:

$$A = QR \text{ where } R \text{ is upper triangular and } Q^T Q = I$$

# Recall: Gram-Schmidt (Unit 5)

- Orthogonalizes a set of vectors
- For each new vector, subtract its (weighted) dot product overlap with all prior vectors, making it orthogonal to them
- A-orthogonal Gram-Schmidt uses an A-weighted dot/inner product
- Given candidate vector  $\bar{S}^q$ , subtract out the A-overlap with  $s^1$  to  $s^{q-1}$  so that the resulting vector  $s^q$  has  $\langle s^q, s^{\hat{q}} \rangle_A = 0$  for  $\hat{q} \in \{1, 2, \dots, q-1\}$
- That is,  $s^q = \bar{S}^q - \sum_{\hat{q}=1}^{q-1} \frac{\langle \bar{S}^q, s^{\hat{q}} \rangle_A}{\langle s^{\hat{q}}, s^{\hat{q}} \rangle_A} s^{\hat{q}}$  where the two non-normalized  $s^{\hat{q}}$  both require division by their norm
  - Note:  $\langle s^{\hat{q}}, s^{\hat{q}} \rangle_A = \|s^{\hat{q}}\|_A^2$
  - Proof:  $\langle s^q, s^{\tilde{q}} \rangle_A = \langle \bar{S}^q, s^{\tilde{q}} \rangle_A - \frac{\langle \bar{S}^q, s^{\tilde{q}} \rangle_A}{\langle s^{\tilde{q}}, s^{\tilde{q}} \rangle_A} \langle s^{\tilde{q}}, s^{\tilde{q}} \rangle_A = 0$  for  $\tilde{q} \in \{1, 2, \dots, q-1\}$



# Gram-Schmidt for QR Factorization

- From  $A$ , create a full rank  $Q$  with orthonormal columns
- For each column  $a_k$ , subtract the overlap with all prior columns in  $Q$  and make the result unit length:

$$q_k = \frac{a_k - \sum_{\hat{k}=1}^{k-1} \langle a_k, q_{\hat{k}} \rangle q_{\hat{k}}}{\left\| a_k - \sum_{\hat{k}=1}^{k-1} \langle a_k, q_{\hat{k}} \rangle q_{\hat{k}} \right\|_2}$$

- Define  $r_{\hat{k}k} = \langle a_k, q_{\hat{k}} \rangle$  for  $k > \hat{k}$ , and  $r_{kk} = \left\| a_k - \sum_{\hat{k}=1}^{k-1} \langle a_k, q_{\hat{k}} \rangle q_{\hat{k}} \right\|_2$
- Then  $q_k = \frac{a_k - \sum_{\hat{k}=1}^{k-1} r_{\hat{k}k} q_{\hat{k}}}{r_{kk}}$ , and thus  $a_k = r_{kk} q_k + \sum_{\hat{k}=1}^{k-1} r_{\hat{k}k} q_{\hat{k}} = \sum_{\hat{k}=1}^k r_{\hat{k}k} q_{\hat{k}}$
- This gives  $A = QR$  where  $R$  is upper triangular and  $Q^T Q = I$

# Gram-Schmidt for QR (an example)

- Example:  $A = QR$  with upper triangular  $R$

$$\begin{pmatrix} 3 & -3 & 3 \\ 2 & -1 & 1 \\ 2 & -1 & -1 \\ 2 & -3 & 3 \\ 2 & -3 & 5 \end{pmatrix} = \begin{pmatrix} 3/5 & 0 & 0 \\ 2/5 & 1/2 & 1/2 \\ 2/5 & 1/2 & -1/2 \\ 2/5 & -1/2 & -1/2 \\ 2/5 & -1/2 & 1/2 \end{pmatrix} \begin{pmatrix} 5 & -5 & 5 \\ 0 & 2 & -4 \\ 0 & 0 & 2 \end{pmatrix}$$

- Note that  $Q^T Q = I_{3 \times 3}$  since the columns of  $Q$  are orthonormal
- However,  $Q Q^T \neq I_{5 \times 5}$  since  $Q$  is only a subset of an orthogonal matrix

# Not Good for Large Matrices

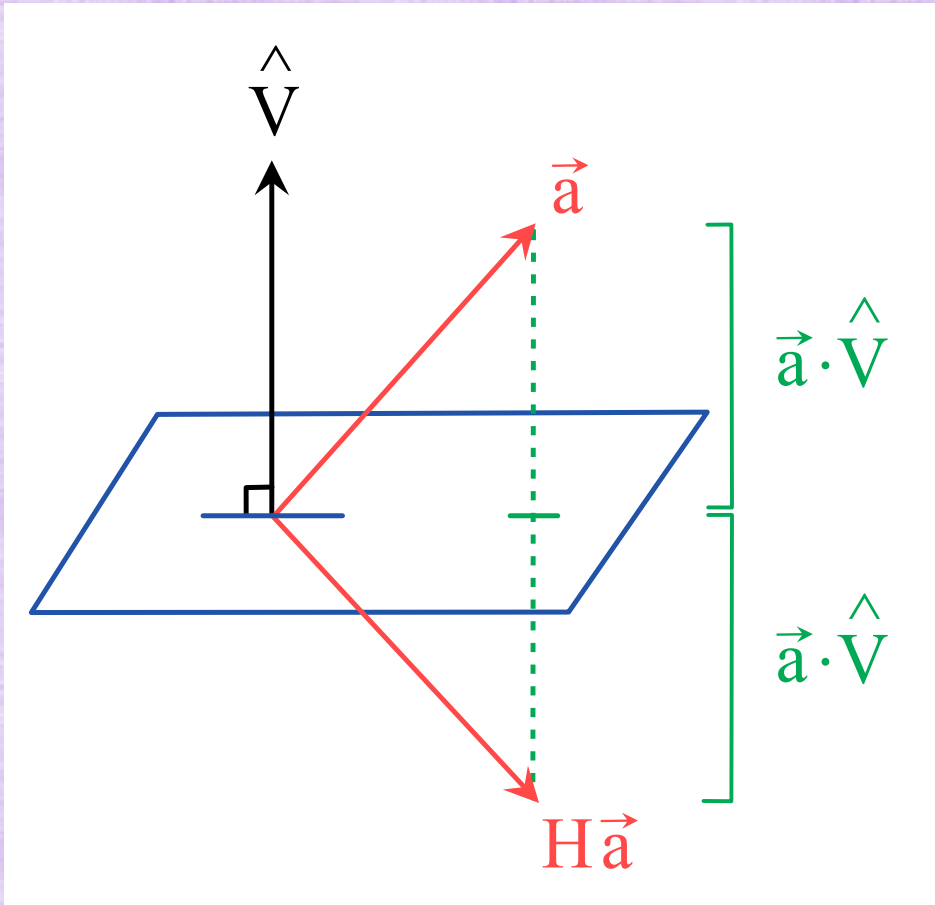
- Gram-Schmidt has too much numerical drift, when used on a large number of vectors
- So, we won't use Gram-Schmidt to find  $A = QR$
- However, Gram-Schmidt is a good way to conceptualize the  $QR$  factorization

# Understanding QR Factorization

- Consider  $A = QR$  with an upper triangular  $R$  and  $Q^T Q = I$
- $Q$  is size  $m \times n$  (just like  $A$ ) with  $n$  orthonormal columns
- Let  $\tilde{Q}$  be the matrix that contains the  $m - n$  orthonormal columns that span the space perpendicular to the range of  $Q$
- Then, the size  $m \times m$  matrix  $\hat{Q} = (Q \quad \tilde{Q})$  is an orthogonal matrix
- $\|r\|_2 = \|\hat{Q}^T r\|_2 = \left\| \begin{pmatrix} Q^T \\ \tilde{Q}^T \end{pmatrix} (b - QRc) \right\|_2 = \left\| \begin{pmatrix} Q^T b - Rc \\ \tilde{Q}^T b \end{pmatrix} \right\|_2 = \left\| \begin{pmatrix} \hat{b}_Q - Rc \\ \hat{b}_{\tilde{Q}} \end{pmatrix} \right\|_2$
- The first block row shows that  $\|r\|_2$  is minimized by solving  $Rc = Q^T b$ 
  - Since  $R$  is upper triangular, this can be solved via back-substitution
  - This results in  $\|r\|_2 = \|\tilde{Q}^T b\|_2 = \|\hat{b}_{\tilde{Q}}\|_2$  as the minimum value for  $\|r\|_2$



# Householder Transform



- A unit normal  $\hat{v}$  implicitly defines a plane orthogonal to it
- $H = I - 2\hat{v}\hat{v}^T$  reflects vectors across that plane
- $Ha = a - 2(\hat{v}^T a) \hat{v}$
- $H$  is orthogonal with  $H = H^T = H^{-1}$
- Don't form the size  $m \times m$  matrix  $H$
- Instead, apply it via  $\hat{v}$

# Householder Transform for QR

- Choose the directions  $v_k = a - Ha$  in order to zero out elements

- E.g.  $v_k = \begin{pmatrix} a_1 \\ \vdots \\ a_{k-1} \\ a_k \\ a_{k+1} \\ \vdots \\ a_n \end{pmatrix} - \begin{pmatrix} a_1 \\ \vdots \\ a_{k-1} \\ \gamma \\ 0 \\ \vdots \\ 0 \end{pmatrix} = \hat{a}_k - \gamma \hat{e}_k$  where  $\hat{a}_k = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ a_k \\ a_{k+1} \\ \vdots \\ a_n \end{pmatrix}$

- As a reflection,  $Ha$  has the same length as  $a$ , so  $\gamma = \pm \|\hat{a}_k\|_2$
- For robustness,  $v_k = \hat{a}_k + S(a_k) \|\hat{a}_k\|_2 \hat{e}_k$  where  $S(a_k) = \pm 1$  is the sign function
- Finally,  $v_k$  is normalized to  $\hat{v}_k = \frac{v_k}{\|v_k\|_2}$

# Householder Transform (an example)

- Let  $a_1 = \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix}$  and consider  $v_1 = \hat{a}_1 + S(a_1)\|\hat{a}_1\|_2\hat{e}_1$
- Then,  $\hat{a}_1 = \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix}$ ,  $v_1 = \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix} + S(2)\sqrt{9}\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \\ 2 \end{pmatrix}$ ,  $\hat{v}_1 = \frac{1}{\sqrt{30}}\begin{pmatrix} 5 \\ 1 \\ 2 \end{pmatrix}$
- So,  $H_1 a_1 = a_1 - 2(\hat{v}_1^T a_1) \hat{v}_1 = \begin{pmatrix} 2 \\ 1 \\ 2 \end{pmatrix} - 2\frac{15}{\sqrt{30}}\frac{1}{\sqrt{30}}\begin{pmatrix} 5 \\ 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -3 \\ 0 \\ 0 \end{pmatrix}$

# Householder Transform (another example)

- Let  $a_2 = \begin{pmatrix} 6 \\ 3 \\ 4 \end{pmatrix}$  and consider  $v_2 = \hat{a}_2 + S(a_2)\|\hat{a}_2\|_2\hat{e}_2$
- Then,  $\hat{a}_2 = \begin{pmatrix} 0 \\ 3 \\ 4 \end{pmatrix}$ ,  $v_2 = \begin{pmatrix} 0 \\ 3 \\ 4 \end{pmatrix} + S(3)\sqrt{25}\begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 8 \\ 4 \end{pmatrix}$ ,  $\hat{v}_2 = \frac{1}{\sqrt{80}}\begin{pmatrix} 0 \\ 8 \\ 4 \end{pmatrix}$
- So,  $H_2 a_2 = a_2 - 2(\hat{v}_2^T a_2) \hat{v}_2 = \begin{pmatrix} 6 \\ 3 \\ 4 \end{pmatrix} - 2 \frac{40}{\sqrt{80}} \frac{1}{\sqrt{80}} \begin{pmatrix} 0 \\ 8 \\ 4 \end{pmatrix} = \begin{pmatrix} 6 \\ -5 \\ 0 \end{pmatrix}$



# Householder Transform for QR

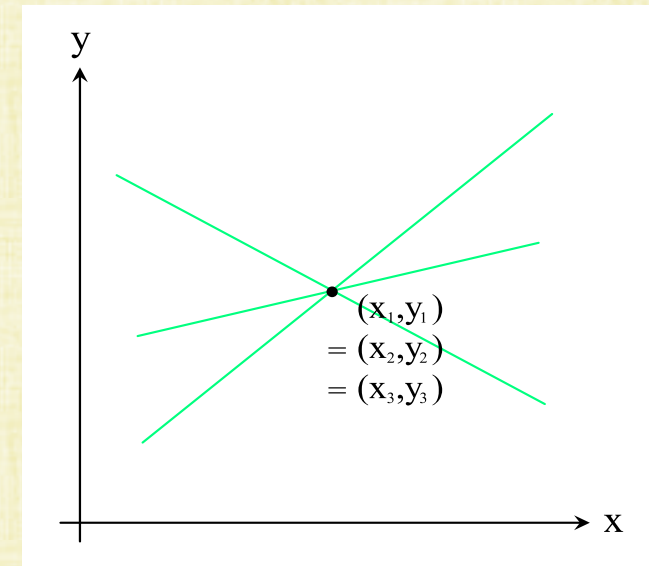
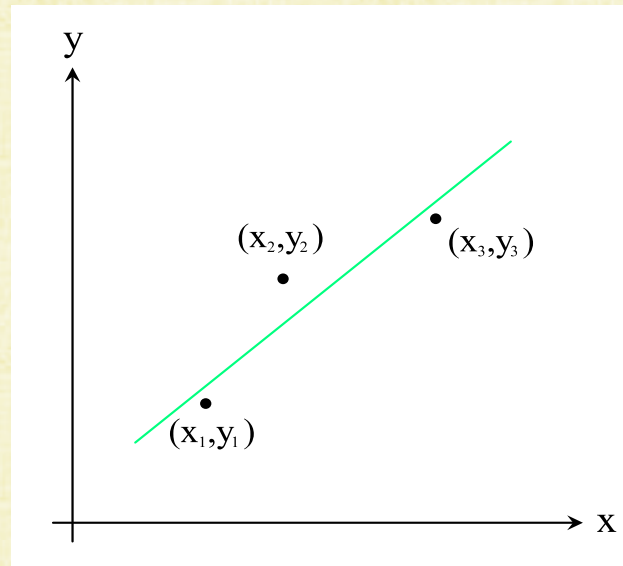
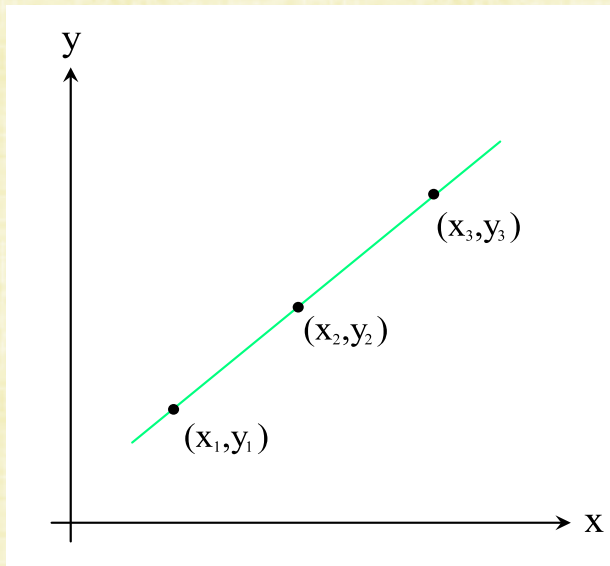
- For each column of  $A$ , construct the Householder transform that zeroes out entries below the diagonal (similar to Gaussian elimination)
- Then  $H_n H_{n-1} \cdots H_2 H_1 A = \begin{pmatrix} R \\ 0 \end{pmatrix}$  and  $H_n H_{n-1} \cdots H_2 H_1 b = \begin{pmatrix} \hat{b}_Q \\ \hat{b}_{\tilde{Q}} \end{pmatrix}$ 
  - $H_n$  is required to get zeroes at the bottom of the last column
- Apply each  $H_k$  efficiently via  $\hat{v}_k$  to all columns  $\geq k$ 
  - For columns  $\hat{k}$  with  $\hat{k} < k$ ,  $\hat{v}_k^T a_{\hat{k}} = 0$  implies that  $H_k$  has no effect
- Letting  $\hat{Q}^T = H_n H_{n-1} \cdots H_2 H_1$  gives  $\hat{Q}^T A = \begin{pmatrix} R \\ 0 \end{pmatrix}$  or  $A = \hat{Q} \begin{pmatrix} R \\ 0 \end{pmatrix}$
- So,  $\|r\|_2 = \|\hat{Q}^T r\|_2 = \left\| \hat{Q}^T \left( b - \hat{Q} \begin{pmatrix} R \\ 0 \end{pmatrix} c \right) \right\|_2 = \left\| \begin{pmatrix} \hat{b}_Q - R c \\ \hat{b}_{\tilde{Q}} \end{pmatrix} \right\|_2$
- Solve  $Rc = \hat{b}_Q$  via back-substitution to minimize  $\|r\|_2$  to a value of  $\|\hat{b}_{\tilde{Q}}\|_2$

# Unit 11

# Zero Singular Values

# Underdetermined Systems

- Consider drawing a line  $y = c_1 + c_2x$  through 3 data points
- When the points are colinear, there is a unique solution
- When the points are not colinear, there is a least squares solution
- When the points are co-located (i.e. identical), there are infinite solutions





# Underdetermined Systems

- The Vandermonde matrix equation is 
$$\begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$$
- Suppose that  $x_1 = x_2 = x_3$ , so that the columns are multiples of each other and the matrix is only rank 1
- If  $y_1 = y_2 = y_3$ , the right-hand side is in the range of the (rank 1) columns implying infinite solutions
- Otherwise, the right-hand side is not in the range of the columns implying no solutions
  - Toss away the linearly dependent second column along with  $c_2$ , and do least squares on  $c_1$

## (Careful) Variable Classification

- Consider  $\begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 0 \end{pmatrix}$
- The first two rows,  $c_1 = 1$  and  $c_1 = 2$ , overdetermine  $c_1$
- The third row,  $c_2 = 3$ , uniquely determines  $c_2$
- The third column,  $\vec{0}c_3$ , leaves  $c_3$  underdetermined with infinite possibilities
- It's often misleading to classify an entire system (as either having a unique solution, no solution, or infinite solutions)
- Rather, one should do the best they can with each variable

# Understanding Underdetermined Systems

- Transform  $Ac = b$  into  $\Sigma \hat{c} = \hat{b}$  (as usual)
- For each  $\sigma_k \neq 0$ , compute  $\hat{c}_k = \frac{\hat{b}_k}{\sigma_k}$  (as usual)
- When  $\sigma_k = 0$ ,  $\hat{c}_k$  is undefined (moreover, division by a small  $\sigma_k$  is dubious)
- Wide matrices have extra columns of zeros, leaving some  $\hat{c}_k$  undetermined (just like  $\sigma_k = 0$  columns)
- Tall matrices have extra rows with  $0 = \hat{b}_k$  ( $\sigma_k = 0$  rows contribute to this too)
  - Nonzero  $\hat{b}_k$  in such rows implies a nonzero residual



# Understanding Underdetermined Systems

- Could write  $U(\hat{\Sigma} \quad 0)V^T$  for wide matrices, similar to  $A = U \begin{pmatrix} \hat{\Sigma} \\ 0 \end{pmatrix} V^T$  for tall matrices
  - But,  $\hat{\Sigma}$  may contain zeros on the diagonal (this is true for tall rank-deficient matrices too)
- For any matrix, can write  $A = U \begin{pmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} V^T$  with  $\hat{\Sigma}$  diagonal and full rank
- Then,  $\Sigma \hat{c} = \hat{b}$  has the form  $\begin{pmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \hat{c}_r \\ \hat{c}_z \end{pmatrix} = \begin{pmatrix} \hat{b}_r \\ \hat{b}_z \end{pmatrix}$
- $\|r\|_2 = \|U^T(b - Ac)\|_2 = \left\| \begin{pmatrix} \hat{b}_r \\ \hat{b}_z \end{pmatrix} - \begin{pmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \hat{c}_r \\ \hat{c}_z \end{pmatrix} \right\|_2 = \left\| \begin{pmatrix} \hat{b}_r \\ \hat{b}_z \end{pmatrix} - \begin{pmatrix} \hat{\Sigma} \hat{c}_r \\ 0 \end{pmatrix} \right\|_2$
- Solving  $\hat{\Sigma} \hat{c}_r = \hat{b}_r$  to obtain  $\hat{c}_r = \hat{\Sigma}^{-1} \hat{b}_r$  minimizes the residual to  $\|r\|_2 = \|\hat{b}_z\|_2$
- Any values are acceptable for the non-determined  $\hat{c}_z$



# Minimum Norm Solution

- Set  $\hat{c}_z = 0$  to stress that these parameters have no bearing on the solution
- This is more sensical than setting the  $\hat{c}_z$  to some arbitrary nonzero values as if those values mattered
- Example:
  - Consider a variable related to how a hat is worn while driving, which could matter when the hat blocks the sun or keeps longer hair away from the eyes
  - Someone with short hair driving at night would likely have no driving dependence on a hat; so, reporting information about hats is just misleading in those cases
- Finally,  $\mathbf{c} = V\hat{\mathbf{c}} = V \begin{pmatrix} \hat{c}_r \\ \hat{c}_z \end{pmatrix} = V \begin{pmatrix} \hat{\Sigma}^{-1} \hat{b}_r \\ 0 \end{pmatrix} = \sum_{\sigma_k \neq 0} v_k \frac{\hat{b}_k}{\sigma_k} = \sum_{\sigma_k \neq 0} v_k \frac{u_k^T b}{\sigma_k}$

# Pseudo-Inverse

- The minimum norm solution is  $c = \left( \sum_{\sigma_k \neq 0} \frac{v_k u_k^T}{\sigma_k} \right) b = A^+ b$  where the pseudo-inverse is  $A^+ = \sum_{\sigma_k \neq 0} \frac{1}{\sigma_k} v_k u_k^T$
- When  $A$  is square and full rank  $A^+ = A^{-1}$
- Each term is an outer product between corresponding columns of  $U$  and  $V$ , weighted by one over their corresponding singular value
- Each term is a size  $n \times m$  matrix, so this a sum of matrices

# Sum of Rank One Matrices

- $$Ac = U \begin{pmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} V^T c = U \begin{pmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \hat{c}_r \\ \hat{c}_z \end{pmatrix} = U \begin{pmatrix} \hat{\Sigma} \hat{c}_r \\ 0 \end{pmatrix} = \sum_{\sigma_k \neq 0} u_k \sigma_k \hat{c}_k =$$
$$\sum_{\sigma_k \neq 0} u_k \sigma_k v_k^T c = \left( \sum_{\sigma_k \neq 0} \sigma_k u_k v_k^T \right) c$$

- That is,  $A = \sum_{\sigma_k \neq 0} \sigma_k u_k v_k^T$

- Each term is an outer product between corresponding columns of  $U$  and  $V$ , weighted by their corresponding singular value
- Each term is a size  $m \times n$  matrix (the same size as  $A$ )
- Each term is rank 1, since every column in the term is a multiple of  $u_k$

# Recall: Understanding $Ac$ (unit 3)

$$\begin{aligned}
 Ac &= \begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.761 & -.057 & .646 \\ .408 & -.816 & .408 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} \\
 &= \begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} 25.5 & 0 & 0 \\ 0 & 1.29 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} v_1^T c \\ v_2^T c \\ v_3^T c \end{pmatrix} \\
 &= \begin{pmatrix} .141 & .825 & -.420 & -.351 \\ .344 & .426 & .298 & .782 \\ .547 & .028 & .644 & -.509 \\ .750 & -.371 & -.542 & .079 \end{pmatrix} \begin{pmatrix} \sigma_1 v_1^T c \\ \sigma_2 v_2^T c \\ \sigma_3 v_3^T c \\ 0 \end{pmatrix} \\
 &= u_1 \sigma_1 v_1^T c + u_2 \sigma_2 v_2^T c + u_3 \sigma_3 v_3^T c + u_4 0
 \end{aligned}$$

- $Ac$  projects  $c$  onto the basis vectors in  $V$ , scales by the associated singular values, and uses those results as weights on the basis vectors in  $U$



# Matrix Approximation

- Use the  $p$  largest singular values:  $A \approx \sum_{k=1}^p \sigma_k u_k v_k^T$
- The pseudo-inverse is approximated similarly:  $A^+ \approx \sum_{k=1}^p \frac{1}{\sigma_k} v_k u_k^T$
- This **is** the best rank  $p$  approximation to  $A$ , and the main idea behind principal component analysis (PCA)
  - Often, thousands/millions of terms can be thrown away keeping only 10 to 100 terms
- Can instead drop small singular values:  $A \approx \sum_{\sigma_k > \epsilon} \sigma_k u_k v_k^T$
- This makes the pseudo-inverse better conditioned:  $A^+ \approx \sum_{\sigma_k > \epsilon} \frac{1}{\sigma_k} v_k u_k^T$ 
  - This obviously relies on a good choice of  $\epsilon > 0$

# Recall: Approximating $A$ (unit 3)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} \approx$$

$$\begin{pmatrix} .141 & .815 & -.400 & -.551 \\ .344 & .416 & .213 & .781 \\ .547 & .013 & .641 & -.519 \\ .750 & -.371 & -.542 & .019 \end{pmatrix} \begin{pmatrix} 25.5 & & & \\ & & & \\ & & & \\ & & & \end{pmatrix} \begin{pmatrix} .504 & .574 & .644 \\ -.781 & .037 & .016 \\ .133 & .010 & .133 \end{pmatrix}$$

- The first singular value is much bigger than the second, and so represents the vast majority of what  $A$  does (note, the vectors in  $U$  and  $V$  are unit length)
- Thus, one could approximate  $A$  quite well by only using the terms associated with the largest singular value
- This is not a valid factorization, but an approximation (and the idea behind PCA)

# Rank One Updates

- For real time applications (real time decision making, etc.), iteratively add one term at a time (slowly improving the estimate)
- $c = A^+ b \approx \frac{u_1^T b}{\sigma_1} v_1 + \frac{u_2^T b}{\sigma_2} v_2 + \frac{u_3^T b}{\sigma_3} v_3 + \dots$
- Note the efficient ordering of the operations:
  - $u_k^T b$  is  $m$  multiplies, and (after 1 division) the result times  $v_k$  is  $n$  multiplies (for a total of  $m + n$  multiplies)
  - Don't form the size  $n \times m$  matrix!
  - Multiplying the size  $n \times m$  matrix  $v_k u_k^T$  times  $b$  is  $m \cdot n$  multiplies

# Computing the SVD

- $A^T A = V \Sigma^T \Sigma V^T$  so  $(A^T A)V = V(\Sigma^T \Sigma)$
- $AA^T = U \Sigma \Sigma^T U^T$  so  $(AA^T)U = U(\Sigma \Sigma^T)$
- If  $\sigma_k \neq 0$ , then  $\sigma_k^2$  is an eigenvalue of both  $A^T A$  and  $AA^T$  (with eigenvectors  $v_k$  and  $u_k$  respectively)
- Work with the smaller of  $A^T A$  and  $AA^T$ , which are both SP(S)D, to find the eigenvalues  $\sigma_k^2$
- Then,  $\sigma_k^2$  can be used in both  $A^T A$  and  $AA^T$  to find the corresponding eigenvectors ( $v_k$  and  $u_k$  respectively)



# Finding Eigenvectors from Eigenvalues

- Given an eigenvalue  $\lambda$ , form the matrix  $\hat{A} - \lambda I$
- If  $\hat{A}$  is symmetric, then  $\hat{A} - \lambda I$  is symmetric
- $\hat{A} - \lambda I$  has (at least) a rank 1 null space (from the definition of eigenvalues)
  - Proof:  $(\hat{A} - \lambda I)v = \hat{A}v - \lambda v = \lambda v - \lambda v = 0$  for eigenvector  $v$
- Solve the linear system  $(\hat{A} - \lambda I)v = 0$  to find the eigenvector  $v$

# Condition Number of Eigenproblems

- The condition number for finding an eigenvalue is different than the condition number for solving a linear system
- The condition number for finding an eigenvalue/eigenvector pair is  $\frac{1}{v_L^T v_R}$  where  $v_L$  and  $v_R$  are the normalized left and right eigenvectors
- Symmetric (Hermitian) matrices have identical left and right eigenvectors; so,  $v_L^T v_R = 1$  and the condition number is 1

# Characteristic Polynomial

- The eigenvalue problem is typically written as  $\hat{A}v = \lambda v$
- So,  $(\hat{A} - \lambda I)v = 0$  implying that  $\hat{A} - \lambda I$  is singular
- Setting  $\det(\hat{A} - \lambda I) = 0$  leads to a degree  $n$  characteristic polynomial equation in  $\lambda$  (for a size  $n \times n$  matrix  $\hat{A}$ )
- Finding the roots of this polynomial equation can be quite difficult
  - Recall how difficult it was to find roots for a mere cubic equation
- Finding roots for degree  $n > 3$  polynomials is undesirable!

# Similarity Transforms

- Similarity transforms,  $T^{-1}\hat{A}T$ , preserve the eigenstructure
  - Proof:  $T^{-1}\hat{A}Tv = \lambda v$  or  $\hat{A}(Tv) = \lambda(Tv)$  still has eigenvalue  $\lambda$  with a modified eigenvector  $T^{-1}v$
- When  $\hat{A}$  is real and symmetric (complex and Hermitian), there exists an orthogonal (unitary)  $T$  that makes  $T^{-1}\hat{A}T$  diagonal with real eigenvalues
  - In fact,  $T = V$  for  $A^T A = V\Sigma^T \Sigma V^T$  and  $T = U$  for  $AA^T = U\Sigma\Sigma^T U^T$
- Other interesting facts:
  - When  $\hat{A}$  has distinct eigenvalues, a  $T$  exists that makes  $T^{-1}\hat{A}T$  diagonal
  - Schur form: For any (square) matrix, a unitary  $T$  exists to make  $T^{-1}\hat{A}T$  upper triangular with eigenvalues on the diagonal
  - Jordan form: Any (square) matrix can be put into a form with eigenvalues on the diagonal and nonzero off-diagonal elements only occurring on the band above the diagonal and only for defective eigenvalues (repeated eigenvalues that don't possess a full set of eigenvectors)



# Similarity Transforms via QR Iteration

- Starting with  $\hat{A}^0 = \hat{A}$
- Compute the factorization  $\hat{A}^q = Q^q R^q$  with orthogonal  $Q^q$
- Then, define  $\hat{A}^{q+1} = R^q Q^q$ 
  - Note:  $R^q Q^q = (Q^q)^T Q^q R^q Q^q = (Q^q)^T \hat{A}^q Q^q$  is a similarity transform of  $\hat{A}^q$
- When the eigenvalues are distinct,  $\hat{A}^q$  converges to a triangular matrix
- When  $\hat{A}$  is symmetric,  $\hat{A}^q$  converges to a diagonal matrix

# Power Method

- Computes the largest eigenvalue (great for rank 1 updates)
- Start with a  $c^0 \neq 0$ , and iterate  $c^{q+1} = \hat{A}c^q$

Proof:

- Write  $c^0$  as a linear combination of eigenvectors:  $c^0 = \sum_k \alpha_k v_k$
- Then,  $c^q = \hat{A}^q c^0 = \sum_k \alpha_k \hat{A}^q v_k = \sum_k \alpha_k \lambda_k^q v_k = \lambda_{max}^q \sum_k \alpha_k \left( \frac{\lambda_k}{\lambda_{max}} \right)^q v_k$
- As  $q \rightarrow \infty$ ,  $\left( \frac{\lambda_k}{\lambda_{max}} \right)^q \rightarrow 0$  for  $\lambda_k < \lambda_{max}$ ; so,  $c^q \rightarrow \lambda_{max}^q \alpha_{max} v_{max}$
- As  $q \rightarrow \infty$ ,  $\frac{(c^{q+1})_i}{(c^q)_i} \rightarrow \frac{\lambda_{max}^{q+1} \alpha_{max} (v_{max})_i}{\lambda_{max}^q \alpha_{max} (v_{max})_i} = \lambda_{max}$  for every component  $i$  of  $c$
- Deflation removes an eigenvalue from  $\hat{A}$  by subtracting off its rank 1 update
  - The deflated  $A^T A - \sigma_k^2 v_k v_k^T$  or  $AA^T - \sigma_k^2 u_k u_k^T$  can then be used to compute the next largest eigenvalue (repeatedly)

# Power Method (a few notes)

- If  $c^0 = \sum_k \alpha_k v_k$  happens to have  $\alpha_{max} = 0$ , the method might fail (but roundoff errors can help)
- $c^q$  needs to be periodically renormalized to stop it from growing too large
- When  $c^0$  and  $\hat{A}$  are real-valued, cannot obtain complex numbers
- When the largest eigenvalue is repeated, one needs to determine a basis for the multiple associated eigenvectors
- Inverse Iteration can be used to find the smallest eigenvalue of  $\hat{A}$ , since the largest eigenvalue of  $\hat{A}^{-1}$  is the smallest eigenvalue of  $\hat{A}$ 
  - $c^{q+1} = \hat{A}^{-1} c^q$  is updated by solving  $\hat{A} c^{q+1} = c^q$  to find  $c^{q+1}$
  - Useful for finding the condition number  $\frac{\sigma_{max}}{\sigma_{min}}$

# Unit 12